

Business Case: Delhivery - Feature Engineering

Name : Asit Nayak

```
# importing the required modules and packages

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import re
from scipy.stats import norm,zscore,boxcox,probplot
from scipy.stats import ttest_ind,ttest_rel,mannwhitneyu,wilcoxon
from scipy.stats import shapiro,levene,kstest,anderson
import statsmodels.api as sm
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler , MinMaxScaler , OneHotEncoder
import warnings
warnings.filterwarnings('ignore')
```

```
# pd_reading the data
delhivery_data = pd.read_csv('/content/sample_data/delhivery_data.csv')
```

```
# setting the option of displaying all the columns
pd.set_option('display.max_columns', 50)
```

```
# making a deep copy for backup
dd = delhivery_data.copy()
dd.head()
```

▼ Exploration of data

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	od_end_time
training		2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:00.000000

dd.shape

(65532, 24)	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:00.000000
-------------	----------------------------	--	---------	-------------------------	--------------	----------------------------	--------------	-------------------------------	----------------------------	----------------------------

dd.columns

Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type', 'trip_uuid', 'source_center', 'source_name', 'destination_center', 'destination_name', 'od_start_time', 'od_end_time', 'start_scan_to_end_scan', 'is_cutoff_factor', 'cutoff_factor', 'cutoff_timestamp', 'actual_distance_to_destination', 'actual_time', 'osrm_time', 'osrm_distance', 'factor', 'segment_actual_time', 'segment_osrm_time', 'segment_osrm_distance', 'segment_factor'], dtype='object')	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:00.000000
--	----------------------------	--	---------	-------------------------	--------------	----------------------------	--------------	-------------------------------	----------------------------	----------------------------

dd.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 65532 entries, 0 to 65531
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0    data                                65532 non-null  object
1    trip_creation_time                  65532 non-null  object
2    route_schedule_uuid                65532 non-null  object
3    route_type                         65532 non-null  object
4    trip_uuid                          65532 non-null  object
5    source_center                      65531 non-null  object
6    source_name                        65391 non-null  object
7    destination_center                 65531 non-null  object
8    destination_name                   65435 non-null  object
9    od_start_time                     65531 non-null  object
10   od_end_time                        65531 non-null  object
11   start_scan_to_end_scan             65531 non-null  float64
12   is_cutoff                          65531 non-null  object
13   cutoff_factor                     65531 non-null  float64
14   cutoff_timestamp                   65531 non-null  object
15   actual_distance_to_destination     65531 non-null  float64
16   actual_time                        65531 non-null  float64
17   osrm_time                         65531 non-null  float64
18   osrm_distance                     65531 non-null  float64
19   factor                            65531 non-null  float64
20   segment_actual_time                65531 non-null  float64
21   segment_osrm_time                  65531 non-null  float64
22   segment_osrm_distance              65531 non-null  float64
23   segment_factor                     65531 non-null  float64
dtypes: float64(11), object(13)
memory usage: 12.0+ MB
```

Statistical summary

```
dd.describe().T
```

	count	mean	std	min	25%	50%	75%	max
start_scan_to_end_scan	65531.0	937.287833	1011.057079	20.000000	163.000000	449.000000	1528.000000	4535.000000
cutoff_factor	65531.0	226.931727	335.333076	9.000000	22.000000	66.000000	286.000000	1927.000000
actual_distance_to_destination	65531.0	228.076713	335.542702	9.000267	23.373592	66.116690	286.191741	1927.447705
actual_time	65531.0	406.070470	580.964622	9.000000	52.000000	132.000000	500.000000	4532.000000
osrm_time	65531.0	208.683722	300.031194	6.000000	27.000000	64.000000	251.500000	1686.000000
osrm_distance	65531.0	277.767490	410.247141	9.072900	29.930400	78.607500	334.891250	2326.199100
factor	65531.0	2.106788	1.497202	0.203125	1.607866	1.857143	2.216409	77.387097
segment_actual_time	65531.0	36.024828	51.271562	-211.000000	20.000000	29.000000	40.000000	2464.000000
segment_osrm_time	65531.0	18.467061	14.268028	0.000000	11.000000	17.000000	22.000000	997.000000
segment_osrm_distance	65531.0	22.768476	17.152820	0.000000	12.065200	23.506000	27.787450	1367.592800
segment_factor	65531.0	2.219145	5.444168	-23.444444	1.350000	1.684211	2.250000	574.250000

```
dd.describe(include=object).T
```

	count	unique	top	freq
data	65532	2	training	47390
trip_creation_time	65532	6692	2018-10-01 05:04:55.268931	101
route_schedule_uuid	65532	1341	thanos::sroute:dca6268f-741a-4d1a-b1b0-aab1309...	764
route_type	65532	2	FTL	44991
trip_uuid	65532	6692	trip-153837029526866991	101
source_center	65531	1381	IND000000ACB	10405
source_name	65391	1374	Gurgaon_Bilaspur_HB (Haryana)	10405
destination_center	65531	1343	IND000000ACB	6844
destination_name	65435	1332	Gurgaon_Bilaspur_HB (Haryana)	6844
od_start_time	65531	11891	2018-09-21 18:37:09.322207	81
od_end_time	65531	11891	2018-09-24 09:59:15.691618	81
is_cutoff	65531	2	True	53755
cutoff_timestamp	65531	50501	2018-09-24 05:19:20	19

▼ Duplication detection

```
dd[dd.duplicated()]

data trip_creation_time route_schedule_uuid route_type trip_uuid source_center source_name destination_center destination_name od_start_time od_end_time start_sca
```

▼ Insights

```
dd.isna().any()
```

0

data	False
trip_creation_time	False
route_schedule_uuid	False
route_type	False
trip_uuid	False
source_center	True
source_name	True
destination_center	True
destination_name	True
od_start_time	True
od_end_time	True
start_scan_to_end_scan	True
is_cutoff	True
cutoff_factor	True
cutoff_timestamp	True
actual_distance_to_destination	True
actual_time	True
osrm_time	True
osrm_distance	True
factor	True
segment_actual_time	True
segment_osrm_time	True
segment_osrm_distance	True
segment_factor	True

dtype: bool

```
dd.isna().sum()
```

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	1
source_name	141
destination_center	1
destination_name	97
od_start_time	1
od_end_time	1
start_scan_to_end_scan	1
is_cutoff	1
cutoff_factor	1
cutoff_timestamp	1
actual_distance_to_destination	1
actual_time	1
osrm_time	1
osrm_distance	1
factor	1
segment_actual_time	1
segment_osrm_time	1
segment_osrm_distance	1
segment_factor	1

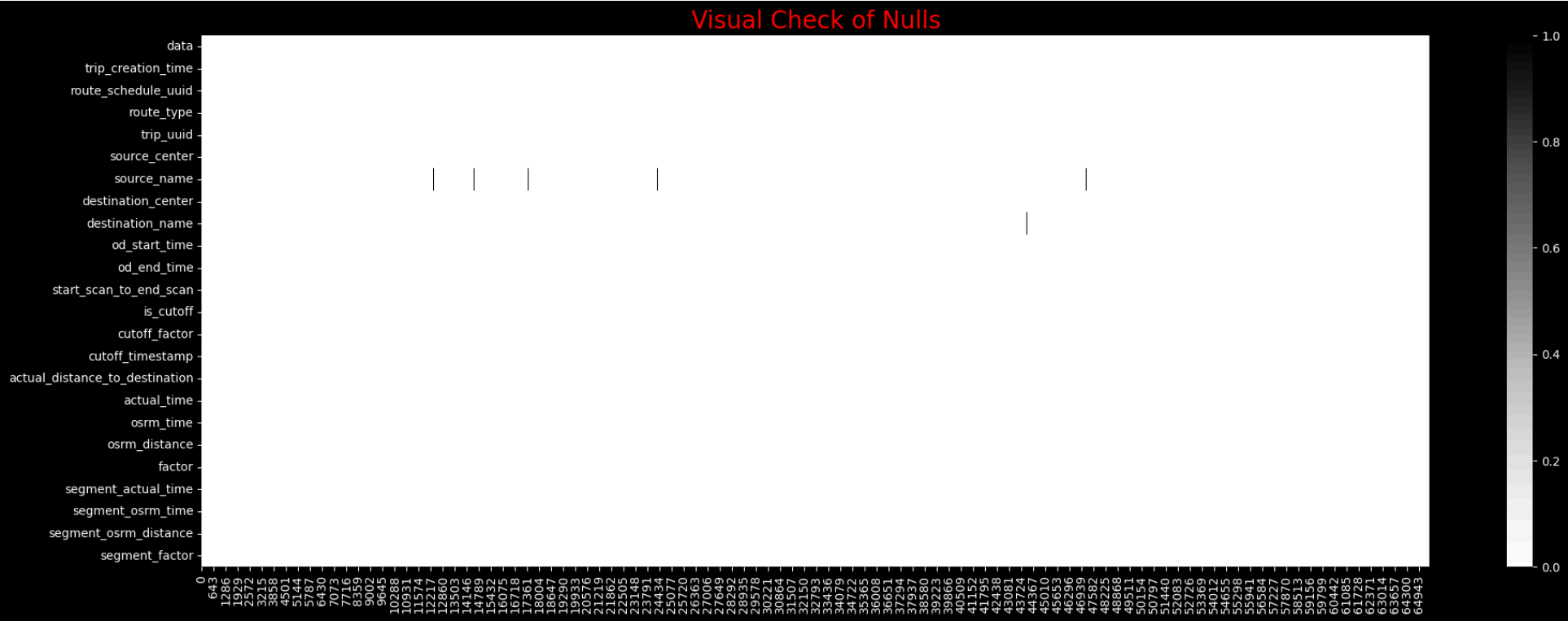
dtype: int64

```
def missing_data(df):
    total_missing_df = df.isnull().sum().sort_values(ascending=False)
    percent_missing_df = (df.isnull().sum()/df.isna().count()*100).sort_values(ascending=False) # ----> /len(dd)
    missing_data_df = pd.concat([total_missing_df, percent_missing_df], axis=1, keys=['Total', 'Percent'])
    return missing_data_df

missing_pct = missing_data(dd)
missing_pct[missing_pct['Total']>0]
```

	Total	Percent
source_name	141	0.215162
destination_name	97	0.148019
destination_center	1	0.001526
source_center	1	0.001526
start_scan_to_end_scan	1	0.001526
od_end_time	1	0.001526
od_start_time	1	0.001526
is_cutoff	1	0.001526
actual_time	1	0.001526
osrm_time	1	0.001526
osrm_distance	1	0.001526
factor	1	0.001526
segment_actual_time	1	0.001526
cutoff_factor	1	0.001526
cutoff_timestamp	1	0.001526
actual_distance_to_destination	1	0.001526
segment_osrm_distance	1	0.001526
segment_osrm_time	1	0.001526

```
plt.figure(figsize=(23,8))
plt.style.use('dark_background')
sns.heatmap(dd.isnull().T,cmap='Greys')
plt.title('Visual Check of Nulls',fontsize=20,color='r')
plt.show()
```



```
dd[dd['destination_center'].isna()]
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	od_end_time
65531	training	2018-09-20 20:44:57.424801	thanos::route:ed5b80be- 7abf-424d-b8cd- d81556a...	FTL	trip- 153747629742455165	NaN	NaN	NaN	NaN	NaN	

```
dd = dd.dropna(subset=['destination_center'])
```

```
def missing_data(df):  
    total_missing_df = df.isnull().sum().sort_values(ascending =False)  
    percent_missing_df = (df.isnull().sum()/df.isna().count()*100).sort_values(ascending=False) # ----> /len(dd)  
    missing_data_df = pd.concat([total_missing_df, percent_missing_df], axis=1, keys=['Total', 'Percent'])
```

```
return missing_data_df

missing_pct = missing_data(dd)
missing_pct[missing_pct['Total']>0]
```

	Total	Percent	
source_name	140	0.213639	
destination_name	96	0.146496	

```
# Dropping unknown fields

unknown_fields = ['is_cutoff', 'cutoff_factor', 'cutoff_timestamp', 'factor', 'segment_factor']
dd = dd.drop(columns = unknown_fields)
```

```
dd.sample()
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	od_end_t:
9024	test	2018-09-28 19:07:27.563134	thanos::sroute:6e01eb09- 11ca-4a88-839f- 2fe0836...	Carting	trip- 153816164756290062	IND400072AAB	Mumbai Hub (Maharashtra)	IND421302AAR	Mumbai_East_I_21 (Maharashtra)	2018-09-28 19:07:27.563134	2018-09- 20:49:49.894:

```
dd.shape
```

(65531, 19)

```
#checking the unique values for columns
for _ in dd.columns:
    print()
    print(f'Total Unique Values in {_} column are :- {dd[_].nunique()}')
    print(f'Unique Values in {_} column are :-\n {dd[_].unique() if dd[_].nunique() <= 10 else str(dd[_].unique()[:15]) + " and more"}')
    print()
    print('-'*120)
```

Total Unique Values in actual_time column are :- 2876
Unique Values in actual_time column are :-
[14. 24. 40. 62. 68. 15. 44. 65. 76. 102. 38. 117. 141. 183.
46.] and more

Total Unique Values in osrm_time column are :- 1466
Unique Values in osrm_time column are :-
[11. 20. 28. 40. 44. 17. 29. 39. 45. 24. 41. 68. 80. 95. 16.] and more

Total Unique Values in osrm_distance column are :- 63833
Unique Values in osrm_distance column are :-
[11.9653 21.7243 32.5395 45.562 54.2181 12.1171 21.289 35.8252
47.19 53.2334 26.8622 54.4326 89.668 108.3939 129.3519] and more

Total Unique Values in segment_actual_time column are :- 526
Unique Values in segment_actual_time column are :-
[14. 10. 16. 21. 6. 15. 28. 26. 38. 37. 41. 23. 46. 30. 50.] and more

Total Unique Values in segment_osrm_time column are :- 167
Unique Values in segment_osrm_time column are :-
[11. 9. 7. 12. 5. 6. 10. 24. 27. 26. 14. 15. 30. 18. 38.] and more

Total Unique Values in segment_osrm_distance column are :- 57990
Unique Values in segment_osrm_distance column are :-
[11.9653 9.759 10.8152 13.0224 3.9153 12.1171 9.1719 14.5362 11.3648
6.0434 26.8622 30.1058 35.2353 17.2476 20.958] and more

Changing the datatype of columns

dd.sample()

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	
53020	training	2018-09-18 10:27:42.872583	thanos::sroute:b9622248- 4dfb-4964-8f1d-ec5f83f...	FTL	trip- 153726646287232095	IND000000ACB	Gurgaon_Bilaspur_HB (Haryana)	IND302014AAA	Jaipur_Hub (Rajasthan)	2018-09-18 10:27:42.872583	22

dd.dtypes

0

data	object
trip_creation_time	object
route_schedule_uuid	object
route_type	object
trip_uuid	object
source_center	object
source_name	object
destination_center	object
destination_name	object
od_start_time	object
od_end_time	object
start_scan_to_end_scan	float64
actual_distance_to_destination	float64
actual_time	float64
osrm_time	float64
osrm_distance	float64
segment_actual_time	float64
segment_osrm_time	float64
segment_osrm_distance	float64

dtype: object

```
# Converting the datatypes to category for columns like data and route_type as they only have 2 values.
dd['data'] = dd['data'].astype('category')
dd['route_type'] = dd['route_type'].astype('category')

# Converting time columns to datetime format
datetime_cols = ['trip_creation_time', 'od_start_time', 'od_end_time']
for _ in datetime_cols:
    dd[_] = pd.to_datetime(dd[_])
```

dd.info()

```
<class 'pandas.core.frame.DataFrame'>
Index: 65531 entries, 0 to 65530
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0    data                                65531 non-null  category
1    trip_creation_time                  65531 non-null  datetime64[ns]
2    route_schedule_uuid                65531 non-null  object
```

```

3 route_type                65531 non-null category
4 trip_uuid                 65531 non-null object
5 source_center             65531 non-null object
6 source_name              65391 non-null object
7 destination_center       65531 non-null object
8 destination_name         65435 non-null object
9 od_start_time            65531 non-null datetime64[ns]
10 od_end_time              65531 non-null datetime64[ns]
11 start_scan_to_end_scan   65531 non-null float64
12 actual_distance_to_destination 65531 non-null float64
13 actual_time              65531 non-null float64
14 osrm_time                65531 non-null float64
15 osrm_distance            65531 non-null float64
16 segment_actual_time      65531 non-null float64
17 segment_osrm_time        65531 non-null float64
18 segment_osrm_distance    65531 non-null float64
dtypes: category(2), datetime64[ns](3), float64(8), object(6)
memory usage: 9.1+ MB

```

```

float_cols = []
for _ in dd.columns:
    if dd[_].dtype=='float64':
        float_cols.append(_)
float_cols

```

```

['start_scan_to_end_scan',
 'actual_distance_to_destination',
 'actual_time',
 'osrm_time',
 'osrm_distance',
 'segment_actual_time',
 'segment_osrm_time',
 'segment_osrm_distance']

```

```

# reducing the float64 to float32 to save memory
for _ in float_cols:
    dd[_] = dd[_].astype('float32')

```

```
dd.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 65531 entries, 0 to 65530
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   data                  65531 non-null  category
1   trip_creation_time    65531 non-null  datetime64[ns]
2   route_schedule_uuid   65531 non-null  object
3   route_type            65531 non-null  category
4   trip_uuid             65531 non-null  object
5   source_center         65531 non-null  object
6   source_name           65391 non-null  object
7   destination_center    65531 non-null  object
8   destination_name      65435 non-null  object
9   od_start_time         65531 non-null  datetime64[ns]
10  od_end_time           65531 non-null  datetime64[ns]
11  start_scan_to_end_scan 65531 non-null  float32
12  actual_distance_to_destination 65531 non-null  float32
13  actual_time            65531 non-null  float32

```

```

14 osrm_time                65531 non-null float32
15 osrm_distance            65531 non-null float32
16 segment_actual_time      65531 non-null float32
17 segment_osrm_time        65531 non-null float32
18 segment_osrm_distance    65531 non-null float32
dtypes: category(2), datetime64[ns](3), float32(8), object(6)
memory usage: 7.1+ MB

```

▼ Insights:

- Earlier the dataset was using 25.6+ MB of memory but now it has been reduced to 14.6 + MB. Around 40.63 % reduction in the memory usage.

```

# Time period of data
dd['trip_creation_time'].max(), dd['trip_creation_time'].min() , dd['trip_creation_time'].max()-dd['trip_creation_time'].min()

```

```

(Timestamp('2018-10-03 23:59:42.701692'),
Timestamp('2018-09-12 00:05:40.333071'),
Timedelta('21 days 23:54:02.368621'))

```

```

# Time period of data
dd['od_start_time'].max(), dd['od_start_time'].min(), dd['od_start_time'].max() - dd['od_start_time'].min()

```

```

(Timestamp('2018-10-06 00:08:33.866586'),
Timestamp('2018-09-12 00:05:40.333071'),
Timedelta('24 days 00:02:53.533515'))

```

```

# Time period of data
dd['od_end_time'].max(), dd['od_end_time'].min(), dd['od_end_time'].max() - dd['od_end_time'].min()

```

```

(Timestamp('2018-10-06 19:46:59.723686'),
Timestamp('2018-09-12 00:50:10.814399'),
Timedelta('24 days 18:56:48.909287'))

```

```

data_time_frame = dd['od_end_time'].max() - dd['trip_creation_time'].min()
data_time_frame

```

```

Timedelta('24 days 19:41:19.390615')

```

▼ Null Treatment:

Replace null values in 'source_name' and 'destination_name' columns with 'unknown' through scikit imputation

```

columns_to_impute = ['source_name', 'destination_name']
imputer = SimpleImputer(strategy='constant', fill_value='unknown')
dd[columns_to_impute] = imputer.fit_transform(dd[columns_to_impute])

```

but 'unknown' transactions will be more and to be omitted while analyzing ... Hence no use of imputing....

```

dd[(dd.source_name.isna()) & (dd.destination_name.isna())]

```

```
data trip_creation_time route_schedule_uuid route_type trip_uuid source_center source_name destination_center destination_name od_start_time od_end_time start_sca
```

dd[dd.source_name.isna()]

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	od_end
112	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	153786558437756691	IND342902A1B	NaN	IND302014AAA	Jaipur_Hub (Rajasthan)	2018-09-26 06:58:08.054001	2018-09-26 15:54:14.2
113	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	153786558437756691	IND342902A1B	NaN	IND302014AAA	Jaipur_Hub (Rajasthan)	2018-09-26 06:58:08.054001	2018-09-26 15:54:14.2
114	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	153786558437756691	IND342902A1B	NaN	IND302014AAA	Jaipur_Hub (Rajasthan)	2018-09-26 06:58:08.054001	2018-09-26 15:54:14.2
115	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	153786558437756691	IND342902A1B	NaN	IND302014AAA	Jaipur_Hub (Rajasthan)	2018-09-26 06:58:08.054001	2018-09-26 15:54:14.2
116	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	153786558437756691	IND342902A1B	NaN	IND302014AAA	Jaipur_Hub (Rajasthan)	2018-09-26 06:58:08.054001	2018-09-26 15:54:14.2
...	...	...	...	...	...	...	...	...	...	...	...
63304	test	2018-09-28 00:26:28.863690	thanos::sroute:ae08e84c-e6e4-411a-8c2f-27af4cf...	FTL	153809438886343536	IND282002AAD	NaN	IND281004AAA	Mathura_DC (Uttar Pradesh)	2018-09-28 00:26:28.863690	2018-09-28 04:03:17.6
63305	test	2018-09-28 00:26:28.863690	thanos::sroute:ae08e84c-e6e4-411a-8c2f-27af4cf...	FTL	153809438886343536	IND282002AAD	NaN	IND281004AAA	Mathura_DC (Uttar Pradesh)	2018-09-28 00:26:28.863690	2018-09-28 04:03:17.6
63336	training	2018-09-25 19:54:19.853543	thanos::sroute:da7a341a-ba3d-436d-b4b5-607d1a0...	FTL	153790525985329256	IND509103AAC	NaN	IND518002AAA	Kurnool_AbbasNgr_I (Andhra Pradesh)	2018-09-26 04:39:01.457853	2018-09-26 07:38:00.7
63337	training	2018-09-25 19:54:19.853543	thanos::sroute:da7a341a-ba3d-436d-b4b5-607d1a0...	FTL	153790525985329256	IND509103AAC	NaN	IND518002AAA	Kurnool_AbbasNgr_I (Andhra Pradesh)	2018-09-26 04:39:01.457853	2018-09-26 07:38:00.7
63338	training	2018-09-25 19:54:19.853543	thanos::sroute:da7a341a-ba3d-436d-b4b5-607d1a0...	FTL	153790525985329256	IND509103AAC	NaN	IND518002AAA	Kurnool_AbbasNgr_I (Andhra Pradesh)	2018-09-26 04:39:01.457853	2018-09-26 07:38:00.7
140 rows x 19 columns											

dd[dd.destination_name.isna()]

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	
110	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	trip-153786558437756691	IND342601AAA	Piparcity_BsstdDPP_D (Rajasthan)	IND342902A1B	NaN	2018-09-26 05:04:49.254901	0
111	training	2018-09-25 08:53:04.377810	thanos::sroute:4460a38d-ab9b-484e-bd4e-f4201d0...	FTL	trip-153786558437756691	IND342601AAA	Piparcity_BsstdDPP_D (Rajasthan)	IND342902A1B	NaN	2018-09-26 05:04:49.254901	0
982	test	2018-10-01 20:56:18.155260	thanos::sroute:d0ebdacc-d09b-47d3-be77-c9c4a05...	FTL	trip-153842737815495661	IND573103AAA	Arsikere_HsnRdDPP_D (Karnataka)	IND577116AAA	NaN	2018-10-02 01:22:21.450243	0
983	test	2018-10-01 20:56:18.155260	thanos::sroute:d0ebdacc-d09b-47d3-be77-c9c4a05...	FTL	trip-153842737815495661	IND573103AAA	Arsikere_HsnRdDPP_D (Karnataka)	IND577116AAA	NaN	2018-10-02 01:22:21.450243	0
4882	training	2018-09-24 07:18:06.087341	thanos::sroute:2f43f11e-d3ba-4590-9355-82928e1...	FTL	trip-153777348608709328	IND202001AAB	Aligarh_KhirByps_I (Uttar Pradesh)	IND282002AAD	NaN	2018-09-24 15:02:13.760270	1
...	...	...	...	...	...	...	...	...	...	...	...
53427	test	2018-09-27 12:50:12.657545	thanos::sroute:633a10ac-43ff-4a98-8f79-9bbf5cb...	FTL	trip-153805261265714576	IND202001AAB	Aligarh_KhirByps_I (Uttar Pradesh)	IND282002AAD	NaN	2018-09-27 21:02:31.142260	0
53428	test	2018-09-27 12:50:12.657545	thanos::sroute:633a10ac-43ff-4a98-8f79-9bbf5cb...	FTL	trip-153805261265714576	IND202001AAB	Aligarh_KhirByps_I (Uttar Pradesh)	IND282002AAD	NaN	2018-09-27 21:02:31.142260	0
56623	test	2018-10-03 00:41:44.419686	thanos::sroute:9e556660-5a55-42dc-b668-acacd7f...	FTL	trip-153852730441947714	IND328001AAA	Dholpur_GtRoad_D (Rajasthan)	IND282002AAD	NaN	2018-10-03 05:34:43.503014	0
56624	test	2018-10-03 00:41:44.419686	thanos::sroute:9e556660-5a55-42dc-b668-acacd7f...	FTL	trip-153852730441947714	IND328001AAA	Dholpur_GtRoad_D (Rajasthan)	IND282002AAD	NaN	2018-10-03 05:34:43.503014	0

```
ddd = ddd.copy()

56623 test 00:41:44.419686 9e55-42dc-b668- FTL 153852730441947714 IND328001AAA (Rajasthan) IND282002AAD NaN 05:34:43.503014 0

missing_source_name = ddd.loc[ddd['source_name'].isnull(), 'source_center'].unique()
missing_source_name

array(['IND342902A1B', 'IND577116AAA', 'IND282002AAD', 'IND465333A1B',
      'IND841301AAC', 'IND509103AAC', 'IND126116AAA'], dtype=object)

missing_destination_name = ddd.loc[ddd['destination_name'].isnull(), 'destination_center'].unique()
missing_destination_name

array(['IND342902A1B', 'IND577116AAA', 'IND282002AAD', 'IND465333A1B',
      'IND841301AAC', 'IND505326AAB', 'IND852118A1B', 'IND126116AAA',
      'IND509103AAC', 'IND221005A1A', 'IND250002AAC'], dtype=object)

# checking if element of one np.array isin another np.array

#np.all(df.loc[df['source_name'].isnull(), 'source_center'].isin(missing_destination_name))

np.in1d(missing_source_name, missing_destination_name).all()
```

```
np.True_
```

```
for _ in missing_source_name:
    unique_source_name = ddd.loc[ddd['source_center'] == _, 'source_name'].unique()
    if pd.isna(unique_source_name):
        print("Source Center :", _, "-" * 5, "Source Name :", 'NA')
    else :
        print("Source Center :", _, "-" * 5, "Source Name :", unique_source_name)
```

```
Source Center : IND342902A1B ----- Source Name : NA
Source Center : IND577116AAA ----- Source Name : NA
Source Center : IND282002AAD ----- Source Name : NA
Source Center : IND465333A1B ----- Source Name : NA
Source Center : IND841301AAC ----- Source Name : NA
Source Center : IND509103AAC ----- Source Name : NA
Source Center : IND126116AAA ----- Source Name : NA
```

```
for _ in missing_destination_name:
    unique_destination_name = ddd.loc[ddd['destination_center'] == _, 'destination_name'].unique()
    if pd.isna(unique_destination_name):
        print("Destination Center :", _, "-" * 5, "Destination Name :", 'NA')
    else :
        print("Destination Center :", _, "-" * 5, "Destination Name :", unique_destination_name)
```

```
Destination Center : IND342902A1B ----- Destination Name : NA
Destination Center : IND577116AAA ----- Destination Name : NA
Destination Center : IND282002AAD ----- Destination Name : NA
Destination Center : IND465333A1B ----- Destination Name : NA
Destination Center : IND841301AAC ----- Destination Name : NA
Destination Center : IND505326AAB ----- Destination Name : NA
Destination Center : IND852118A1B ----- Destination Name : NA
Destination Center : IND126116AAA ----- Destination Name : NA
Destination Center : IND509103AAC ----- Destination Name : NA
Destination Center : IND221005A1A ----- Destination Name : NA
Destination Center : IND250002AAC ----- Destination Name : NA
```

```
count = 1
for i in missing_destination_name:
    ddd.loc[ddd['destination_center'] == i, 'destination_name'] = ddd.loc[ddd['destination_center'] == i,
                                                                           'destination_name'].replace(np.nan, f'location_{count}')
    count += 1
```

```
d = {}
for i in missing_source_name:
    d[i] = ddd.loc[ddd['destination_center'] == i, 'destination_name'].unique()
for idx, val in d.items():
    if len(val) == 0:
        d[idx] = [f'location_{count}']
        count += 1
d2 = {}
for idx, val in d.items():
    d2[idx] = val[0]
for i, v in d2.items():
    print(i, v)
```

```
IND342902A1B location_1
IND577116AAA location_2
IND282002AAD location_3
IND465333A1B location_4
IND841301AAC location_5
IND509103AAC location_9
IND126116AAA location_8
```

```
for i in missing_source_name:
    ddd.loc[ddd['source_center'] == i, 'source_name'] = ddd.loc[ddd['source_center'] == i, 'source_name'].replace(np.nan, d2[i])
```

```
ddd.source_name.value_counts()
```

	count
source_name	
Gurgaon_Bilaspur_HB (Haryana)	10405
Bangalore_Nelmngla_H (Karnataka)	4488
Bhiwandi_Mankoli_HB (Maharashtra)	4258
Pune_Tathawde_H (Maharashtra)	1849
Hyderabad_Shamshbd_H (Telangana)	1711
...	...
Mumbai_Teipal_I (Maharashtra)	1
Sonepur_Sabalpur_D (Bihar)	1
Chikhli_KKndrDPP_D (Maharashtra)	1
Addanki_Oilmilrd_D (Andhra Pradesh)	1
Allahabad_Mirapati_L (Uttar Pradesh)	1

1381 rows x 1 columns

dtype: int64

```
ddd.destination_name.value_counts()
```

```

                                count
destination_name
Gurgaon_Bilaspur_HB (Haryana)    6844
Bangalore_Nelmngla_H (Karnataka) 4510
Bhiwandi_Mankoli_HB (Maharashtra) 2680
Hyderabad_Shamshbd_H (Telangana) 2602

len(delhivery_data)

65532      ...      ...
      Udupi (Karnataka)      1

df = dd.dropna()

df.isna().sum().any()

np.False
      Koppa_Sangetha_D (Karnataka)      1

df.info()

Out[6]:
Out[6]: In[6]: pandas.core.frame.DataFrame
Index: 65295 entries, 0 to 65530
Data columns (total 19 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   data                                  65295 non-null  category
 1   trip_creation_time                    65295 non-null  datetime64[ns]
 2   route_schedule_uuid                  65295 non-null  object
 3   route_type                            65295 non-null  category
 4   trip_uuid                             65295 non-null  object
 5   source_center                         65295 non-null  object
 6   source_name                           65295 non-null  object
 7   destination_center                    65295 non-null  object
 8   destination_name                       65295 non-null  object
 9   od_start_time                         65295 non-null  datetime64[ns]
10  od_end_time                           65295 non-null  datetime64[ns]
11  start_scan_to_end_scan                 65295 non-null  float32
12  actual_distance_to_destination          65295 non-null  float32
13  actual_time                             65295 non-null  float32
14  osrm_time                              65295 non-null  float32
15  osrm_distance                          65295 non-null  float32
16  segment_actual_time                    65295 non-null  float32
17  segment_osrm_time                      65295 non-null  float32
18  segment_osrm_distance                  65295 non-null  float32
dtypes: category(2), datetime64[ns](3), float32(8), object(6)
memory usage: 7.1+ MB

```

Insights:

- Only two fields have a tiny fraction of missing values, less than 0.05% of the whole dataset.

- Since we have plenty of data to work with, we're choosing to just get rid of the missing values instead of trying to guess them using methods like using the average or most common value.
- I'm dropping the missing values to keep things simple and not mess up how the features are spread out. But if a lot more data was missing, we could have used other methods like guessing based on what's there or using the most common values.

✧ Exploratory data analysis

```
cp = ['gray','red','dimgrey','tomato','dimgray','orangered','k','salmon','gray','red','dimgrey','tomato','dimgray','orangered','k','salmon']
```

```
df.sample()
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	
51842	training	2018-09-18 04:32:21.900842	thanos::sroute:14c55592- ba2e-4f72-820c- 3a22334...	FTL	153724514190061713	IND562132AAA	Bangalore_Nelmgla_H (Karnataka)	IND462022AAA	Bhopal_Trnsport_H (Madhya Pradesh)	2018-09-18 04:32:21.900842	2

```
plt.figure(figsize=(24,6))
plt.suptitle('Pie Percentage distribution',fontsize=13,fontfamily='serif',fontweight='bold',backgroundcolor=cp[-1],color='w')

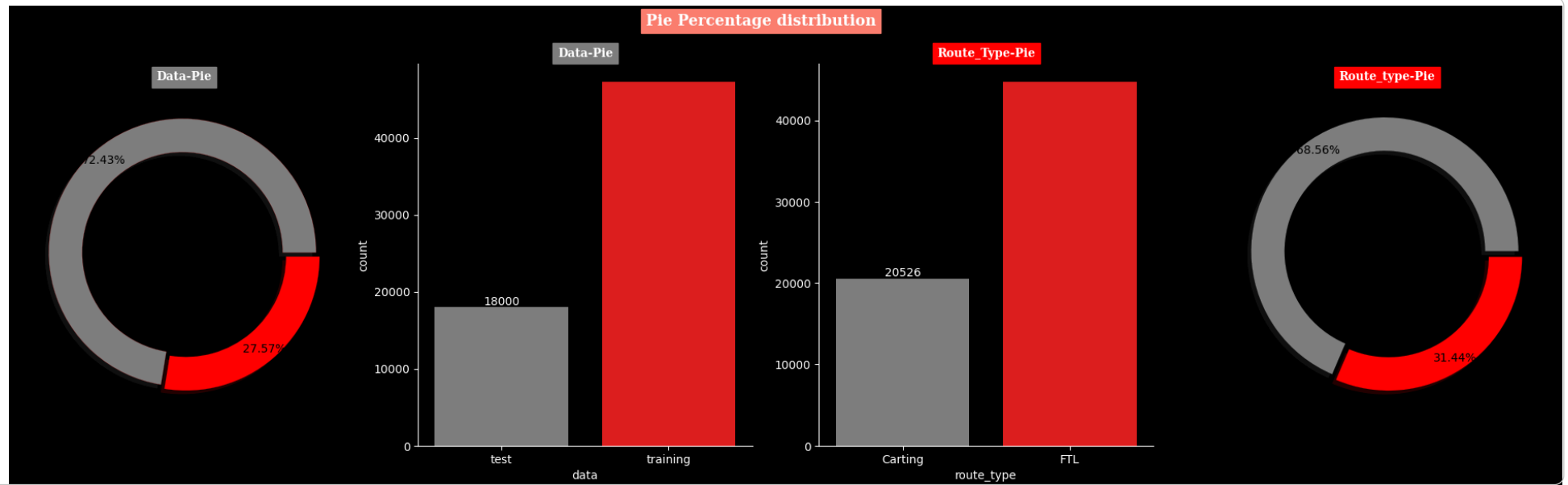
plt.subplot(141)
plt.pie(df['data'].value_counts(), labels=df['data'].value_counts().index, colors=cp,counterclock=True , explode=(0.02,0.02) , autopct='%.2f%', pctdistance=0.905,
        textprops={'color':'k','fontsize':10} , shadow=True, radius=1,wedgeprops=dict(edgecolor='r',linewidth=0.1,width=0.25))
plt.title('Data-Pie',fontsize=10,fontfamily='serif',fontweight='bold',backgroundcolor=cp[0],color='w')

plt.subplot(142)
a = sns.barplot(x=df['data'].value_counts().index, y=df['data'].value_counts(), palette=cp)
a.bar_label(a.containers[0], label_type='edge',fmt='%d')
plt.title('Data-Pie',fontsize=10,fontfamily='serif',fontweight='bold',backgroundcolor=cp[0],color='w')

plt.subplot(143)
b = sns.barplot(x=df['route_type'].value_counts().index, y=df['route_type'].value_counts(), palette=cp)
b.bar_label(b.containers[0], label_type='edge',fmt='%d')
plt.title('Route-Type-Pie',fontsize=10,fontfamily='serif',fontweight='bold',backgroundcolor=cp[1],color='w')

plt.subplot(144)
plt.pie(df['route_type'].value_counts(), labels=df['route_type'].value_counts().index, colors=cp,counterclock=True , explode=(0.03,0.02) , autopct='%.2f%', pctdistance=0.9
        textprops={'color':'k','fontsize':10} , shadow=True, radius=1,wedgeprops=dict(edgecolor='k',linewidth=0.1,width=0.25))
plt.title('Route-type-Pie',fontsize=10,fontfamily='serif',fontweight='bold',backgroundcolor=cp[1],color='w')

sns.despine()
plt.show()
```



```
plt.figure(figsize=(24,17))
plt.style.use('default')

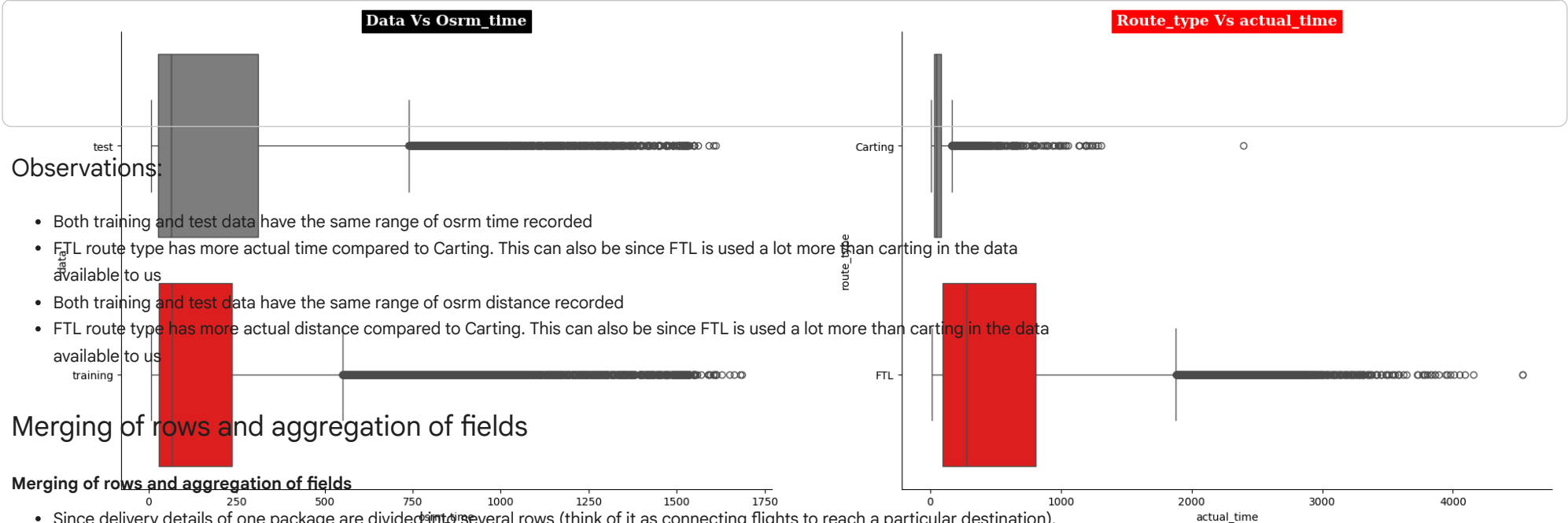
plt.subplot(221)
sns.boxplot(data=df,y='data',x='osrm_time',palette=cp)
plt.title('Data Vs Osrn_time',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='k',color='w')

plt.subplot(222)
sns.boxplot(data=df,y='route_type',x='actual_time',palette=cp)
plt.title('Route_type Vs actual_time',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='r',color='w')

plt.subplot(223)
sns.boxplot(data=df,y='data',x='osrm_distance',palette=cp)
plt.title('Data Vs Osrn_distance',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='r',color='w')

plt.subplot(224)
sns.boxplot(data=df,y='route_type',x='actual_distance_to_destination',palette=cp)
plt.title('Route_type Vs actual_distance_to_destination',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='k',color='w')
sns.despine()
plt.show()
```





✓ Merging of rows and aggregation of fields

Merging of rows and aggregation of fields

- Since delivery details of one package are divided into several rows (think of it as connecting flights to reach a particular destination). Now think about how we should treat their fields if we combine these rows? What aggregation would make sense if we merge. What would happen to the numeric fields if we merge them?

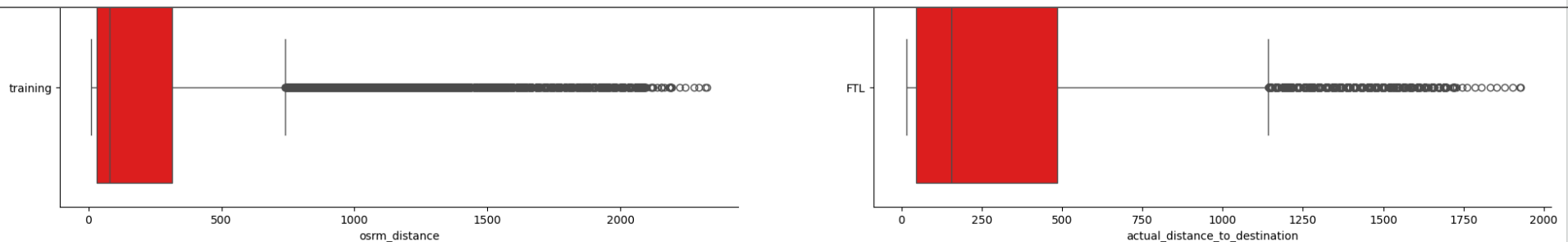
```
# Grouping by segment
# Creating a unique identifier for each segment of a trip

segment_cols = ['segment_actual_time', 'segment_osrm_distance', 'segment_osrm_time']

df['segment_key'] = df['trip_uuid'] + '+' + df['source_center'] + '+' + df['destination_center']

for col in segment_cols:
    df[col + '_sum'] = df.groupby('segment_key')[col].cumsum()

df[['segment_key', 'segment_actual_time', 'segment_actual_time_sum', 'segment_osrm_distance', 'segment_osrm_distance_sum', 'segment_osrm_time', 'segment_osrm_time_sum']]
```



	segment_key	segment_actual_time	segment_actual_time_sum	segment_osrm_distance	segment_osrm_distance_sum	segment_osrm_time	segment
0	trip-153741093647649320+IND388121AAA+IND388620AAB	14.0	14.0	11.965300	11.965300	11.0	
1	trip-153741093647649320+IND388121AAA+IND388620AAB	10.0	24.0	9.759000	21.724300	9.0	
2	trip-153741093647649320+IND388121AAA+IND388620AAB	16.0	40.0	10.815200	32.539497	7.0	

```
# Aggregating at segment level & Creating a dictionary for aggregation at segment level
```

```
segment_dict = {
    'trip_uuid' : 'first',
    'data': 'first',
    'route_type': 'first',
    'trip_creation_time': 'first',
    'source_name': 'first',
    'destination_name': 'last',
    'od_start_time': 'first',
    'od_end_time': 'last',
    'start_scan_to_end_scan': 'first',
    'actual_distance_to_destination': 'last',
    'actual_time': 'last',
    'osrm_time': 'last',
    'osrm_distance': 'last',
    'segment_actual_time' : 'sum',
    'segment_osrm_time' : 'sum',
    'segment_osrm_distance' : 'sum',
    'segment_actual_time_sum': 'last',
    'segment_osrm_time_sum': 'last',
    'segment_osrm_distance_sum': 'last',
}
```

```
# Grouping by segment_key and aggregating
segment_agg_data = df.groupby('segment_key').agg(segment_dict).reset_index()
segment_agg_data = segment_agg_data.sort_values(by=['segment_key', 'od_end_time'])
segment_agg_data
```

	segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time
0	trip-153671074033284934+IND395009AAA+IND395023AAD	trip-153671074033284934	training	Carting	2018-09-12 00:05:40.333071	Surat_Central_D_12 (Gujarat)	Surat_Central_I_4 (Gujarat)	2018-09-12 02:31:39.246238	2018-09-12 05:16:28.58
1	trip-153671074033284934+IND395023AAD+IND395004AAB	trip-153671074033284934	training	Carting	2018-09-12 00:05:40.333071	Surat_Central_I_4 (Gujarat)	Surat_Central_D_3 (Gujarat)	2018-09-12 00:05:40.333071	2018-09-12 02:01:41.63
2	trip-153671079956500691+IND110024AAA+IND110014AAA	trip-153671079956500691	training	Carting	2018-09-12 00:06:39.565253	Delhi_Lajpat_IP (Delhi)	Delhi_Bhogal (Delhi)	2018-09-12 00:06:39.565253	2018-09-12 00:55:59.56
3	trip-153671110078355292+IND121004AAB+IND121001AAA	trip-153671110078355292	training	Carting	2018-09-12 00:11:40.783923	FBD_Balabhgarh_DPC (Haryana)	Faridabad (Haryana)	2018-09-12 00:11:40.783923	2018-09-12 00:50:10.81
4	trip-153671127071609610+IND585104AAA+IND586128AAA	trip-153671127071609610	training	Carting	2018-09-12 00:14:30.716368	Gulbarga_Nehrugn_I (Karnataka)	Sindagi_KalyanNg_D (Karnataka)	2018-09-12 00:14:30.716368	2018-09-12 03:21:35.02
...	...	...	...	...	...	...	...	...	...

Understanding

- 11821 The rows have been merged based on the unique segment key, which is a combination of trip_uuid, source_center, and destination_center.
- 11822 The aggregated dataset reflects the total values for each segment of the trip.
- 11823
- 11824

Feature engineering

```
# 1. Calculating time difference between od_start_time and od_end_time
segment_agg_data['od_total_time']=(segment_agg_data['od_end_time'] - segment_agg_data['od_start_time'])
segment_agg_data['od_time_diff_hour'] = (segment_agg_data['od_total_time']).dt.total_seconds()/3600
segment_agg_data
```

od_end_t

2018-0
:16:28.58

2018-0
01:41.63

2018-0
55:59.56

2018-0
50:10.81

2018-0
21:35.02

2018-1
41:25.40

2018-1
59:51.62

2018-1
51:27.07

2018-1
46:09.16

2018-1
58:40.72

segment_agg_data.sample()

	segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time
9453	trip-153816778954903006+IND515801AAA+IND515865AAB	trip-153816778954903006	test	FTL	2018-09-28 20:49:49.549248	Guntakal_Verpatem_D (Andhra Pradesh)	Rayadurgam_RailGate_D (Andhra Pradesh)	2018-09-29 01:07:32.853084	2018-09-29 05:14:06.045595

```
# de = segment_agg_data.drop(columns=['source_city','source_state','source_place','destination_place','destination_city','destination_state'])
de = segment_agg_data.copy()
```

de.sample()

	segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time
1216	trip-153688636480084238+IND206001AAA+IND000000ACB	trip-153688636480084238	training	FTL	2018-09-14 00:52:44.801246	Etawah_MhraChng_D (Uttar Pradesh)	Gurgaon_Bilaspur_HB (Haryana)	2018-09-15 04:16:42.014339	2018-09-15 12:39:24.496754

```
# using regex pattern to seperate the city,place,state
def extract_info(name):
    pattern = r'^(?P<city>[^\s_]+)?(?P<place>[^\(\)]*)\s?(?P<state>[A-Za-z\s&+])\)$'
    match = re.match(pattern, name)
    if match:
        city = match.group('city').strip()
        place = match.group('place').strip() if match.group('place') else city
        state = match.group('state').strip()
        return city, place, state
    else:
        return None, None, None
```

```
de[['source_city', 'source_place', 'source_state']] = de['source_name'].apply(lambda x: pd.Series(extract_info(x)))
```

```
de[['destination_city', 'destination_place', 'destination_state']] = de['destination_name'].apply(lambda x: pd.Series(extract_info(x)))
```

de

	segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time
0	trip-153671074033284934+IND395009AAA+IND395023AAD	trip-153671074033284934	training	Carting	2018-09-12 00:05:40.333071	Surat_Central_D_12 (Gujarat)	Surat_Central_I_4 (Gujarat)	2018-09-12 02:31:39.246238	2018-09-12 05:16:28.56
1	trip-153671074033284934+IND395023AAD+IND395004AAB	trip-153671074033284934	training	Carting	2018-09-12 00:05:40.333071	Surat_Central_I_4 (Gujarat)	Surat_Central_D_3 (Gujarat)	2018-09-12 00:05:40.333071	2018-09-12 02:01:41.63
2	trip-153671079956500691+IND110024AAA+IND110014AAA	trip-153671079956500691	training	Carting	2018-09-12 00:06:39.565253	Delhi_Lajpat_IP (Delhi)	Delhi_Bhogal (Delhi)	2018-09-12 00:06:39.565253	2018-09-12 00:55:59.56
3	trip-153671110078355292+IND121004AAB+IND121001AAA	trip-153671110078355292	training	Carting	2018-09-12 00:11:40.783923	FBD_Balabhgarh_DPC (Haryana)	Faridabad (Haryana)	2018-09-12 00:11:40.783923	2018-09-12 00:50:10.81
4	trip-153671127071609610+IND585104AAA+IND586128AAA	trip-153671127071609610	training	Carting	2018-09-12 00:14:30.716368	Gulbarga_Nehrugn_I (Karnataka)	Sindagi_KalyanNg_D (Karnataka)	2018-09-12 00:14:30.716368	2018-09-12 03:21:35.02
...	...	...	...	...	...	...	...	...	...
11820	trip-153861095625827784+IND160102AAD+IND160002AAC	trip-153861095625827784	test	Carting	2018-10-03 23:55:56.258533	Chandigarh_Raiprkl_C (Chandigarh)	Chandigarh_Mehmdpur_H (Punjab)	2018-10-04 04:56:10.943956	2018-10-04 06:41:25.40
11821	trip-153861106442901555+IND208006AAA+IND209304AAA	trip-153861106442901555	test	Carting	2018-10-03 23:57:44.429324	Kanpur_GovndNgr_DC (Uttar Pradesh)	Kanpur_Central_H_6 (Uttar Pradesh)	2018-10-04 02:51:27.075797	2018-10-04 06:59:51.62
11822	trip-153861106442901555+IND209304AAA+IND208006AAA	trip-153861106442901555	test	Carting	2018-10-03 23:57:44.429324	Kanpur_Central_H_6 (Uttar Pradesh)	Kanpur_GovndNgr_DC (Uttar Pradesh)	2018-10-03 23:57:44.429324	2018-10-03 02:51:27.07
11823	trip-153861118270144424+IND583119AAA+IND583101AAA	trip-153861118270144424	test	FTL	2018-10-03 23:59:42.701692	Sandur_WrdN1DPP_D (Karnataka)	Bellary_Dc (Karnataka)	2018-10-04 03:58:40.726547	2018-10-04 08:46:09.16
11824	trip-153861118270144424+IND583201AAA+IND583119AAA	trip-153861118270144424	test	FTL	2018-10-03 23:59:42.701692	Hospet (Karnataka)	Sandur_WrdN1DPP_D (Karnataka)	2018-10-04 02:51:44.712656	2018-10-04 03:58:40.72

11825 rows x 28 columns

```
de[(de['source_place']=='') | (de['destination_place']=='')]
```

	segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time
3	trip-153671110078355292+IND121004AAB+IND121001AAA	trip-153671110078355292	training	Carting	2018-09-12 00:11:40.783923	FBD_Balabhgarh_DPC (Haryana)	Faridabad (Haryana)	2018-09-12 00:11:40.783923	2018-09-12 00:50:10.8145
23	trip-153671320807895983+IND121004AAB+IND121102AAA	trip-153671320807895983	training	Carting	2018-09-12 00:46:48.079257	FBD_Balabhgarh_DPC (Haryana)	Palwal (Haryana)	2018-09-12 00:46:48.079257	2018-09-12 01:53:32.4714
24	trip-153671320807895983+IND121102AAA+IND121004AAB	trip-153671320807895983	training	Carting	2018-09-12 00:46:48.079257	Palwal (Haryana)	FBD_Balabhgarh_DPC (Haryana)	2018-09-12 01:53:32.471405	2018-09-12 05:07:14.2325
107	trip-153672034455004313+IND583201AAA+IND583227AAA	trip-153672034455004313	training	Carting	2018-09-12 02:45:44.550269	Hospet (Karnataka)	Gangavathi_PhrmPlza_D (Karnataka)	2018-09-12 02:45:44.550269	2018-09-12 04:10:21.5312
109	trip-153672034455004313+IND583231AAA+IND583201AAA	trip-153672034455004313	training	Carting	2018-09-12 02:45:44.550269	Koppal_Banikatt_D (Karnataka)	Hospet (Karnataka)	2018-09-12 05:44:07.793902	2018-09-12 07:03:38.2155
...	...	...	...	...	...	...	...	...	...
11499	trip-153854990058036233+IND121004AAB+IND121102AAA	trip-153854990058036233	test	Carting	2018-10-03 06:58:20.580610	FBD_Balabhgarh_DPC (Haryana)	Palwal (Haryana)	2018-10-03 06:58:20.580610	2018-10-03 08:37:27.0806
11523	trip-153855685628554406+IND636016AAA+IND637211AAA	trip-153855685628554406	test	Carting	2018-10-03 08:54:16.285786	Salem (Tamil Nadu)	Tirchngode_Mhdhvpur_D (Tamil Nadu)	2018-10-03 10:09:13.079448	2018-10-03 11:57:10.4096
11614	trip-153858917190862399+IND244901AAB+IND243002AAA	trip-153858917190862399	test	FTL	2018-10-03 17:52:51.908885	Rampur_RoshnBgh_I (Uttar Pradesh)	Bareilly (Uttar Pradesh)	2018-10-03 17:52:51.908885	2018-10-03 02:29:05.3226
11702	trip-153860096739851507+IND132103AAB+IND132001AAA	trip-153860096739851507	test	FTL	2018-10-03 21:09:27.398767	Panipat_PC (Haryana)	Karnal (Haryana)	2018-10-04 03:07:19.274811	2018-10-04 04:05:36.4825
11824	trip-153861118270144424+IND583201AAA+IND583119AAA	trip-153861118270144424	test	FTL	2018-10-03 23:59:42.701692	Hospet (Karnataka)	Sandur_WrdN1DPP_D (Karnataka)	2018-10-04 02:51:44.712656	2018-10-04 03:58:40.7265

372 rows x 28 columns

```
de.loc[de['source_place']=='', 'source_place']=de['source_city']
de.loc[de['destination_place']=='', 'destination_place']=de['destination_city']
```

```
de[de.source_place.isna()]
```

segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time	start_scan_to_end_scan	actual_distance_to_destination
-------------	-----------	------	------------	--------------------	-------------	------------------	---------------	-------------	------------------------	--------------------------------

```
de.isna().sum()
```

	0
segment_key	0
trip_uuid	0
data	0
route_type	0
trip_creation_time	0
source_name	0
destination_name	0
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_actual_time_sum	0
segment_osrm_time_sum	0
segment_osrm_distance_sum	0
od_total_time	0
od_time_diff_hour	0
source_city	0
source_place	0
source_state	0
destination_city	0
destination_place	0
destination_state	0

dtype: int64

```
de.loc[de.source_city=='Bangalore','source_city']='Bengaluru'  
de.loc[de.destination_city=='Bangalore','destination_city']='Bengaluru'
```

```
np.set_printoptions(threshold=np.inf)
```

```
de['source_city'].unique()
```

```
'Dumraon' 'NeemKaThana' 'Jhunjhunu' 'Khetri' 'Ambah'
```

```
Delhivery, Recommendations, Shipment, Invoice, Amazon,  
'Bhadrachalam', 'Manuguru', 'Dhanbad', 'Chandpur', 'Aonla',
```

```
de['source_place'].unique()
```

```
(Mumbai,DL) (Bokaro,DL) (Gurgaon,DL) (Husnagar,DL)
```

```
de['source_state'].unique()
```

```
array(['Gujarat', 'Delhi', 'Haryana', 'Karnataka', 'Madhya Pradesh',  
      'Assam', 'Telangana', 'Tamil Nadu', 'Punjab', 'Maharashtra',  
      'Rajasthan', 'West Bengal', 'Andhra Pradesh', 'Goa',  
      'Uttar Pradesh', 'Jharkhand', 'Pondicherry', 'Orissa',  
      'Himachal Pradesh', 'Meghalaya', 'Arunachal Pradesh', 'Bihar',  
      'Uttarakhand', 'Kerala', 'Chandigarh', 'Chhattisgarh',  
      'Jammu & Kashmir', 'Dadra and Nagar Haveli', 'Tripura', 'Mizoram',  
      'Nagaland'], dtype=object)
```

```
de['source_state'].value_counts().to_frame().T.style.background_gradient(cmap='Reds')
```

source_state	Maharashtra	Karnataka	Haryana	Tamil Nadu	Uttar Pradesh	Telangana	Andhra Pradesh	Gujarat	West Bengal	Rajasthan	Punjab	Bihar	Delhi	Kerala	Madhya Pradesh	Assam	Uttarakhand	Jharkha
count	1612	1586	942	895	853	646	644	621	580	534	469	454	399	358	337	186	173	

```
state_counts = de['source_state'].value_counts().to_frame().reset_index()
state_counts.columns = ['State', 'Count']
```

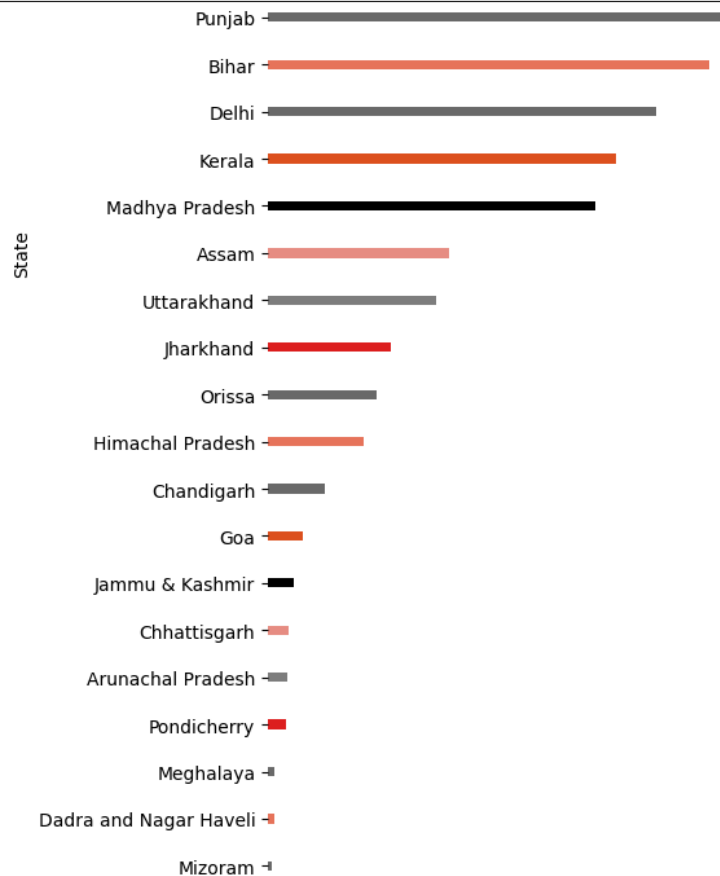
```
plt.figure(figsize=(15,12))
a = sns.barplot(y='State', x='Count', data=state_counts,palette=cp,width=0.2)
a.bar_label(a.containers[0], label_type='edge')
plt.xticks([])
plt.ylabel('State')
plt.xlabel('Count')
plt.title('Number of Deliveries from Source State',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor='r',color='w')
plt.tight_layout()
sns.despine(bottom=True,left=True)
plt.show()
```

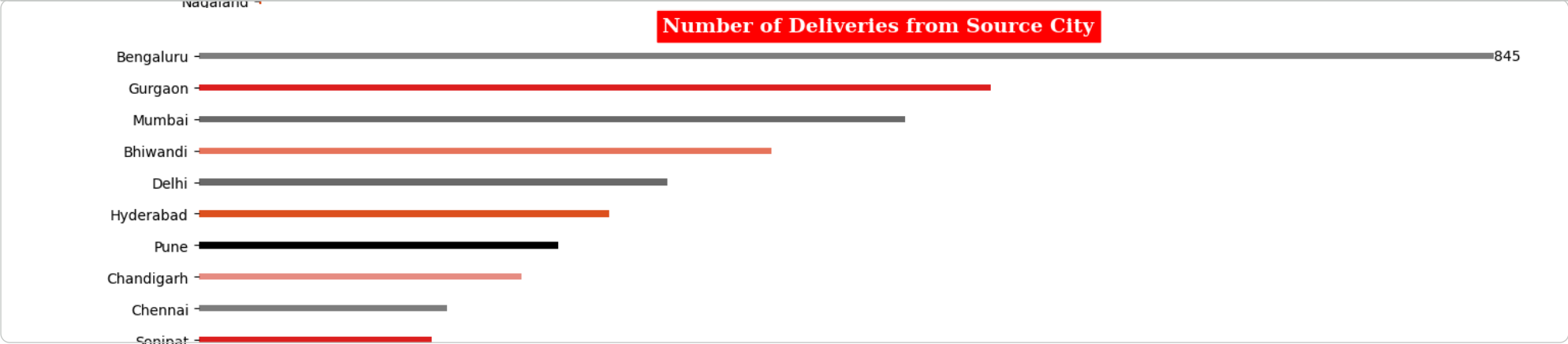


Number of Deliveries from Source State

```
city_counts = de['source_city'].value_counts().to_frame().reset_index()[ :30]
city_counts.columns = ['City', 'Count']
```

```
plt.figure(figsize=(15,10))
a = sns.barplot(y='City', x='Count', data=city_counts,palette=cp,width=0.2)
a.bar_label(a.containers[0], label_type='edge')
plt.xticks([])
plt.ylabel('City')
plt.xlabel('Count')
plt.title('Number of Deliveries from Source City',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='r',color='w')
plt.tight_layout()
sns.despine(bottom=True,left=True)
plt.show()
```





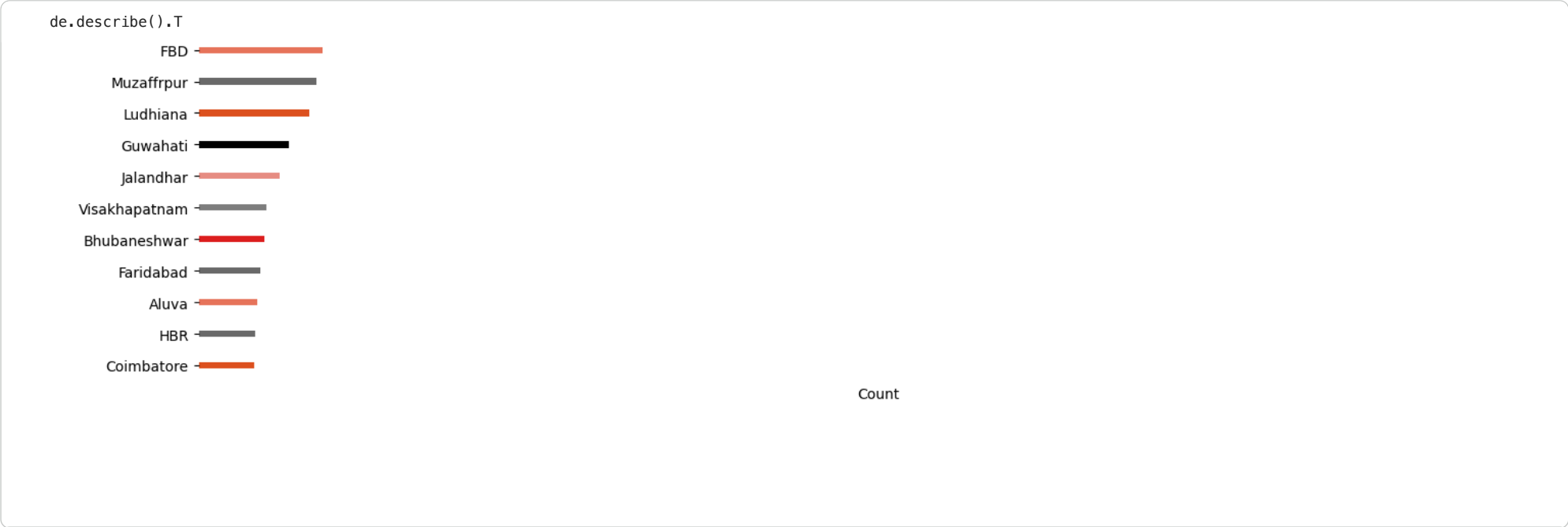
Insights:

Source State contributors

- Maharashtra, Karnataka ,Tamil Nadu, Haryana, and Uttar Pradesh are the top contributors where maximum bookings are recorded in this month indicating significant engagement.

Source City contributors

- Cities like Bengaluru, Gurgaon, Mumbai, Bhiwandi, Delhi, Hyderabad where the major no.of booking are recorded.



	count	mean	min	25%	50%	75%	max	std	
trip_creation_time	11825	2018-09-22 12:11:40.598120960	2018-09-12 00:05:40.333071	2018-09-17 02:34:17.063773952	2018-09-22 01:57:30.163540992	2018-09-27 19:08:42.276193024	2018-10-03 23:59:42.701692	NaN	
od_start_time	11825	2018-09-22 16:02:15.316135680	2018-09-12 00:05:40.333071	2018-09-17 06:15:35.408922112	2018-09-22 05:39:49.930206976	2018-09-27 22:10:21.209164032	2018-10-06 00:08:33.866586	NaN	
od_end_time	11825	2018-09-22 21:00:51.149106688	2018-09-12 00:50:10.814399	2018-09-17 11:15:54.588134912	2018-09-22 10:24:51.003270912	2018-09-28 02:15:20.030569984	2018-10-06 19:46:59.723686	NaN	
start_scan_to_end_scan	11825.0	297.975739	20.0	90.0	153.0	311.0	4535.0	433.11499	
actual_distance_to_destination	11825.0	92.792709	9.003577	21.677084	35.242565	65.73407	1927.447754	206.866745	
actual_time	11825.0	200.696823	9.0	51.0	84.0	171.0	4532.0	379.023407	
osrm_time	11825.0	91.038986	6.0	25.0	39.0	73.0	1686.0	182.983994	
osrm_distance	11825.0	115.265846	9.0729	27.7304	43.738899	86.1399	2326.199219	251.00206	
segment_actual_time	11825.0	198.864441	9.0	50.0	84.0	169.0	4504.0	375.508667	
segment_osrm_time	11825.0	101.919067	6.0	25.0	42.0	81.0	1938.0	212.381927	
segment_osrm_distance	11825.0	125.680771	9.0729	28.4884	45.82	92.825798	2640.924805	281.610596	
segment_actual_time_sum	11825.0	198.864441	9.0	50.0	84.0	169.0	4504.0	375.508667	
segment_osrm_time_sum	11825.0	101.919067	6.0	25.0	42.0	81.0	1938.0	212.381927	
segment_osrm_distance_sum	11825.0	125.680771	9.0729	28.4884	45.82	92.825798	2640.924805	281.610596	
trip_duration	11825	0 days	0 days	0 days	0 days	0 days	3 days	0 days	

de.describe(include='object').T

	count	unique	top	freq	
segment_key	11825	11825	trip-153861118270144424+IND583201AAA+IND583119AAA	1	
trip_uuid	11825	6681	trip-153710494321650505	8	
source_name	11825	1371	Gurgaon_Bilaspur_HB (Haryana)	485	
destination_name	11825	1330	Gurgaon_Bilaspur_HB (Haryana)	452	
source_city	11825	1150	Bengaluru	845	
source_place	11825	1145	Bilaspur_HB	485	
source_state	11825	31	Maharashtra	1612	
destination_city	11825	1144	Bengaluru	799	
destination_place	11825	1109	Bilaspur_HB	452	
destination_state	11825	32	Karnataka	1595	

de['destination_city'].unique()

Shirata', 'Kathagiri', 'Tumkur', 'Karnat', 'Guduvada',
 'Machilipatnam', 'Nandigama', 'Gobicheti', 'Sathyamangalam',
 'Mudigere', 'Adoor', 'Kottarakkara', 'Pthnmthitt', 'Punalur',
 'Morgram', 'Chaksu', 'Lonavala', 'Lalru', 'Vallior', 'Nagarcoil',
 'Kolar', 'Mulbagal', 'Chintamani', 'Malur', 'Bangarapet',
 'Giridih', 'Rusera', 'Manjhaul', 'Udgir', 'Islampure',
 'Dharmnagar', 'Agartala', 'Jassur', 'Dalhousie', 'Chamba',
 'Pathankot', 'Changlang', 'Piparcity', 'Manthuka', 'Karungaply',
 'Chopan', 'Gonda', 'Kozhikode', 'Udupi', 'Sirsa', 'Mandapeta',
 'Kakinada', 'Paithan', 'Georai', 'Shevgaon', 'Beed', 'Shadnagar',
 'Vikarabad', 'Tandur', 'Dhone', 'Padrauna', 'Khargram',
 'Rampurhat', 'Kallachi', 'Vadakara', 'Arsikere', 'Areacode',
 'Badarpur', 'Bikramgang', 'Bellmpalli', 'Mancherial', 'Paranpur',
 'SikandraRao', 'Dhrmsthala', 'Tonk', 'Parbhani', 'Mandsaur',
 'Sitamau', 'Hansi', 'Pen', 'Pataudi', 'Kolasib', 'LakhimpurN',
 'Champhai', 'Aizawl', 'Khanapur', 'Bakhtiarpur', 'Faridpur',
 'Gangarpur', 'Kulithalai', 'Ariyalur', 'Jayamkondan', 'Lalpet',
 'Shamshabad', 'Farrukhbad', 'Shahabad', 'Shahjhnpur', 'Aliganj',
 'Jalalabad', 'Nandurbar', 'Shindkheda', 'Shahada', 'Samsi',
 'Rajkot', 'Wankaner', 'BariSadri', 'Sangareddy', 'Pilibanga',
 'Vishakhapatnam', 'Bobbili', 'Hoshangabad', 'Maihar', 'Barauni',
 'Kalpakkam', 'Dharwad', 'Chandauli', 'Gahmar', 'Jowai', 'Siwan',
 'Karkala', 'Nohar', 'ChrkhiDdri', 'Chimkurthy', 'Draksharamam',
 'Chhibramau', 'Ratia', 'Fatehabad', 'Sujanpur', 'Avinashi',
 'Nanjangud', 'Chamarjngr', 'Gundlupet', 'Kollegala',
 'Tirumakudalu', 'Krishnarajngr', 'Gonikoppal', 'HDKote',
 'Moranhat', 'MirzapurWB', 'Berhampore', 'Kothanalloor',
 'Tikamgarh', 'Arimbur', 'Cochin', 'Nandikotkr', 'Atmakur',
 'Maharajapuram', 'Eluru', 'Budhana', 'Kaman', 'Dola', 'Shrirampur',
 'Sihora', 'Sonari', 'Panagarh', 'Digboi', 'Paralakhemundi',
 'Kokrajhar', 'Bongaigaon', 'Barpeta', 'Ghatkesar', 'Betnoti',
 'Baripada', 'Balrampur', 'Agra', 'Bhiwani', 'Teok', 'Mariani',
 'Malappuram', 'Nipani', 'Kaliyaganj', 'Panaji', 'Basti', 'Anjar',
 'Manbazar', 'Khatra', 'Bardhaman', 'Printhlmna', 'Kasganj',
 'DalsinghSarai', 'Karukachal', 'Thirthahalli', 'Tarkeshwar',
 'Khanakul', 'Tulsipur', 'Balasore', 'Chhatarpur', 'Kunnathund',
 'Jath', 'Chikodi', 'Khamgaon', 'Itahar', 'Jalore', 'Bhinmal',
 'Sirohi', 'Jagtial', 'Moodbidri', 'Khurja', 'RampuraPhul',
 'Suratgarh', 'Arambag', 'Manglamaro', 'Keshiary', 'Contai',
 'Thuraiyur', 'Mandla', 'Tallada', 'Morena', 'Cjb', 'Perundurai',
 'Simlapal', 'Sundargarh', 'Rajgangpur', 'Khandela', 'Sakri',
 'Shirpur', 'Sinnar', 'Khalilabad', 'DoomDooma', 'Namsai', 'Roing',
 'Tezu', 'Bhadaur', 'Barnala', 'Nagpur', 'Islampur', 'Bongaon',
 'MarwarJn', 'Falna', 'Sumerpur', 'Malout', 'Bhilad', 'Fazilka',
 'Azamgarh', 'Badlapur', 'Shahganj', 'Phusro', 'Palampur', 'Kangra',
 'DehraGopipur', 'Palakollu', 'Atapadi', 'Chanchal', 'Manikchak',
 'Tenkasi', 'Ambasamdrn', 'Sangamner', 'Jamjodhpur', 'ChandroknaRD',
 'MughalSarai', 'Aunrihar', 'Amalapuram', 'Chikmagalur', 'Sonepur',
 'Sambalpur', 'Bargarh', 'Balangir', 'Baraut', 'Helencha',
 'Sangola', 'Bilaspur', 'Khatauli', 'Varanasi', 'Kozhenchery',
 'Ambegaon', 'Raiganj', 'Baruipur', 'Anakapalle', 'Narsiptnm',
 'Dabhoi', 'Gomoh', 'Cuttack', 'Taranagar', 'Luxettipet', 'Basta',
 'Koyilandy', 'Tirurangadi', 'Khetri', 'Barhi', 'Gujiliam', 'Daman',
 'Uthangarai', 'Saraimeer', 'Tangi', 'Tadipatri', 'Proddatur',
 'Badvel', 'Cuddapah', 'Durg', 'Thachnttukra', 'Theni',
 'Chasechumunll', 'dtuac-shiest')

```
de['destination_place'].unique()
```

```

    'Mrtktgr_D', 'PreetDPP_D', 'Tejpat_C', 'Kotwatin_D', 'Peroke_n',
    'Udupi', 'AnprnDPP_D', 'Mainrd_D', 'SantaNGR_D', 'Bazarrd_D',
    'JalnaRd_D', 'SnthiNGR_D', 'RmNyrDPP_D', 'BawLiDPPP_D', 'BoiDPP_D',
    'SrifoDPP_D', 'ZamQuatr_D', 'Mandodi_D', 'HsnRdDPP_D',
    'Puthalam_D', 'Konapara_D', 'Dehrird_D', 'MrktYrd_DC',
    'BasthDPP_D', 'Hitech_D', 'Nijgan_D', 'East_I_20', 'JydevNGR_D',
    'Belngdi_D', 'Chukhndi_D', 'South_D_12', 'Patel Nagar',
    'Rjndrngr_D', 'Old City', 'YashDPP_D', 'Aliganj', 'DelRdDPP_D',
    'HSR_Layout_PC', 'BljiMrkt_D', 'Diakkawn_D', 'SashPhkn_D',
    'AwmpiVng_D', 'HunthrVg_I', 'ShjnprRD_D', 'Whitefld_L',
    'KhdimDPP_D', 'AnnaNGR_D', 'Kmraingr_D', 'Chithbrm_D',
    'PriyrNGR_D', 'Lnrgguda_D', 'Pnchlght_D', 'KDRoad_D', 'NavdaCln_D',
    'KaimgnjRD_D', 'farukngr_D', 'Aurangabad', 'DhuleRd_D',
    'KakaCplx_D', 'Nandrbar_D', 'MdhsnDPP_D', 'RatuaDPP_D',
    'PedakRd_P', 'JivanDPP_D', 'BhmprDPP_D', 'Pothredy_D',
    'Vishakhapatnam', 'ColegRd_D', 'MngalDPP_D', 'Sadras_D',
    'ShubsNGR_D', 'Wardnor4_D', 'Ldthlabh_D', 'HnmntNgr_D',
    'MarketRd_D', 'PuranDPP_D', 'MSRCLgRd_D', 'Anaipeta_D',
    'Bhainpura_D', 'AgrohDPP_D', 'SchdvDPP_D', 'BhunaDPP_D',
    'SainkSCL_D', 'Valluvar_D', 'RPRoad_D', 'BrDbbleRd_D', 'Swamylyt_D',
    'NarsipuraRd_D', 'Narasipr_D', 'JublieRD_D', 'Thomas_D',
    'BegurRD_D', 'TiloiDPP_D', 'VidyaDPP_D', 'Majoor_D', 'MndiRoad_D',
    'Veluthur_D', 'Kdvantra_D', 'UmarDPP_D', 'IndranGr_D',
    'KrsnakcL_D', 'Agraharm_DC', 'ThanaDPP_D', 'ZuariNgr_IP',
    'Ponda_Dc', 'Margao_Dc', 'Kadugodi_D', 'ChainDPP_D', 'DavkharRd_D',
    'KtnRdDPP_D', 'JyotiNgr_D', 'Ward14_D', 'PigonDPP_D', 'Chpaguri_D',
    'ShivBari_D', 'Central_D_9', 'Potheri', 'Chandigarh', 'MhliadPP_D',
    'KalikDPP_D', 'PBRDDPP_D', 'Idgah_P', 'Chtrpuza_D', 'Munduprm_D',
    'Pettah_D', 'AkkolRD_D', 'GovndNgr_DC', 'Goa', 'GndhiNgr_D',
    'Kalyan West _Dc', 'SohnaRd_D', 'KoralDPP_D', 'ITICollg_L',
    'Bankura_D', 'EmsPnmbi_D', 'BnkrGate_D', 'MnimlaRd_D',
    'NadthiCx_D', 'Sangetha_D', 'Naraynpr_D', 'Ramnagar_D',
    'MiraRoad_M', 'Ganeshwr_D', 'Pringla_D', 'Shivprasad_D',
    'WamanDPP_D', 'NatunDPP_D', 'FatprDPP_D', 'RoopNgr_D',
    'RwStnDPP_D', 'Ricco_D', 'Aravind_D', 'MrdiVlge_D', 'Pnjbiyon_D',
    'BalibDPP_D', 'MdnprDPP_D', 'Kanakpur_D', 'Mthrapuri_D',
    'Mharajpr_D', 'PtrlBunk_D', 'Jabalpur', 'VUNagar_DC', 'Prjapati_D',
    'EtwahDPP_D', 'Kovaipudur_Dc', 'Viveka_DC', 'DiyoDPP_D',
    'PlsrddDPP_D', 'DhuleRoad_D', 'KrantiNgr_D', 'MrgnjDPP_D',
    'Pthrgoan_D', 'HelipadRD_D', 'DibngVly_D', 'Farmnala_D',
    'BChwkDPP_D', 'Jaripatk_DC', 'Shantanu_D', 'Kothanur_L',
    'SukntDPP_D', 'SndbrDPP_D', 'SbhRDDPP_D', 'BazarDPP_D',
    'Umargaon_DC', 'GndhiChk_D', 'Muktsar_D', 'Central_L_8',
    'ChtwrDPP_D', 'Bhaluahi_D', 'KcharaRD_D', 'RhmjDPP_D',
    'Bomsndra_PC', 'Cnsrvila_D', 'CroadDPP_D', 'Vidyangr_D',
    'BargaDPP_D', 'Enayetpr_D', 'RailwyRd_D', 'Solaiprm_D',
    'JawanChk_D', 'Court_D', 'East_D_8', 'Vijdurg_D', 'BrezeDPP_D',
    'JJCpxDPP_D', 'GurpdDPP_D', 'Rjndpara_D', 'SrnrHwy_D',
    'ColnyDPP_D', 'Kolar Mandakni', 'Central_H_4', 'GrmNgriya_D',
    'TilakNgr_D', 'Cherukole_D', 'Manchar_D', 'SubrtDPP_D',
    'Bnsibtla_D', 'MjlrDPP_D', 'Kothuru_D', 'Wardno8_D', 'KhsmiDPP_D',
    'Royapuram', 'Varanasi', 'SliprDPP_DC', 'RajRdDPP_D', 'ShivaDPP_D',
    'KeranDPP_D', 'East_D_7', 'Pshrikvu_D', 'Kooriyad_D', 'Parai_D',
    'RgstrOFC_D', 'MsmcyDPP_D', 'SriDPP_D', 'CBRoad_D', 'Sidrd_D',
    'Bhilai_DC', 'Nattukal_D', 'Rathnam_D', 'NditaDPP_D'].dtvne=object)

```

```
de['destination_state'].unique()
```

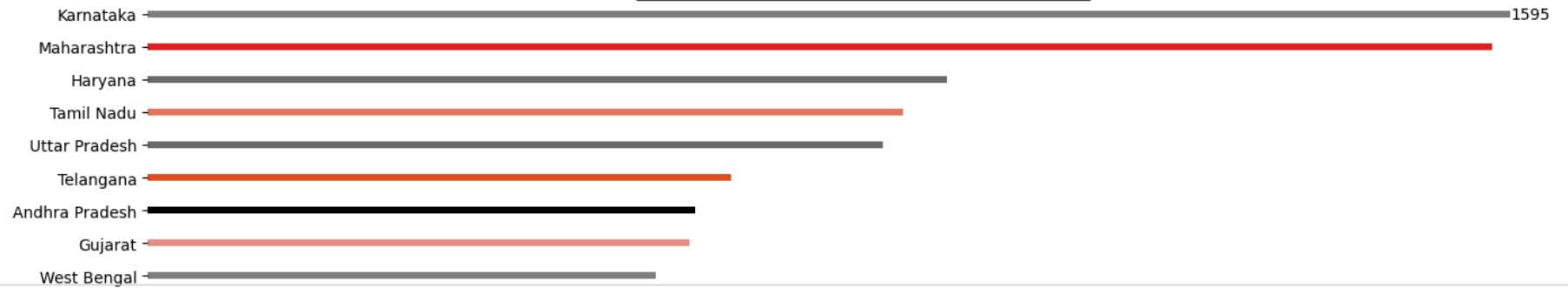
```

array(['Gujarat', 'Delhi', 'Haryana', 'Karnataka', 'Madhya Pradesh',
      'Assam', 'Telangana', 'Tamil Nadu', 'Punjab', 'West Bengal',
      'Dadra and Nagar Haveli', 'Andhra Pradesh', 'Maharashtra',
      'Uttar Pradesh', 'Bihar', 'Jharkhand', 'Pondicherry', 'Orissa',

```

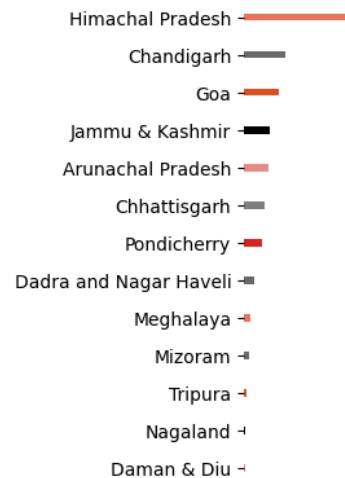
```
'Goa', 'Chandigarh', 'Himachal Pradesh', 'Rajasthan',  
'Arunachal Pradesh', 'Uttarakhand', 'Kerala', 'Chhattisgarh',  
'Jammu & Kashmir', 'Nagaland', 'Meghalaya', 'Tripura', 'Mizoram',  
'Daman & Diu'], dtype=object)
```

```
state_counts = de['destination_state'].value_counts().to_frame().reset_index()  
state_counts.columns = ['State', 'Count']  
  
plt.figure(figsize=(15,10))  
a = sns.barplot(y='State', x='Count', data=state_counts,palette=cp,width=0.2)  
a.bar_label(a.containers[0], label_type='edge')  
plt.xticks([])  
plt.ylabel('State')  
plt.xlabel('Count')  
plt.title('Number of Deliveries to Destination State',fontsize=12,fontfamily='serif',fontweight='bold',backgroundcolor='k',color='w')  
plt.tight_layout()  
sns.despine(bottom=True,left=True)  
plt.show()
```

Number of Deliveries to Destination State

```
city_counts = de['destination_city'].value_counts().to_frame().reset_index()[ :30]
city_counts.columns = ['City', 'Count']

plt.figure(figsize=(15,10))
a = sns.barplot(y='City', x='Count', data=city_counts,palette=cp,width=0.2)
a.bar_label(a.containers[0], label_type='edge')
plt.xticks([])
plt.ylabel('City')
plt.xlabel('Count')
plt.title('Number of Deliveries from destination City',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='k',color='w')
plt.tight_layout()
sns.despine(bottom=True,left=True)
plt.show()
```



Count



Insights:

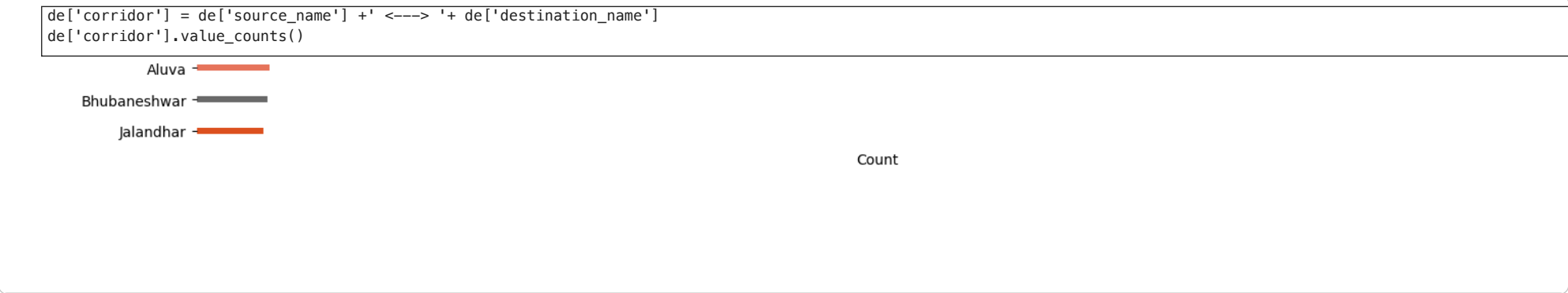
Destination State

- States like Karnataka, Maharashtra, Tamil Nadu, Haryana, and Uttar Pradesh where maximum packages are received in this month indicating significant engagement.

Destination City

- Cities like Bengaluru, Mumbai, Gurgaon, Bhiwandi, Hyderabad, Delhi where the major no.of booking are received.

```
np.set_printoptions(threshold=np.inf)
```



	count
corridor	
Bangalore_Nelmngla_H (Karnataka) <--> Bengaluru_Bomsndra_HB (Karnataka)	64
Pune_Tathawde_H (Maharashtra) <--> Bhiwandi_Mankoli_HB (Maharashtra)	62
Bangalore_Nelmngla_H (Karnataka) <--> Bengaluru_KGAirprt_HB (Karnataka)	62
Bhiwandi_Mankoli_HB (Maharashtra) <--> Mumbai Hub (Maharashtra)	54
Delhi_Gateway_HB (Delhi) <--> Gurgaon_Bilaspur_HB (Haryana)	50
...	...
Giridih_Shivalva_D (Jharkhand) <--> Madhupur_Sitamrd_D (Jharkhand)	1

```
corridor_counts = de['corridor'].value_counts()[:50]

plt.figure(figsize=(18,12))
#corridor_counts.plot(kind='line', marker='d', color='r')
sns.lineplot(y=corridor_counts.index, x=corridor_counts.values, marker='d', color='r')
plt.title('Value Counts of Corridor', fontsize=20, fontfamily='serif', fontweight='bold', backgroundcolor='dimgrey', color='w')
plt.ylabel('Corridor')
plt.xlabel('Count')
plt.tight_layout()
sns.despine()
plt.grid(True)

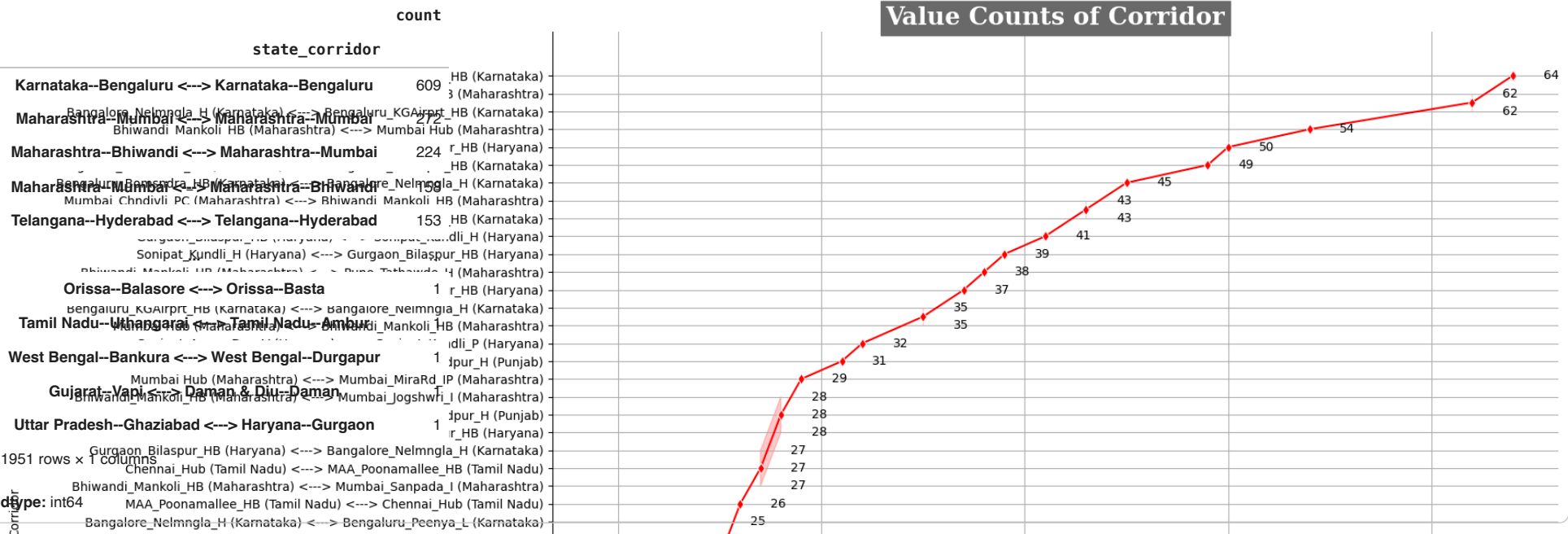
for i, count in enumerate(corridor_counts.values):
    plt.text(count+1.5, corridor_counts.index[i], str(count), ha='left', va='center')

plt.show()
```

✓ Insights:

- The route between Bangalore_Nelamangala_H to Bengaluru_KGAirport_HB, Bengaluru_Bomsndra_HB sees the highest package volume, with 151 and 127 packages sent respectively.
- Bengaluru_Bommasandra_HB to Bengaluru_KGAirport_HB is also popular, with 121 packages sent.
- Bengaluru_KGAirport_HB to Bangalore_Nelamangala_H has moderate activity, with 108 packages sent.
- The data indicates Bengaluru's importance as a transportation hub Corridor within Karnataka, handling significant package traffic.

```
de['state_corridor'] = de['source_state']+'--'+de['source_city'] +' <---> ' + de['destination_state']+'--'+de['destination_city']  
de['state_corridor'].value_counts()
```

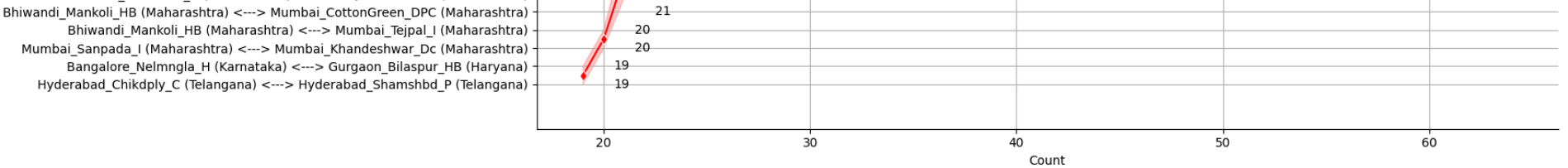


```
state_corridor_counts = de['state_corridor'].value_counts()[:25]

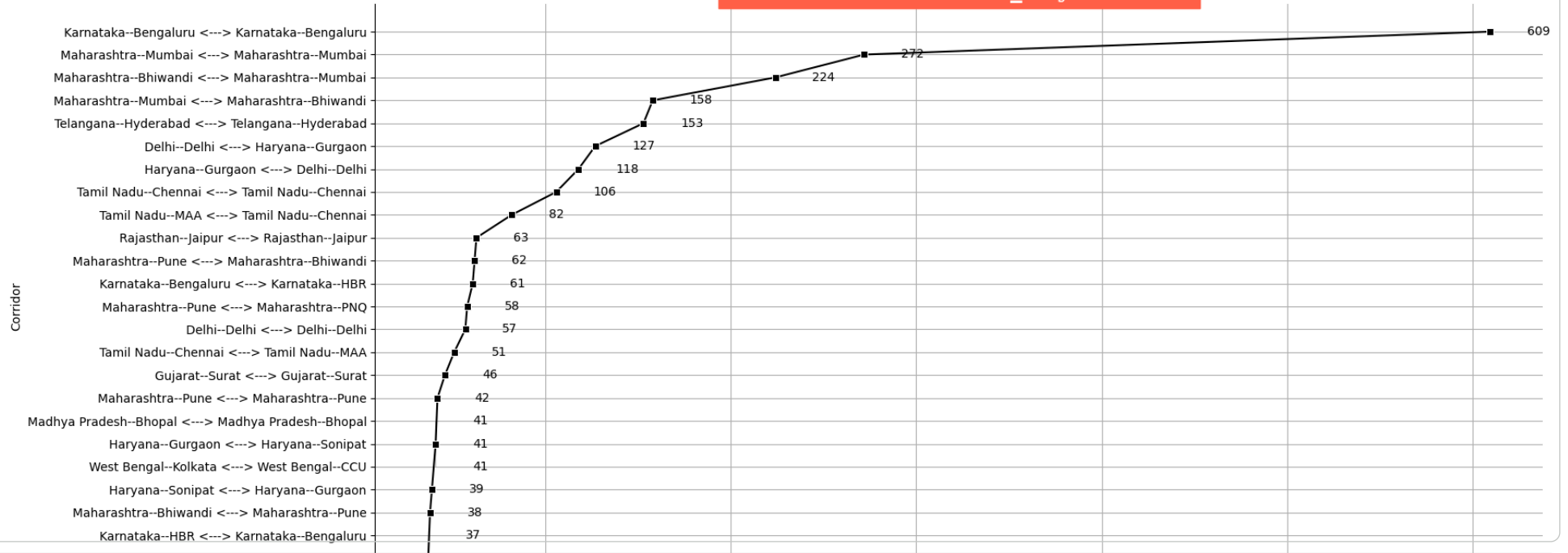
plt.figure(figsize=(18,8))
sns.lineplot(y=state_corridor_counts.index, x=state_corridor_counts.values, marker='s', color='k')
plt.title('Value Counts of state_city Corridor',fontsize=20,fontfamily='serif',fontweight='bold',backgroundcolor='tomato',color='w')
plt.ylabel('Corridor')
plt.xlabel('Count')
plt.tight_layout()
sns.despine()
plt.grid(True)

for i, count in enumerate(state_corridor_counts.values):
    plt.text(count+20, state_corridor_counts.index[i], str(count), ha='left', va='center')

plt.show()
```



Value Counts of state_city Corridor



```
de['city_corridor'] = de['source_city']+'--'+de['source_place'] + ' <---> ' + de['destination_city']+'--'+de['destination_place']
display(de['city_corridor'].value_counts())
```

Count

count

```
city_corridor_counts = de['city_corridor'].value_counts()[:30]

plt.figure(figsize=(18,8))
sns.lineplot(y=city_corridor_counts.index, x=city_corridor_counts.values, marker='o', color='tomato')
plt.title('Value Counts of city_place Corridor',fontsize=20,fontfamily='serif',fontweight='bold',backgroundcolor='dimgray',color='w')
plt.ylabel('Corridor')
plt.xlabel('Count')
plt.tight_layout()
sns.despine()
plt.grid(True)

for i, count in enumerate(city_corridor_counts.values):
    plt.text(count+2, city_corridor_counts.index[i], str(count), ha='left', va='center')

plt.show()
```

Gorakhpur--Matiprn_IP <--> Nichiaul--warano13_D 1

Kandi--DohalDPP_D <--> Ghanashyampur--NditaDPP_D 1

Chinnur--AsnsdhRD_D <--> Karimnagar--KamnHbRD_I 1

2288 rows x 1 columns

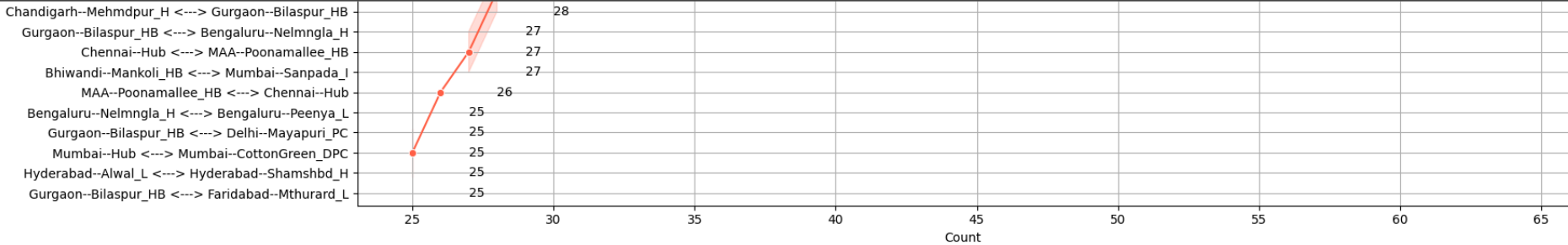
dtype: int64

Value Counts of city_place Corridor

Insights:

- Maharashtra, Karnataka, Haryana, and Tamil Nadu serve as key starting and ending locations for delivery services.
- Mumbai, Gurgaon, Delhi, and Bengaluru are major metropolitan centers from where many deliveries originate.
- A large proportion of nationwide deliveries are destined for Mumbai, Bengaluru, Gurgaon, and Delhi.

```
# 4. Extracting features like month, year, day, etc. from Trip_creation_time
de['trip_creation_month'] = de['trip_creation_time'].dt.month
de['trip_creation_year'] = de['trip_creation_time'].dt.year
de['trip_creation_day'] = de['trip_creation_time'].dt.day
de['trip_creation_hour'] = de['trip_creation_time'].dt.hour
de['trip_creation_weekday'] = de['trip_creation_time'].dt.weekday
de['trip_creation_week'] = de['trip_creation_time'].dt.isocalendar().week
de
```



	segment_key	trip_uuid	data	route_type	trip_creation_time	source_name	destination_name	od_start_time	od_end_time
0	trip-153671074033284934+IND395009AAA+IND395023AAD	trip-153671074033284934	training	Carting	2018-09-12 00:05:40.333071	Surat_Central_D_12 (Gujarat)	Surat_Central_I_4 (Gujarat)	2018-09-12 02:31:39.246238	2018-09-12 05:16:28.56
1	trip-153671074033284934+IND395023AAD+IND395004AAB	trip-153671074033284934	training	Carting	2018-09-12 00:05:40.333071	Surat_Central_I_4 (Gujarat)	Surat_Central_D_3 (Gujarat)	2018-09-12 00:05:40.333071	2018-09-12 02:01:41.63
2	trip-153671079956500691+IND110024AAA+IND110014AAA	trip-153671079956500691	training	Carting	2018-09-12 00:06:39.565253	Delhi_Lajpat_IP (Delhi)	Delhi_Bhogal (Delhi)	2018-09-12 00:06:39.565253	2018-09-12 00:55:59.56

▼ In-depth analysis

```
new_df = de.copy()
```

4	trip-153671127071609610+IND585104AAA+IND586128AAA	trip-153671127071609610	training	Carting	2018-09-12 00:14:30.716368	FBD_Balabhgarh_DPC (Haryana)	Faridabad (Haryana)	2018-09-12 00:11:40.783923	2018-09-12 00:50:10.81
---	---	-------------------------	----------	---------	----------------------------	------------------------------	---------------------	----------------------------	------------------------

```
new_df.columns
```

```
Index(['segment_key', 'trip_uuid', 'data', 'route_type', 'trip_creation_time', 'source_name', 'destination_name', 'od_start_time', 'od_end_time', 'actual_time', 'actual_distance', 'segment_osrm_time', 'segment_osrm_distance', 'segment_osrm_time_sum', 'segment_osrm_distance_sum', 'od_total_time', 'od_time_diff_hour', 'source_city', 'source_place', 'source_state', 'destination_city', 'destination_place', 'destination_state', 'trip_creation_year', 'trip_creation_day', 'trip_creation_hour', 'trip_creation_weekday', 'trip_creation_week'], dtype='object')
```

11820	trip-153671095623827784+IND160702AAD+IND160602AAC	trip-153671095623827784	test	Carting	2018-10-03 23:55:56.258533	Chandigarh_Raiprkl_C (Chandigarh)	Chandigarh_Mehmdpur_H (Punjab)	2018-10-04 04:56:10.943956	2018-10-04 06:41:25.40
11821	trip-153671095623827784+IND208006AAA+IND209904AAA	trip-153671095623827784	test	Carting	2018-10-03 23:57:44.429324	Kanpur_GovndNgr_DC (Uttar Pradesh)	Kanpur_Central_H_6 (Uttar Pradesh)	2018-10-04 02:51:27.075797	2018-10-04 06:59:51.62
11822	trip-153671095623827784+IND208006AAA+IND209904AAA	trip-153671095623827784	test	Carting	2018-10-03	Kanpur_Central_H_6	Kanpur_GovndNgr_DC	2018-10-03	2018-10-03

```
new_df.sample(2)
```

11823	trip-153671127071609610+IND585104AAA+IND586128AAA	trip-153671127071609610	test	FTL	2018-09-12 00:50:10.81	Saraswati (Karnataka)	Bellary_DC (Karnataka)	2018-09-12 00:50:10.81	2018-09-12 00:50:10.81
4284	trip-153736515614896455+IND577401AAA+IND577201AAA	trip-153736515614896455	training	FTL	2018-09-19 13:52:36.149199	Sagara_Vardhard_D (Karnataka)	Shimoga_DC (Karnataka)	2018-09-20 02:36:27.193185	2018-09-20 05:12:56.298090
18224	trip-153797930071860378+IND683511AAA+IND000000ADM	trip-153797930071860378	training	Carting	2018-09-26 16:28:20.718846	Aluva_Peedika_H (Kerala)	Cochin_Kdvantra_D (Kerala)	2018-09-26 16:28:20.718846	2018-09-27 01:41:37.263043

```
create_trip_dict={
    'data': 'first',
    'route_type': 'first',
    'od_start_time': 'first',
    'od_end_time': 'last',
```

```
'od_time_diff_hour' : 'sum',
'trip_creation_time' : 'first',
'trip_creation_month' : 'first',
'trip_creation_year' : 'first',
'trip_creation_day' : 'first',
'trip_creation_hour' : 'first',
'trip_creation_weekday' : 'first',
'trip_creation_week' : 'first',
'start_scan_to_end_scan' : 'sum',
'actual_distance_to_destination' : 'sum',
'actual_time' : 'sum',
'osrm_time' : 'sum',
'osrm_distance' : 'sum',
'segment_actual_time' : 'sum',
'segment_osrm_time' : 'sum',
'segment_osrm_distance' : 'sum',
'segment_actual_time_sum' : 'sum',
'segment_osrm_time_sum' : 'sum',
'segment_osrm_distance_sum' : 'sum',
'source_name' : 'first',
'source_city' : 'first',
'source_state' : 'first',
'source_place' : 'first',
'destination_name' : 'first',
'destination_city' : 'first',
'destination_state' : 'first',
'destination_place' : 'first',
'corridor' : 'first',
'state_corridor' : 'first',
'city_corridor' : 'first'
}

trip_agg_df = new_df.groupby('trip_uuid').agg(create_trip_dict).reset_index()
trip_agg_df
```

	trip_uuid	data	route_type	od_start_time	od_end_time	od_time_diff_hour	trip_creation_time	trip_creation_month	trip_creation_year	trip_creation_day	tr
0	trip-153671074033284934	training	Carting	2018-09-12 02:31:39.246238	2018-09-12 02:01:41.638015	4.680733	2018-09-12 00:05:40.333071	9	2018	12	
1	trip-153671079956500691	training	Carting	2018-09-12 00:06:39.565253	2018-09-12 00:55:59.568645	0.822223	2018-09-12 00:06:39.565253	9	2018	12	
2	trip-153671110078355292	training	Carting	2018-09-12 00:11:40.783923	2018-09-12 00:50:10.814399	0.641675	2018-09-12 00:11:40.783923	9	2018	12	
3	trip-153671127071609610	training	Carting	2018-09-12 00:14:30.716368	2018-09-12 07:54:13.838441	15.511243	2018-09-12 00:14:30.716368	9	2018	12	
4	trip-153671191949943656	training	FTL	2018-09-12 04:33:09.726898	2018-09-12 09:23:49.463096	4.844371	2018-09-12 00:25:19.499696	9	2018	12	
...	...	...	...	...	...	...	...	...	...	...	

```
numerical_columns = trip_agg_df.select_dtypes(include=[np.float32, np.float64])
numerical_columns
```

	od_time_diff_hour	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_d	
6677	trip	test	Carting	2018-10-03	2018-10-03	3.040278	2018-10-03	10	2018	3
0	4.680733		280.0	25.454847	161.0	29.0	30.935898	161.0	29.0	3
1	0.822223		49.0	9.872146	23.0	8.0	9.956600	23.0	14.0	1
2	0.641675		38.0	9.396525	17.0	9.0	10.815900	17.0	9.0	1
3	15.511243		928.0	230.413864	561.0	253.0	284.458191	554.0	258.0	29
4	4.844371		290.0	99.975594	253.0	105.0	124.506302	252.0	132.0	14
...	...		...	...	...	...	...	...	...	
6676	3.126754		186.0	28.114090	86.0	30.0	34.922401	86.0	30.0	3
6677	153861118279144424	test	FTL	03:58:40.726547	03:58:40.726547	3.300793	23:58:42.701692	10	2018	3
6678	4.300482		257.0	57.762333	83.0	62.0	73.462997	82.0	62.0	6
6679	7.035331		421.0	38.684837	282.0	48.0	58.903702	281.0	88.0	10
6680	5.906793		353.0	66.081528	275.0	68.0	80.578705	274.0	67.0	8

6681 rows x 12 columns

Next steps: [New interactive sheet](#)

```
numerical_columns.describe().T
```

	count	mean	std	min	25%	50%	75%	max	
od_time_diff_hour	6681.0	8.808342	10.917330	0.463311	2.512328	4.725590	10.482100	124.315659	
start_scan_to_end_scan	6681.0	527.400513	654.172180	27.000000	150.000000	283.000000	627.000000	7458.000000	
actual_distance_to_destination	6681.0	164.237976	303.202942	9.003577	23.065063	48.713085	161.357483	2185.480225	
actual_time	6681.0	355.222260	558.235168	9.000000	68.000000	150.000000	364.000000	6265.000000	
osrm_time	6681.0	161.133957	269.826721	6.000000	30.000000	60.000000	167.000000	2032.000000	
osrm_distance	6681.0	204.014145	368.380951	9.072900	30.955502	65.985100	204.741394	2839.660156	
segment_actual_time	6681.0	351.979034	553.138428	9.000000	67.000000	148.000000	361.000000	6230.000000	
segment_osrm_time	6681.0	180.391113	311.022125	6.000000	31.000000	65.000000	184.000000	2494.000000	
segment_osrm_distance	6681.0	222.448029	412.019104	9.072900	32.950199	71.223404	215.281693	3418.182861	
segment_actual_time_sum	6681.0	351.979034	553.138428	9.000000	67.000000	148.000000	361.000000	6230.000000	
segment_osrm_time_sum	6681.0	180.391113	311.022125	6.000000	31.000000	65.000000	184.000000	2494.000000	
segment_osrm_distance_sum	6681.0	222.448029	412.019104	9.072900	32.950199	71.223404	215.281693	3418.182861	

```
trip_agg_df.describe(include = object).T
```

	count	unique	top	freq	
trip_uuid	6681	6681	trip-153861118270144424	1	
source_name	6681	742	Gurgaon_Bilaspur_HB (Haryana)	485	
source_city	6681	561	Bengaluru	730	
source_state	6681	29	Maharashtra	1241	
source_place	6681	639	Bilaspur_HB	485	
destination_name	6681	840	Gurgaon_Bilaspur_HB (Haryana)	368	
destination_city	6681	689	Bengaluru	698	
destination_state	6681	32	Maharashtra	1180	
destination_place	6681	704	Bilaspur_HB	368	
corridor	6681	1335	Pune_Tathawde_H (Maharashtra) <---> Bhiwandi_M...	62	
state_corridor	6681	1050	Karnataka--Bengaluru <---> Karnataka--Bengaluru	570	
city_corridor	6681	1335	Pune--Tathawde_H <---> Bhiwandi--Mankoli_HB	62	

```
trip_df = trip_agg_df.copy()
```

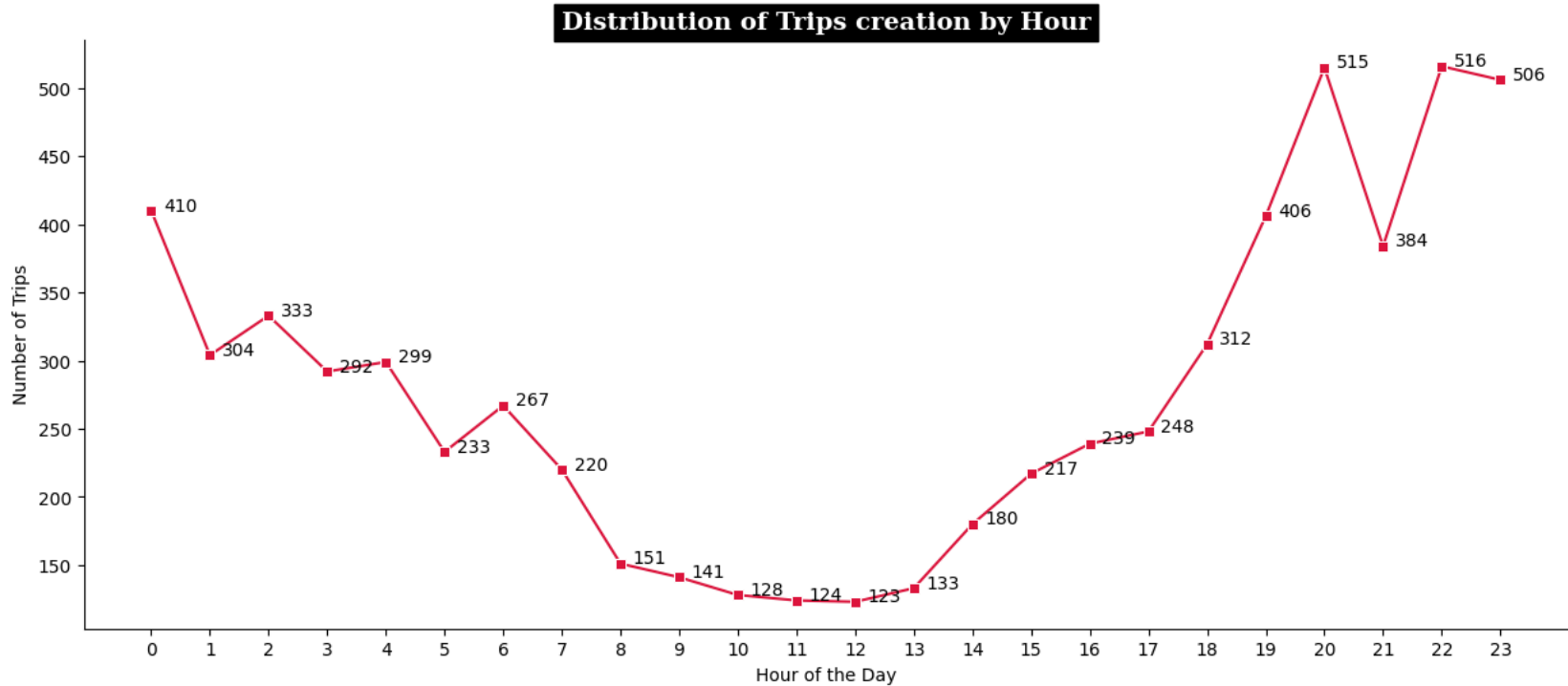
```
trip_creation_by_hour = trip_df.groupby(by='trip_creation_hour')['trip_uuid'].count().reset_index()

plt.figure(figsize=(15,6))
sns.lineplot(data=trip_creation_by_hour, x='trip_creation hour', y='trip uuid', marker='s', color='crimson')
```

```
plt.xticks(np.arange(0, 24))

for i, count in enumerate(trip_creation_by_hour['trip_uuid']):
    plt.text(trip_creation_by_hour['trip_creation_hour'][i]+0.5, count, count, ha='center')

plt.title('Distribution of Trips creation by Hour',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='k',color='w')
plt.xlabel('Hour of the Day')
plt.ylabel('Number of Trips')
sns.despine()
plt.show()
```



```
trip_df.sample()
```

	trip_uuid	data	route_type	od_start_time	od_end_time	od_time_diff_hour	trip_creation_time	trip_creation_month	trip_creation_year	trip_creation_day	trip
6615	trip-153859936712332299	test	Carting	2018-10-03 20:42:47.123593	2018-10-04 04:01:02.073160	9.214738	2018-10-03 20:42:47.123593	10	2018	3	

```
trip_df.trip_creation_year.value_counts()
```

	count
trip_creation_year	
2018	6681

dtype: int64

trip_df.trip_creation_month.value_counts()

	count
trip_creation_month	
9	5884
10	797

dtype: int64

trip_df['trip_creation_month'].value_counts(normalize = True) * 100

	proportion
trip_creation_month	
9	88.070648
10	11.929352

dtype: float64

trip_df.trip_creation_week.value_counts()

	count
trip_creation_week	
38	2264
39	1970
37	1650
40	797

dtype: Int64

trip_df.trip_creation_weekday.value_counts(ascending=True)

trip_creation_weekday	count
6	783
0	906
1	925
3	947
5	950
4	983
2	1187

dtype: int64

```
trip_df['trip_creation_day_week'] = trip_df['trip_creation_time'].dt.day_name()
```

```
trip_df.trip_creation_day.value_counts()
```

	count
trip_creation_day	
18	370
21	367
15	358
13	343
14	341
17	331
12	324
22	323
25	322
20	313
26	301
27	291
1	291
19	289
16	284
24	284
28	275
3	273
23	271
29	269
2	233
30	228

dtype: int64

```
trip_df.sample()
```

	trip_uuid	data	route_type	od_start_time	od_end_time	od_time_diff_hour	trip_creation_time	trip_creation_month	trip_creation_year	trip_creation_day	tr
4106	153781739343259695	trip-training	FTL	2018-09-25 02:20:27.377294	2018-09-25 02:20:27.377294	9.345462	2018-09-24 19:29:53.432970	9	2018	24	

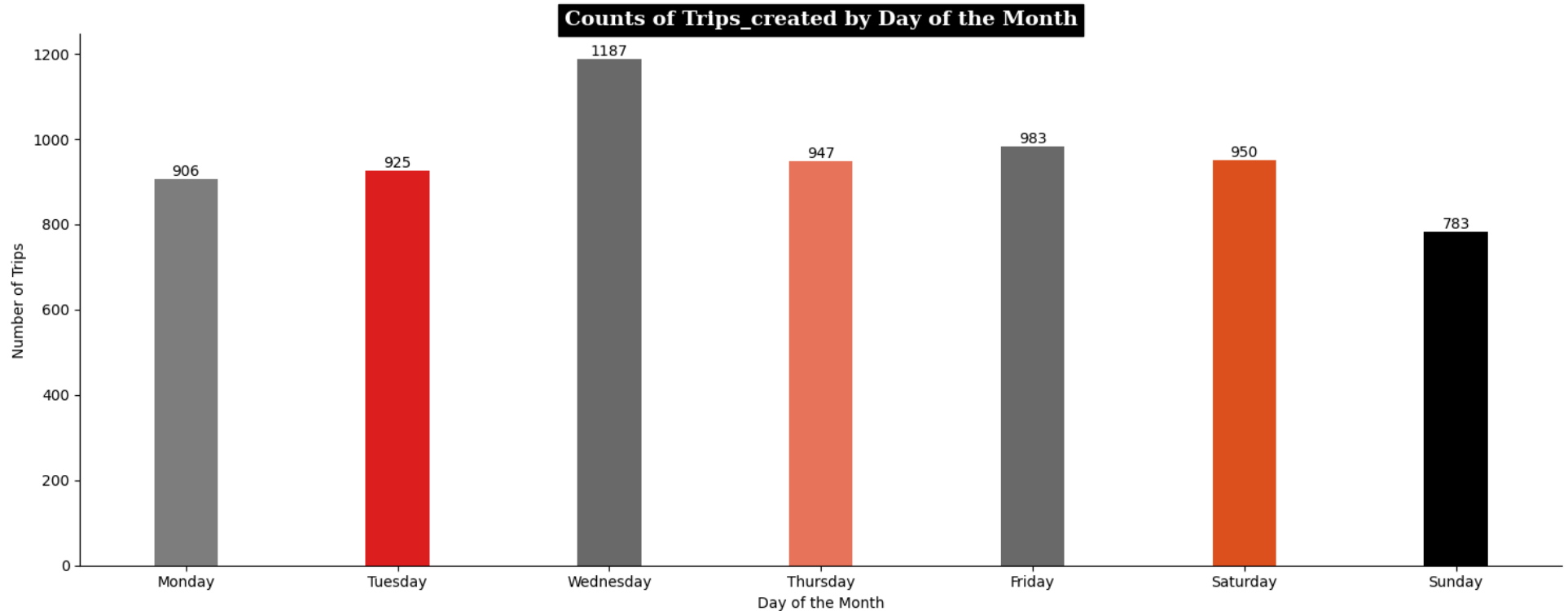
```
plt.figure(figsize=(15,6))
```

```

weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
day_counts = trip_df['trip_creation_day_week'].value_counts().reindex(weekday_order)

sns.barplot(x=day_counts.index, y=day_counts.values, palette=cp,width=0.3)
for i, count in enumerate(day_counts.values):
    plt.text(i, count, str(count), ha='center', va='bottom')
plt.title('Counts of Trips_created by Day of the Month',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='k',color='w')
plt.xlabel('Day of the Month')
plt.ylabel('Number of Trips')
plt.tight_layout()
sns.despine()
plt.show()

```



```
trip_df['trip_creation_dayofdate'] = trip_df['trip_creation_time'].dt.day
```

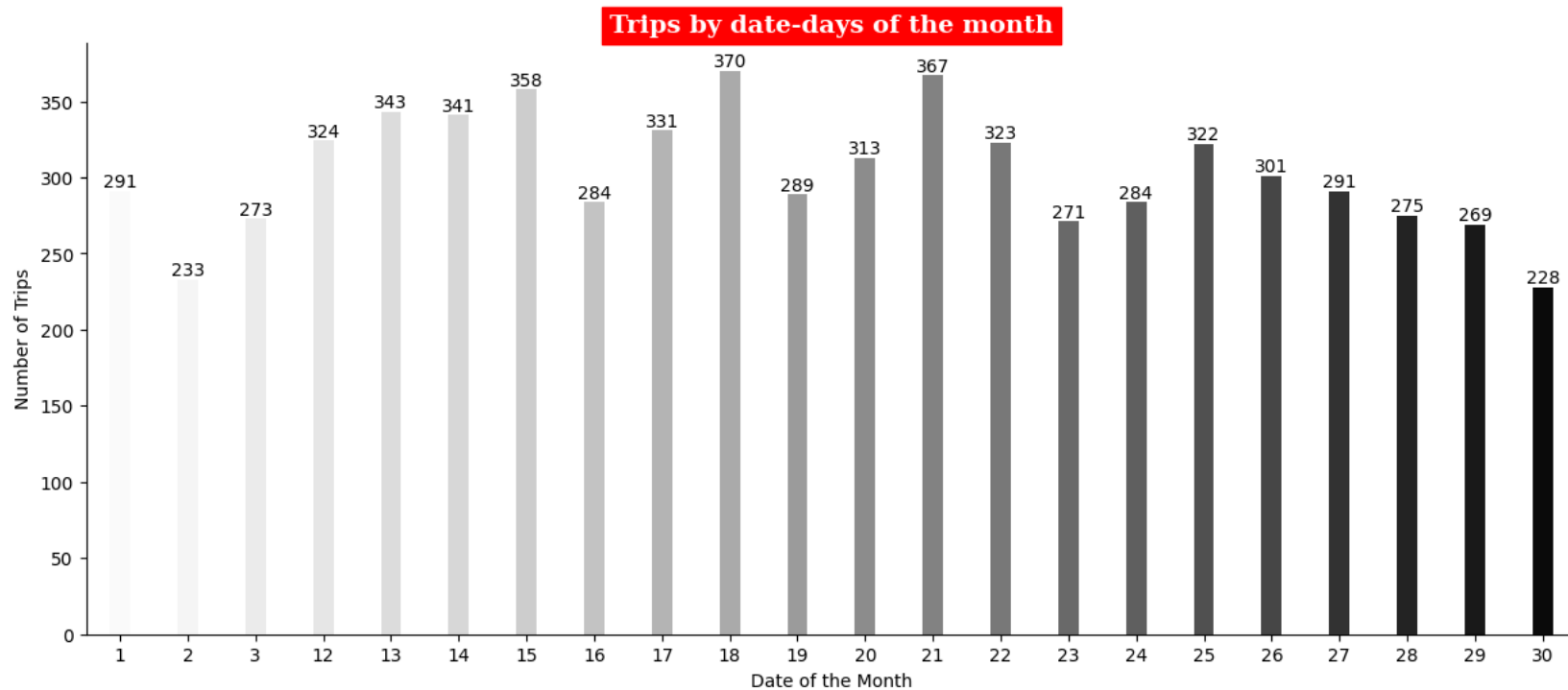
```

trips_by_dateday = trip_df.groupby(by = 'trip_creation_dayofdate')['trip_uuid'].count().to_frame().reset_index()

plt.figure(figsize = (15, 6))
sns.barplot(data =trip_df,x = trips_by_dateday['trip_creation_dayofdate'],y = trips_by_dateday['trip_uuid'], palette='Greys',width=0.3)
for i, count in enumerate(trips_by_dateday['trip_uuid']):
    plt.text(i, count, str(count), ha='center', va='bottom')
plt.title('Trips by date-days of the month',fontsize=14,fontfamily='serif',fontweight='bold',backgroundcolor='r',color='w')
plt.xlabel('Date of the Month')

```

```
plt.ylabel('Number of Trips')  
sns.despine()  
plt.show()
```



Outlier treatment

```
numerical_columns
```

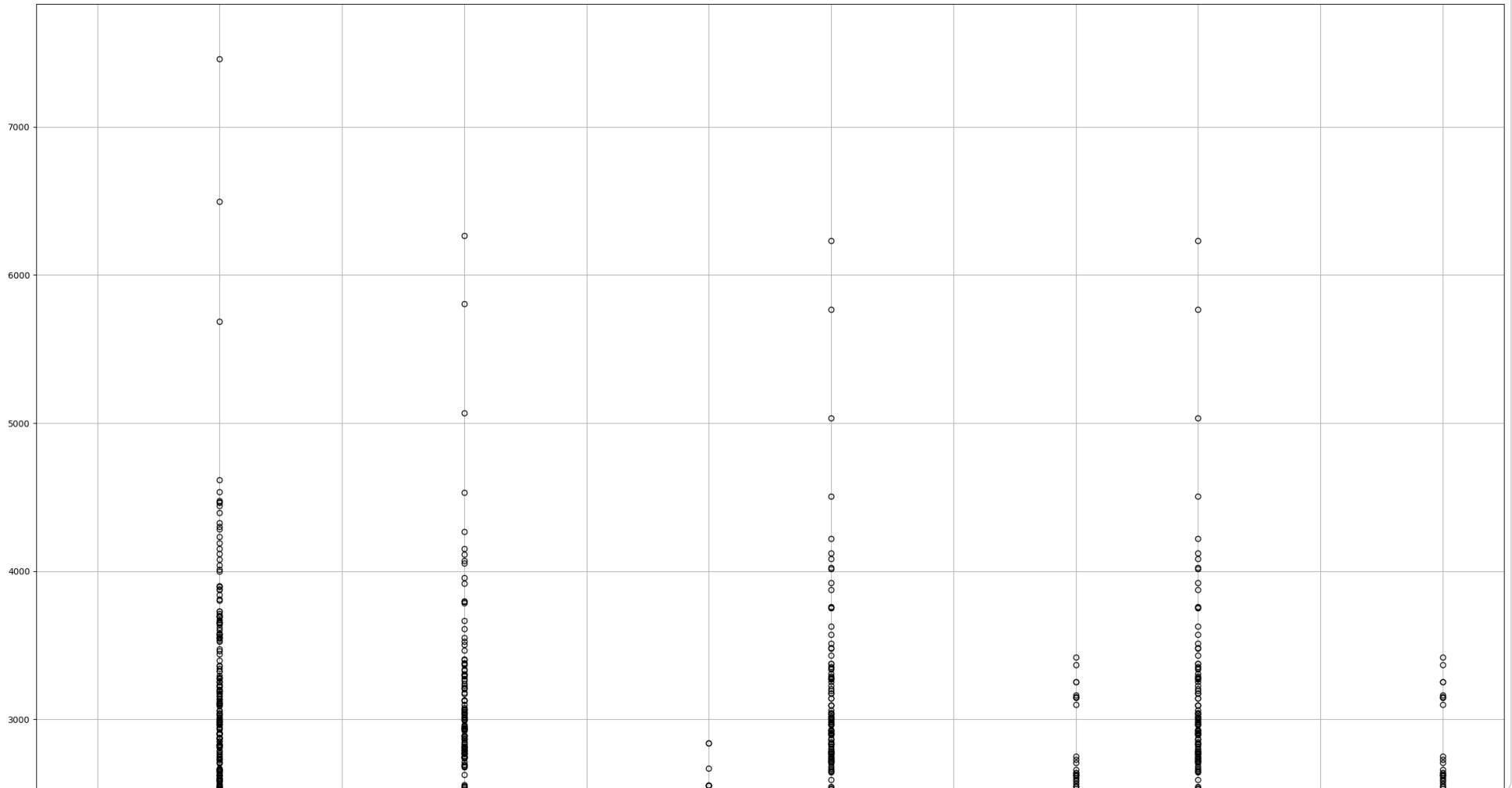
	od_time_diff_hour	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_d
0	4.680733	280.0	25.454847	161.0	29.0	30.935898	161.0	29.0	3
1	0.822223	49.0	9.872146	23.0	8.0	9.956600	23.0	14.0	1
2	0.641675	38.0	9.396525	17.0	9.0	10.815900	17.0	9.0	1
3	15.511243	928.0	230.413864	561.0	253.0	284.458191	554.0	258.0	29
4	4.844371	290.0	99.975594	253.0	105.0	124.506302	252.0	132.0	14
...	...	...	...	...	...	...	...	...	...
6676	3.126754	186.0	28.114090	86.0	30.0	34.922401	86.0	30.0	3
6677	3.040278	181.0	27.010925	62.0	28.0	38.286697	61.0	33.0	3
6678	4.300482	257.0	57.762333	83.0	62.0	73.462997	82.0	62.0	6
6679	7.035331	421.0	38.684837	282.0	48.0	58.903702	281.0	88.0	10
6680	5.906793	353.0	66.081528	275.0	68.0	80.578705	274.0	67.0	8

6681 rows x 12 columns

Next steps: [New interactive sheet](#)

```
plt.figure(figsize=(30, 25))
numerical_columns.boxplot(rot=25, figsize=(35,20), color = 'r')
plt.grid('off')
plt.show()
```



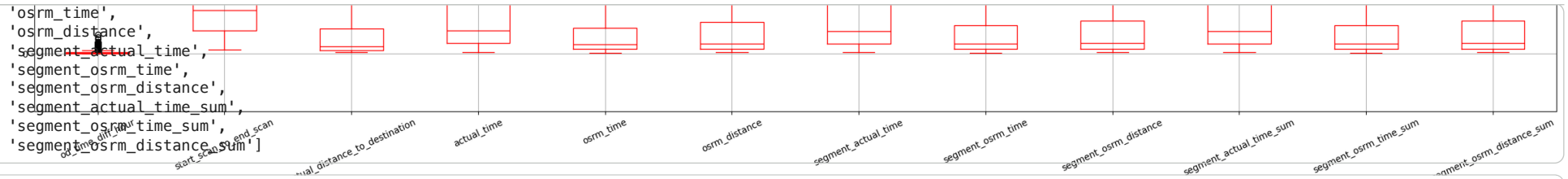


```
numerical_columns.columns
```

```
Index(['od_time_diff_hour', 'start_scan_to_end_scan',
      'actual_distance_to_destination', 'actual_time', 'osrm_time',
      'osrm_distance', 'segment_actual_time', 'segment_osrm_time',
      'segment_osrm_distance', 'segment_actual_time_sum',
      'segment_osrm_time_sum', 'segment_osrm_distance_sum'],
      dtype='object')
```

```
num_cols = numerical_columns.columns.tolist()
num_cols
```

```
['od_time_diff_hour',
 'start_scan_to_end_scan',
 'actual_distance_to_destination',
 'actual_time',
```



```
# obtain the first quartile
Q1 = numerical_columns.quantile(0.25)

# obtain the third quartile
Q3 = numerical_columns.quantile(0.75)

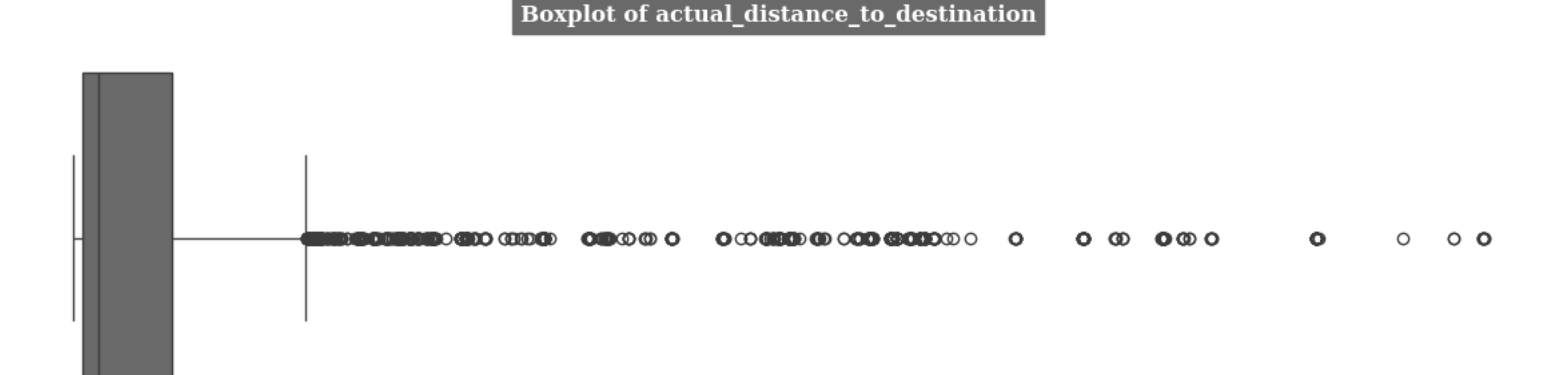
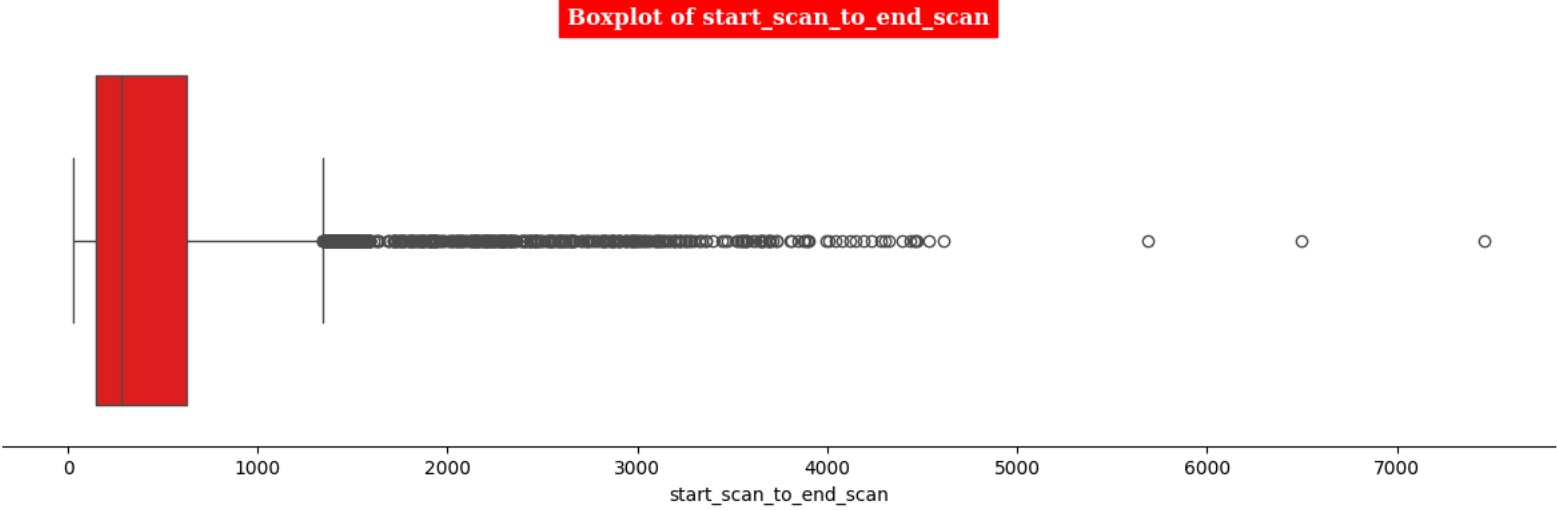
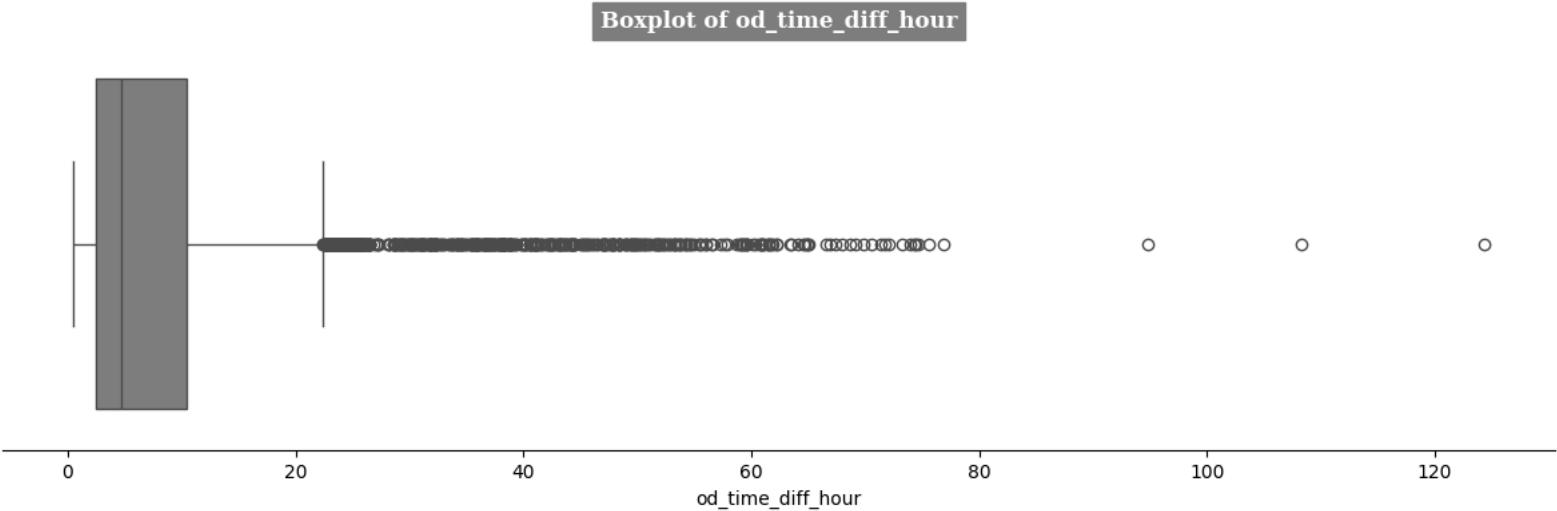
# obtain the IQR
IQR = Q3 - Q1

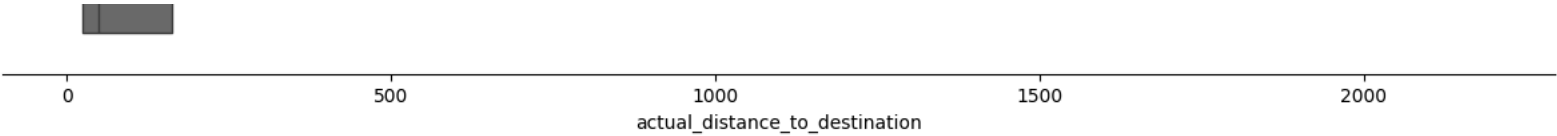
# print the IQR
print(IQR)
```

```
od_time_diff_hour      7.969772
start_scan_to_end_scan 477.000000
actual_distance_to_destination 138.292419
actual_time            296.000000
osrm_time              137.000000
osrm_distance          173.785892
segment_actual_time    294.000000
segment_osrm_time      153.000000
segment_osrm_distance  182.331493
segment_actual_time_sum 294.000000
segment_osrm_time_sum  153.000000
segment_osrm_distance_sum 182.331493
dtype: float64
```

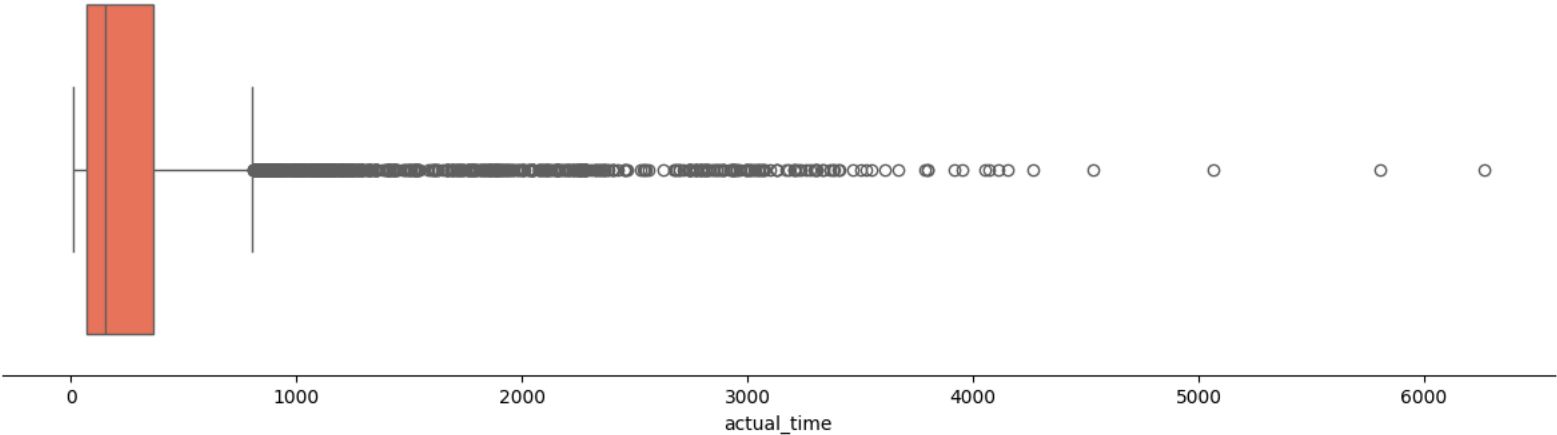
```
for i,col in enumerate(numerical_columns):
    plt.figure(figsize=(15,4))
    sns.boxplot(x=col, data=numerical_columns,color=cp[i])
    sns.despine(left=True)
    plt.yticks([])
    plt.title(f'Boxplot of {col}',fontfamily='serif',fontweight='bold',fontsize=12,backgroundcolor=cp[i],color='w')
    plt.show()
```



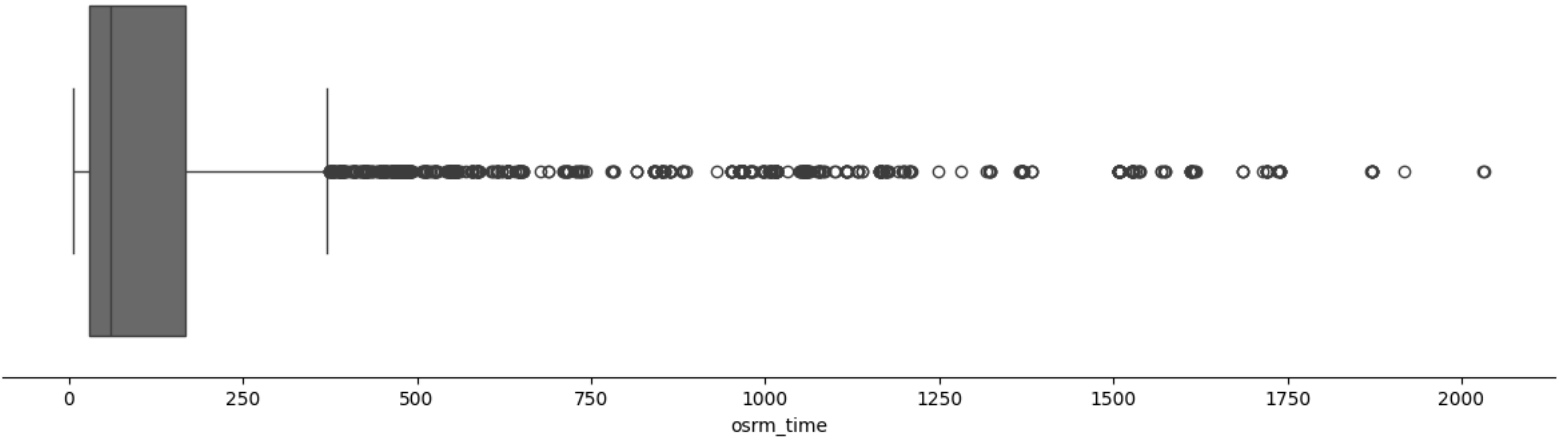




Boxplot of actual_time

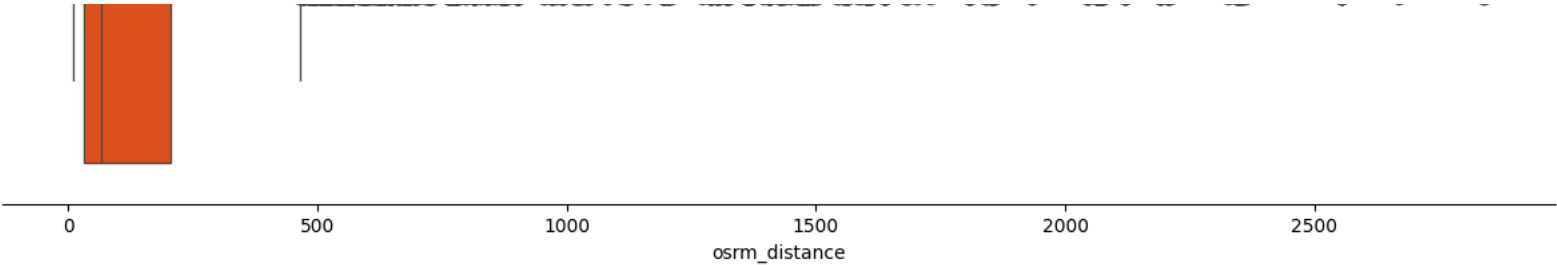


Boxplot of osrm_distance

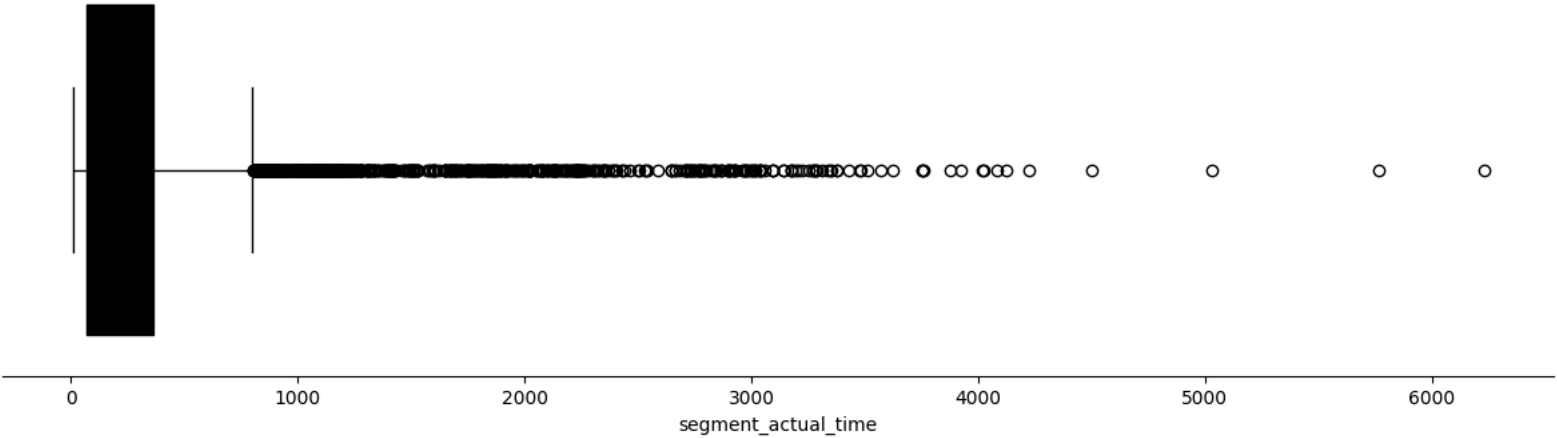


Boxplot of actual_distance_to_destination

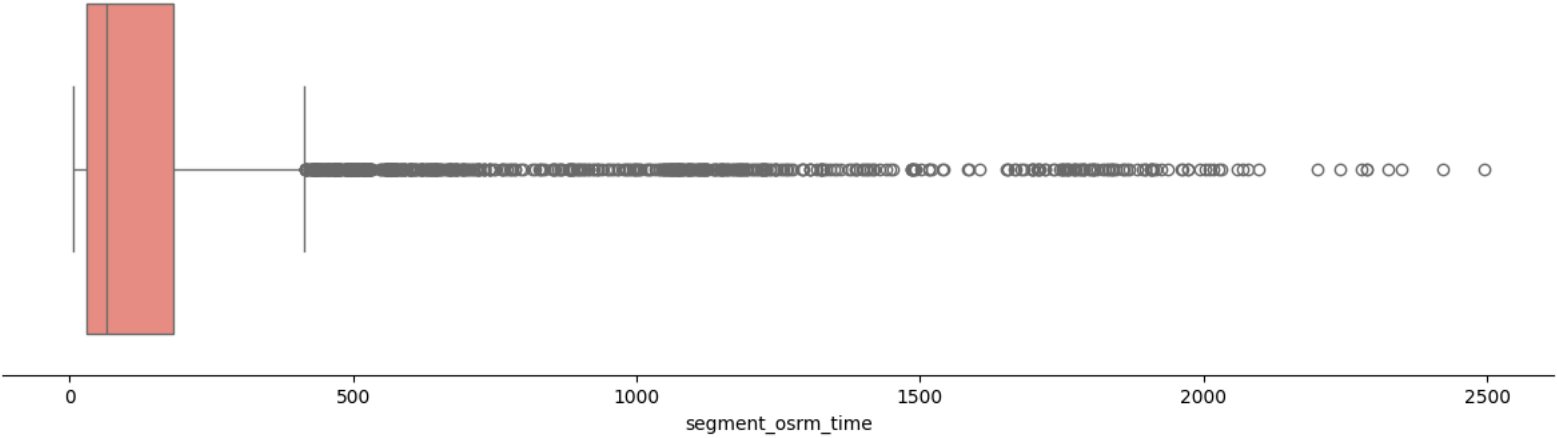




Boxplot of segment_actual_time

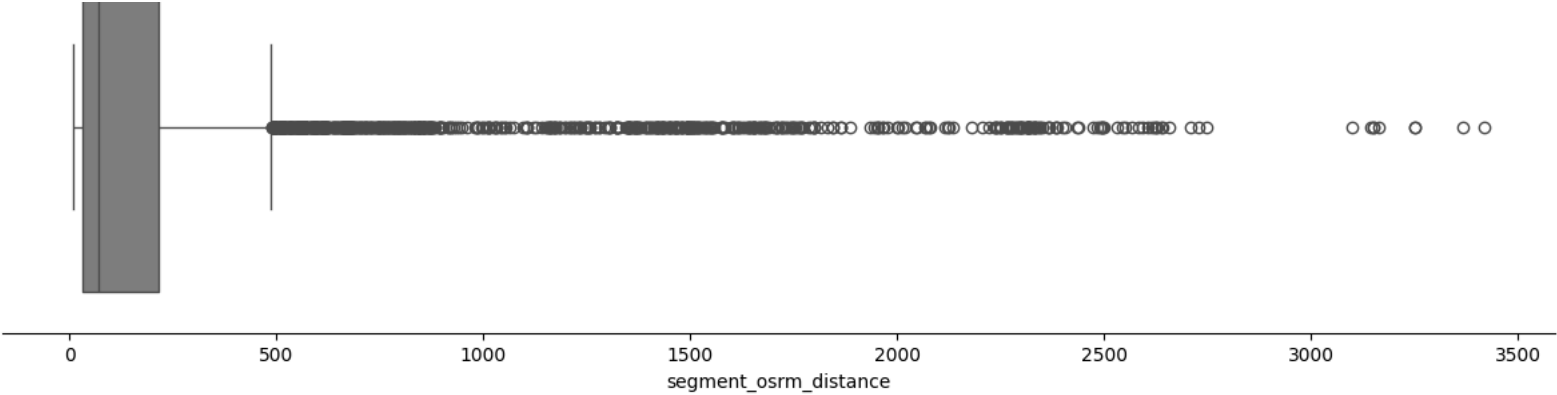


Boxplot of segment_osrm_time

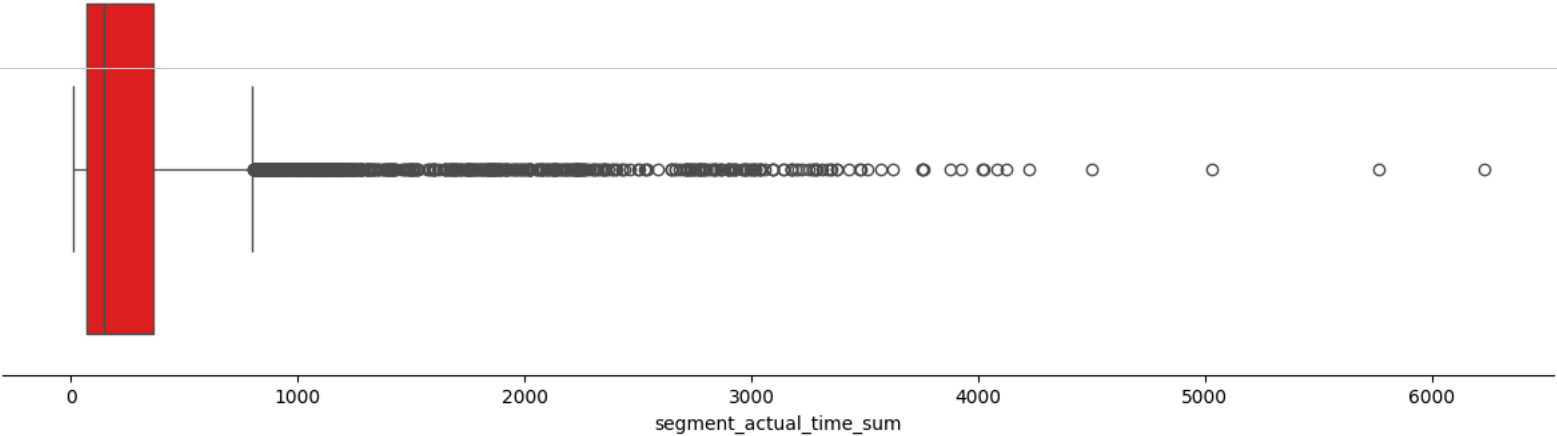


Boxplot of segment_osrm_distance

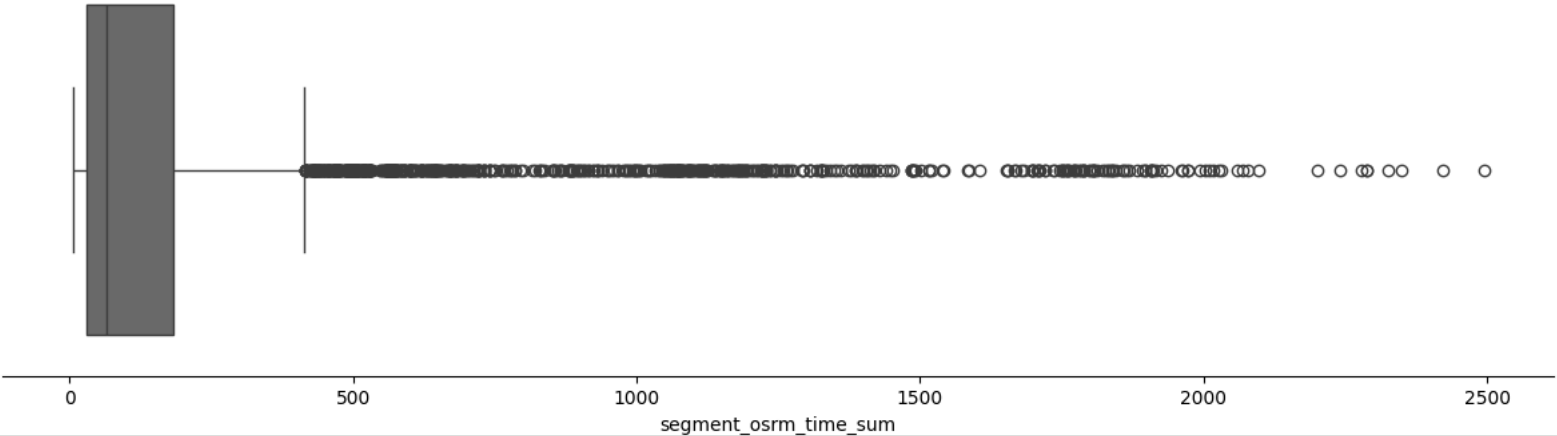




Boxplot of segment_actual_time_sum



Boxplot of segment_osrm_time_sum



v Outlier removal

Boxplot of segment_osrm_distance_sum

```
for i, col in enumerate(numerical_columns):  
  
    data = trip_df[col]  
    display(data.to_frame())  
  
    Q1 = np.percentile(data, 25)  
    Q3 = np.percentile(data, 75)  
    IQR = Q3 - Q1  
  
    lower_bound = Q1 - (1.5 * IQR)  
    upper_bound = Q3 + (1.5 * IQR)  
  
    clipped_data = np.clip(data, lower_bound, upper_bound)  
    print(f'Clipped data of {col}')    display(clipped_data.to_frame())  
    print()  
  
    # Plot boxplot of the clipped data  
    plt.figure(figsize=(15, 4))  
    plt.subplot(121)  
    sns.boxplot(x=clipped_data, color=cp[i])  
    sns.despine(left=True)  
    plt.yticks([])  
    plt.title(f'Boxplot of clipped {col}', fontfamily='serif', fontweight='bold', fontsize=12, backgroundcolor=cp[i], color='w')  
  
    filtered_data = data.loc[(data >= lower_bound) | (data <= upper_bound)]  
    print(f'Filtered data of {col}')    display(filtered_data.to_frame())  
    print()  
  
    plt.subplot(122)  
    sns.boxplot(x=filtered_data, color=cp[i])  
    sns.despine(left=True)  
    plt.yticks([])  
    plt.title(f'Boxplot of filtered {col}', fontfamily='serif', fontweight='bold', fontsize=12, backgroundcolor=cp[i], color='w')  
  
    plt.show()
```



od_time_diff_hour 	
0	4.680733
1	0.822223
2	0.641675
3	15.511243
4	4.844371
...	...
6676	3.126754
6677	3.040278
6678	4.300482
6679	7.035331
6680	5.906793

6681 rows x 1 columns

Clipped data of od_time_diff_hour

od_time_diff_hour	
0	4.680733
1	0.822223
2	0.641675
3	15.511243
4	4.844371
...	...
6676	3.126754
6677	3.040278
6678	4.300482
6679	7.035331
6680	5.906793

6681 rows x 1 columns

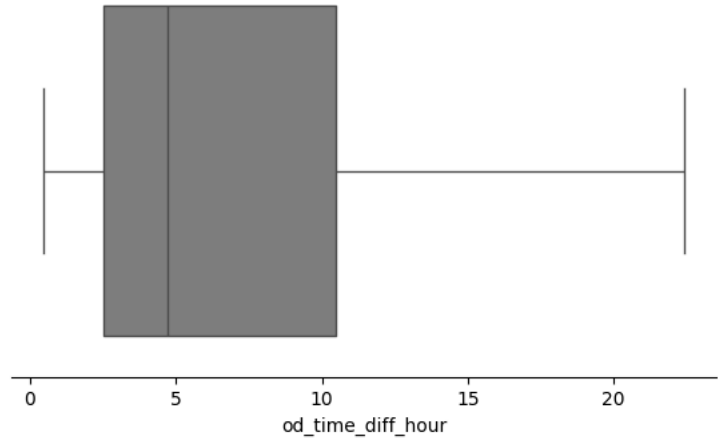
Filtered data of od_time_diff_hour

od_time_diff_hour	
0	4.680733
1	0.822223
2	0.641675
3	15.511243
4	4.844371

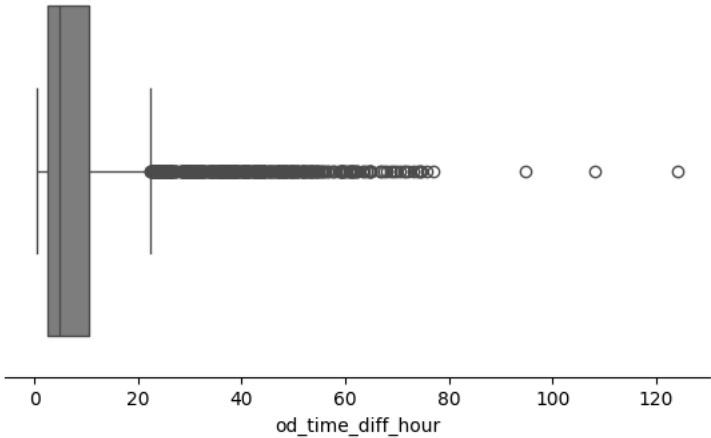
```
...
6676      3.126754
6677      3.040278
6678      4.300482
6679      7.035331
6680      5.906793
```

6681 rows x 1 columns

Boxplot of clipped od_time_diff_hour



Boxplot of filtered od_time_diff_hour



```
start_scan_to_end_scan
0      280.0
1      49.0
2      38.0
3     928.0
4     290.0
...
6676    186.0
6677    181.0
6678    257.0
6679    421.0
6680    353.0
```

6681 rows x 1 columns

Clipped data of start_scan_to_end_scan

```
start_scan_to_end_scan
```

0	280.0
1	49.0
2	38.0
3	928.0
4	290.0
...	...
6676	186.0
6677	181.0
6678	257.0
6679	421.0
6680	353.0

6681 rows x 1 columns

Filtered data of start_scan_to_end_scan

start_scan_to_end_scan	
0	280.0
1	49.0
2	38.0
3	928.0
4	290.0
...	...
6676	186.0
6677	181.0
6678	257.0
6679	421.0
6680	353.0

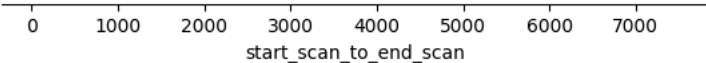
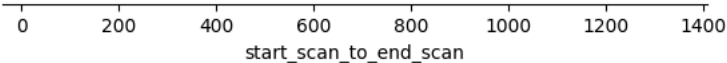
6681 rows x 1 columns

Boxplot of clipped start_scan_to_end_scan



Boxplot of filtered start_scan_to_end_scan





actual_distance_to_destination	
0	25.454847
1	9.872146
2	9.396525
3	230.413864
4	99.975594
...	...
6676	28.114090
6677	27.010925
6678	57.762333
6679	38.684837
6680	66.081528

6681 rows x 1 columns
Clipped data of actual_distance_to_destination

actual_distance_to_destination	
0	25.454847
1	9.872146
2	9.396525
3	230.413864
4	99.975594
...	...
6676	28.114090
6677	27.010925
6678	57.762333
6679	38.684837
6680	66.081528

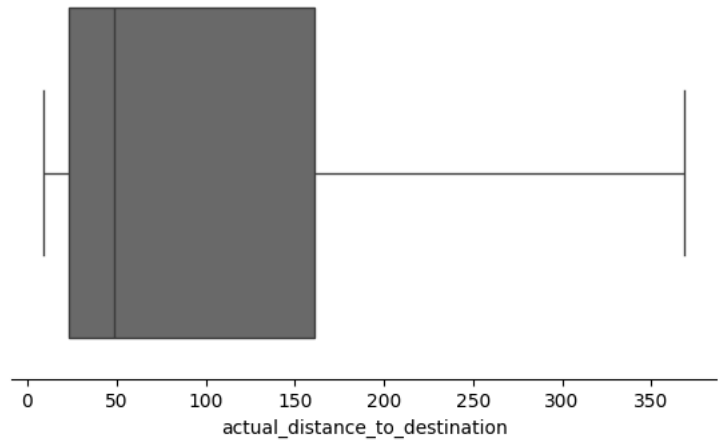
6681 rows x 1 columns
Filtered data of actual_distance_to_destination

actual_distance_to_destination	
0	25.454847
1	9.872146

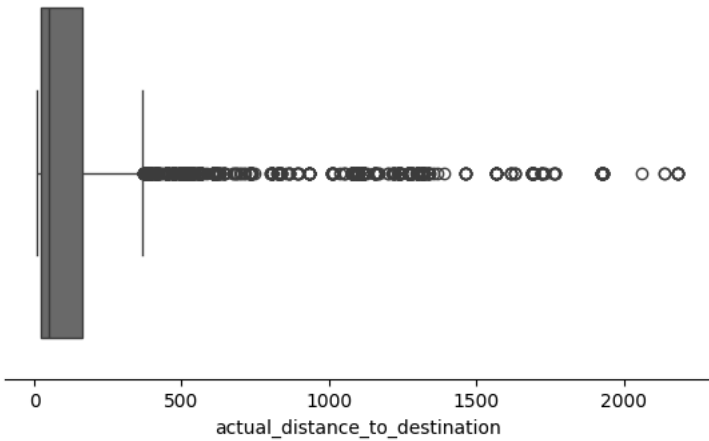
1	9.872146
2	9.396525
3	230.413864
4	99.975594
...	...
6676	28.114090
6677	27.010925
6678	57.762333
6679	38.684837
6680	66.081528

6681 rows x 1 columns

Boxplot of clipped actual_distance_to_destination



Boxplot of filtered actual_distance_to_destination



actual_time	
0	161.0
1	23.0
2	17.0
3	561.0
4	253.0
...	...
6676	86.0
6677	62.0
6678	83.0
6679	282.0

6680 275.0
6681 rows x 1 columns
Clipped data of actual_time

actual_time	
0	161.0
1	23.0
2	17.0
3	561.0
4	253.0
...	...
6676	86.0
6677	62.0
6678	83.0
6679	282.0
6680	275.0

6681 rows x 1 columns
Filtered data of actual_time

actual_time	
0	161.0
1	23.0
2	17.0
3	561.0
4	253.0
...	...
6676	86.0
6677	62.0
6678	83.0
6679	282.0
6680	275.0

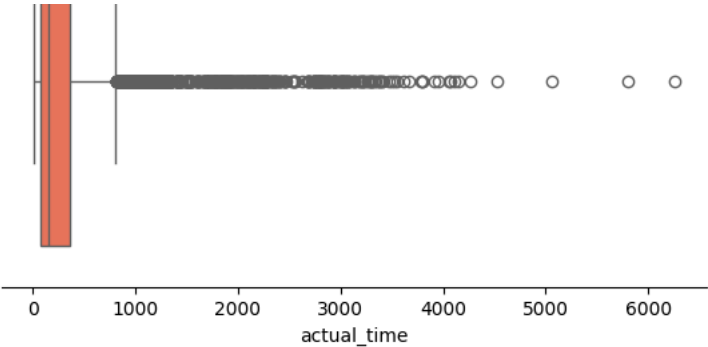
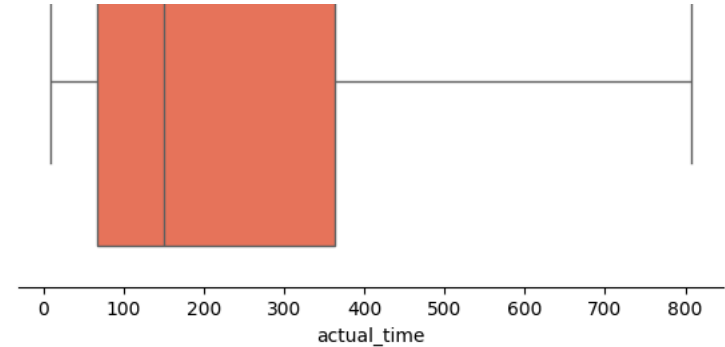
6681 rows x 1 columns

Boxplot of clipped actual_time



Boxplot of filtered actual_time





```
osrm_time
0      29.0
1       8.0
2       9.0
3     253.0
4    105.0
...      ...
6676    30.0
6677    28.0
6678    62.0
6679    48.0
6680    68.0
6681 rows x 1 columns
Clipped data of osrm_time
```

```
osrm_time
0      29.0
1       8.0
2       9.0
3     253.0
4    105.0
...      ...
6676    30.0
6677    28.0
6678    62.0
6679    48.0
6680    68.0
6681 rows x 1 columns
```

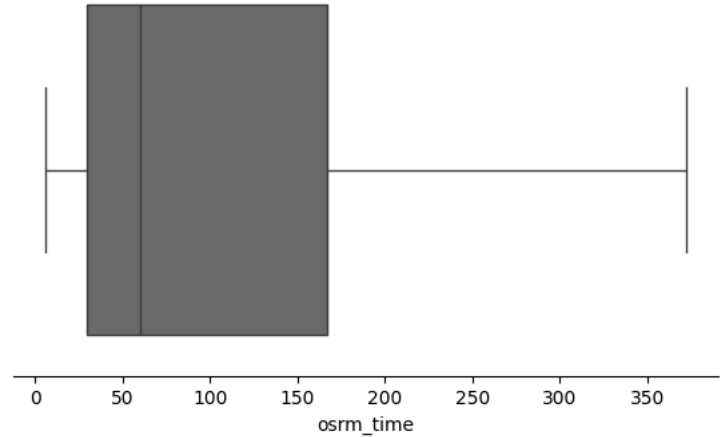
6681 rows x 1 columns

Filtered data of osrm_time

osrm_time	
0	29.0
1	8.0
2	9.0
3	253.0
4	105.0
...	...
6676	30.0
6677	28.0
6678	62.0
6679	48.0
6680	68.0

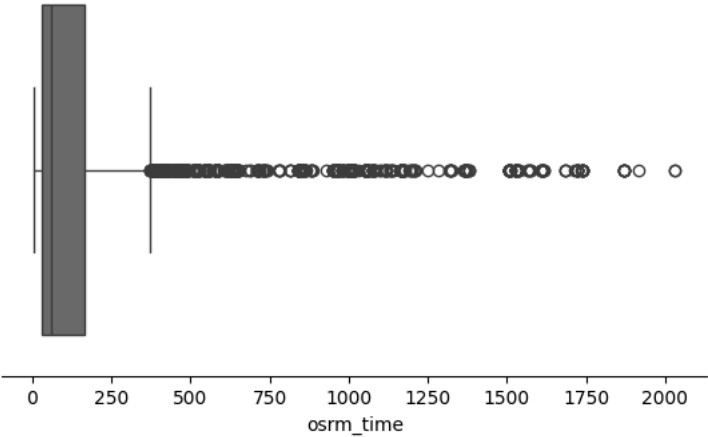
6681 rows x 1 columns

Boxplot of clipped osrm_time



osrm_distance	
0	30.935898
1	9.956600
2	10.815900
3	284.458191
4	124.506302
...	...

Boxplot of filtered osrm_time



```
6676      34.922401
6677      38.286697
6678      73.462997
6679      58.903702
6680      80.578705
```

6681 rows x 1 columns
Clipped data of osrm_distance

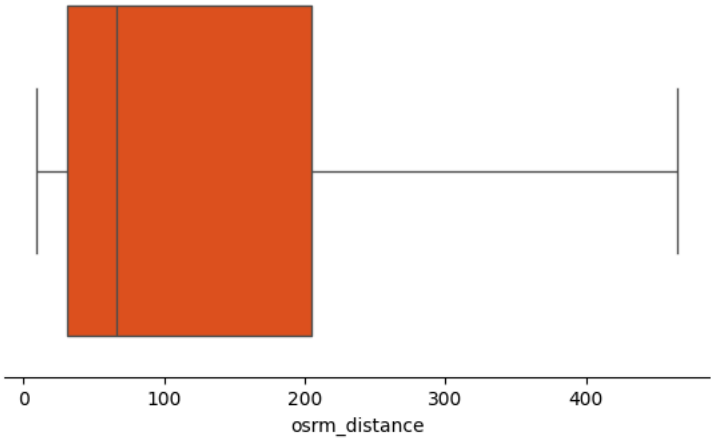
osrm_distance	
0	30.935898
1	9.956600
2	10.815900
3	284.458191
4	124.506302
...	...
6676	34.922401
6677	38.286697
6678	73.462997
6679	58.903702
6680	80.578705

6681 rows x 1 columns
Filtered data of osrm_distance

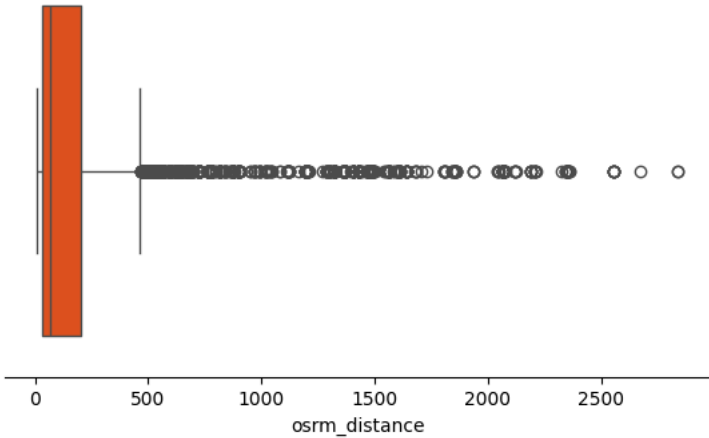
osrm_distance	
0	30.935898
1	9.956600
2	10.815900
3	284.458191
4	124.506302
...	...
6676	34.922401
6677	38.286697
6678	73.462997
6679	58.903702
6680	80.578705

6681 rows x 1 columns

Boxplot of clipped osrm_distance



Boxplot of filtered osrm_distance



segment_actual_time	
0	161.0
1	23.0
2	17.0
3	554.0
4	252.0
...	...
6676	86.0
6677	61.0
6678	82.0
6679	281.0
6680	274.0

6681 rows x 1 columns
Clipped data of segment_actual_time

segment_actual_time	
0	161.0
1	23.0
2	17.0
3	554.0
4	252.0
...	...
6676	86.0

```
6677      61.0
6678      82.0
6679     281.0
6680     274.0
```

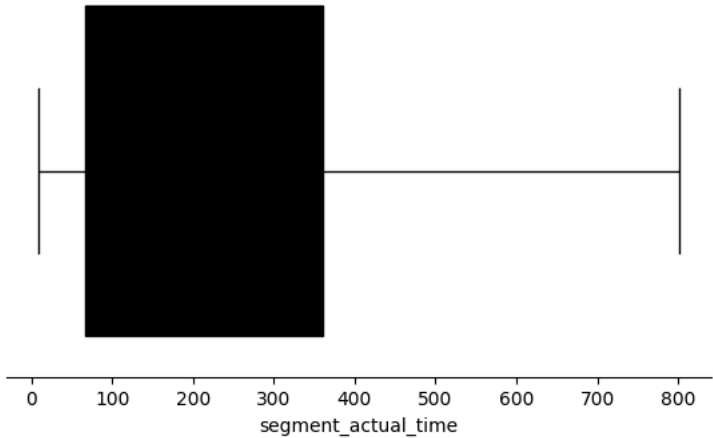
6681 rows x 1 columns

Filtered data of segment_actual_time

	segment_actual_time
0	161.0
1	23.0
2	17.0
3	554.0
4	252.0
...	...
6676	86.0
6677	61.0
6678	82.0
6679	281.0
6680	274.0

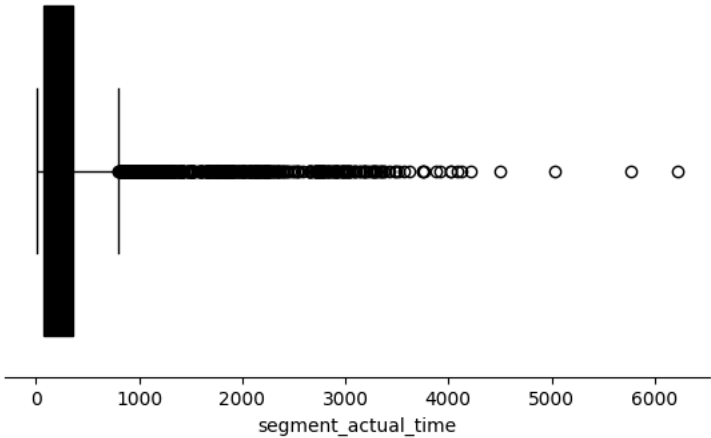
6681 rows x 1 columns

Boxplot of clipped segment_actual_time



	segment_osrm_time
0	29.0
1	14.0

Boxplot of filtered segment_actual_time



.	...
2	9.0
3	258.0
4	132.0
...	...
6676	30.0
6677	33.0
6678	62.0
6679	88.0
6680	67.0

6681 rows x 1 columns
Clipped data of segment_osrm_time

segment_osrm_time	
0	29.0
1	14.0
2	9.0
3	258.0
4	132.0
...	...
6676	30.0
6677	33.0
6678	62.0
6679	88.0
6680	67.0

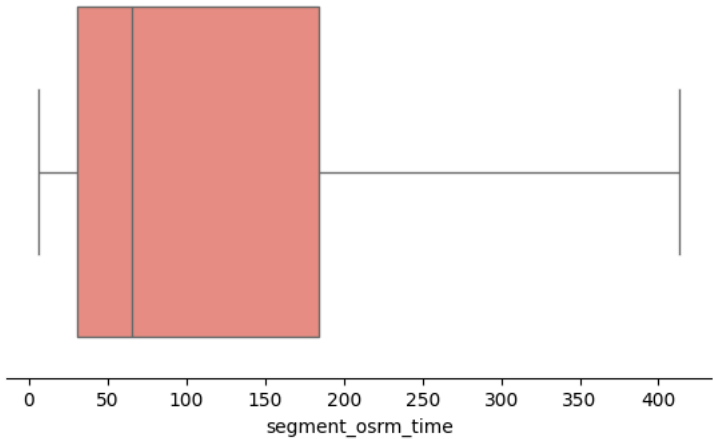
6681 rows x 1 columns
Filtered data of segment_osrm_time

segment_osrm_time	
0	29.0
1	14.0
2	9.0
3	258.0
4	132.0
...	...
6676	30.0
6677	33.0

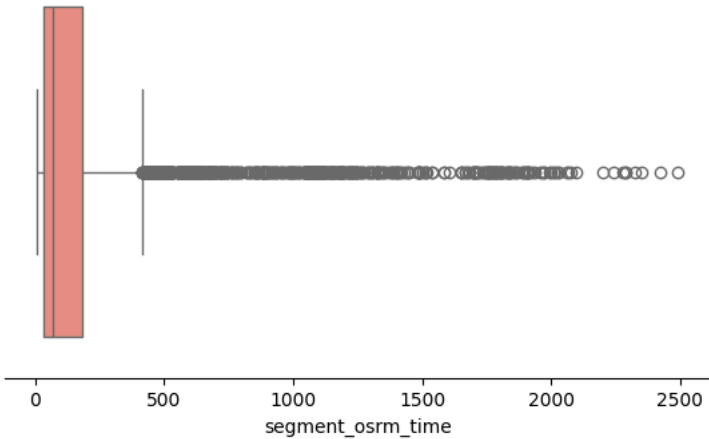
6678 62.0
6679 88.0
6680 67.0

6681 rows x 1 columns

Boxplot of clipped segment_osrm_time



Boxplot of filtered segment_osrm_time



segment_osrm_distance

0	30.935799
1	16.085999
2	10.815900
3	293.844696
4	140.562302
...	...
6676	34.922298
6677	33.639999
6678	64.855103
6679	104.886597
6680	80.578705

6681 rows x 1 columns

Clipped data of segment_osrm_distance

segment_osrm_distance

0	30.935799
1	16.085999
2	10.815900

3	293.844696
4	140.562302
...	...
6676	34.922298
6677	33.639999
6678	64.855103
6679	104.886597
6680	80.578705

6681 rows x 1 columns

Filtered data of segment_osrm_distance

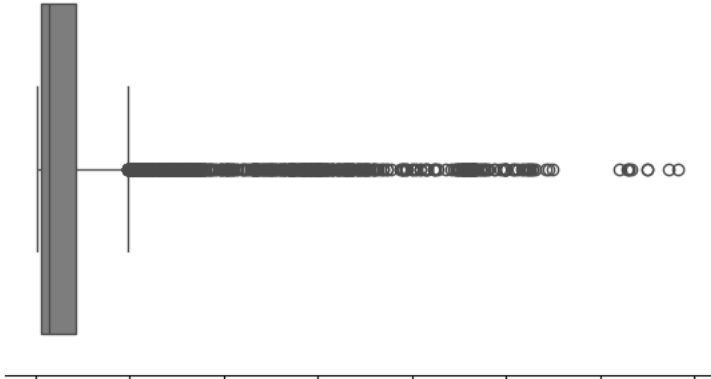
segment_osrm_distance	
0	30.935799
1	16.085999
2	10.815900
3	293.844696
4	140.562302
...	...
6676	34.922298
6677	33.639999
6678	64.855103
6679	104.886597
6680	80.578705

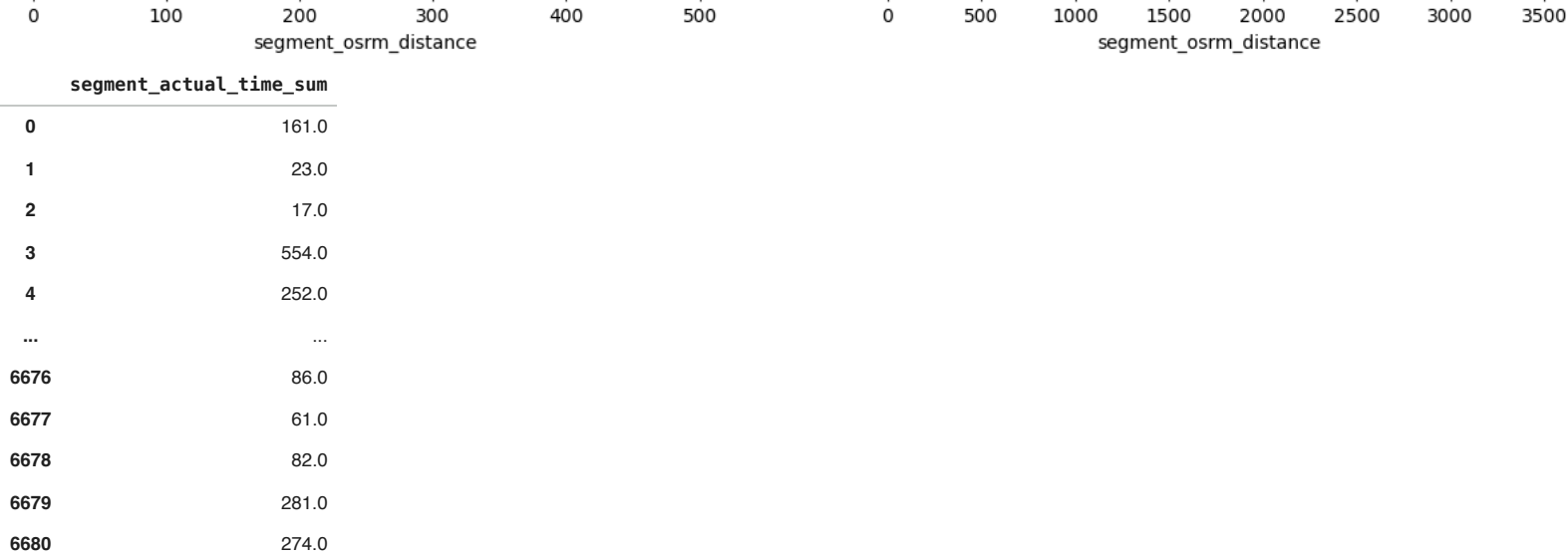
6681 rows x 1 columns

Boxplot of clipped segment_osrm_distance



Boxplot of filtered segment_osrm_distance





6681 rows x 1 columns
Clipped data of segment_actual_time_sum

segment_actual_time_sum	
0	161.0
1	23.0
2	17.0
3	554.0
4	252.0
...	...
6676	86.0
6677	61.0
6678	82.0
6679	281.0
6680	274.0

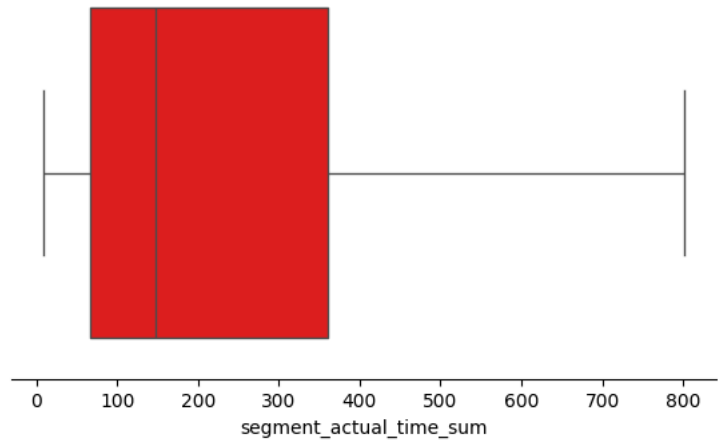
6681 rows x 1 columns
Filtered data of segment_actual_time_sum

segment_actual_time_sum	
0	161.0
1	23.0
2	17.0
3	554.0

4	252.0
...	...
6676	86.0
6677	61.0
6678	82.0
6679	281.0
6680	274.0

6681 rows x 1 columns

Boxplot of clipped segment_actual_time_sum

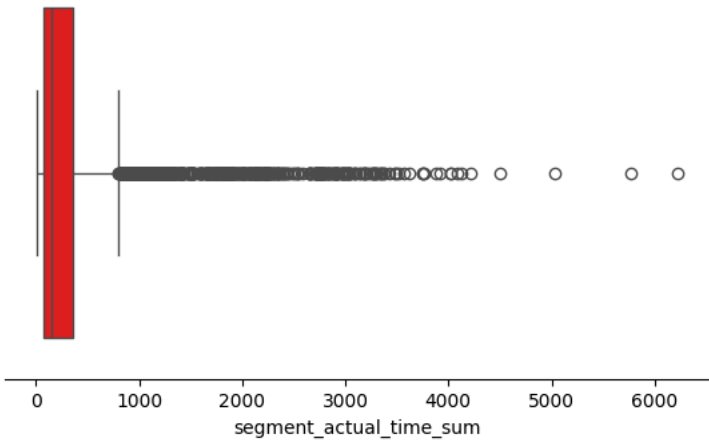


segment_osrm_time_sum	
0	29.0
1	14.0
2	9.0
3	258.0
4	132.0
...	...
6676	30.0
6677	33.0
6678	62.0
6679	88.0
6680	67.0

6681 rows x 1 columns

Clipped data of segment_osrm_time_sum

Boxplot of filtered segment_actual_time_sum



segment_osrm_time_sum	
0	29.0
1	14.0
2	9.0
3	258.0
4	132.0
...	...
6676	30.0
6677	33.0
6678	62.0
6679	88.0
6680	67.0

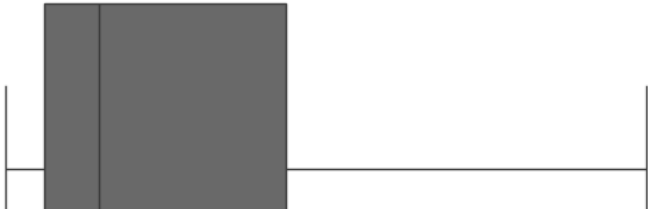
6681 rows x 1 columns

Filtered data of segment_osrm_time_sum

segment_osrm_time_sum	
0	29.0
1	14.0
2	9.0
3	258.0
4	132.0
...	...
6676	30.0
6677	33.0
6678	62.0
6679	88.0
6680	67.0

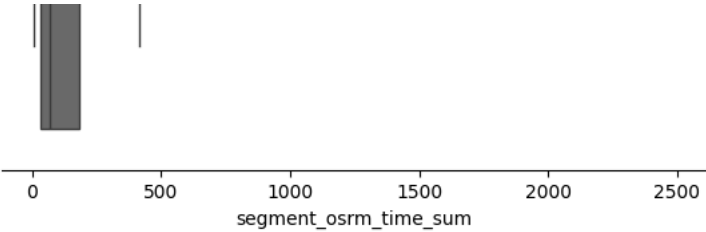
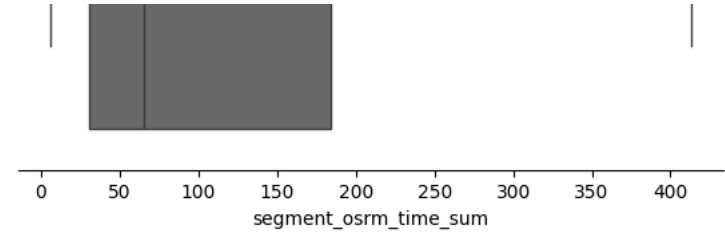
6681 rows x 1 columns

Boxplot of clipped segment_osrm_time_sum



Boxplot of filtered segment_osrm_time_sum





segment_osrm_distance_sum

0	30.935799
1	16.085999
2	10.815900
3	293.844696
4	140.562302
...	...
6676	34.922298
6677	33.639999
6678	64.855103
6679	104.886597
6680	80.578705

6681 rows x 1 columns

Clipped data of segment_osrm_distance_sum

segment_osrm_distance_sum

0	30.935799
1	16.085999
2	10.815900
3	293.844696
4	140.562302
...	...
6676	34.922298
6677	33.639999
6678	64.855103
6679	104.886597
6680	80.578705

6681 rows x 1 columns

Filtered data of segment_osrm_distance_sum

segment_osrm_distance_sum

- 030.935799
- 16.085999
- 210.815900
- 3299.844696
- 410.569309
- 5104.886507
- 667864.855103
- Understanding:
- Here we see that the data after removing outliers has outliers. It has to be understood that q1 and q3 dont have to be always 25th percentile and 75th percentile. Try changing q1 and q3 to 10th percentile to 90th percentile and plot and see...

Clipped data replaces the outlier values with specified values.

Here, I have proceeded with both clipped and filtered data(with reduced outliers) for further analysis.

```
num_df = numerical_columns.copy()
num_df
```

	od_time_diff_hour	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_d
0	4.680733	280.0	25.454847	161.0	29.0	30.935898	161.0	29.0	3
1	0.822223	49.0	9.872146	23.0	8.0	9.956600	23.0	14.0	1
2	0.641675	38.0	9.396525	17.0	9.0	10.815900	17.0	9.0	1
3	15.511249	928.0	230.413864	561.0	253.0	284.458191	554.0	258.0	29
4	4.844371	290.0	99.975594	253.0	105.0	124.506302	252.0	132.0	14
...	...	...	...	...	...	...	...	...	...
6676	3.126754	186.0	28.114090	86.0	30.0	34.922401	86.0	30.0	3
6677	3.040278	181.0	27.010925	62.0	28.0	38.286697	61.0	33.0	3
6678	4.300482	257.0	57.762333	83.0	62.0	73.462997	82.0	62.0	6
6679	7.035331	421.0	38.684837	282.0	48.0	58.903702	281.0	88.0	10
6680	5.906793	353.0	66.081528	275.0	68.0	80.578705	274.0	67.0	8
6681 rows x 12 columns									



```
num_cols
```

```
['od_time_diff_hour',
 'start_scan_to_end_scan',
 'actual_distance_to_destination',
 'actual_time',
 'osrm_time',
 'osrm_distance',
 'segment_actual_time',
 'segment_osrm_time',
 'segment_osrm_distance',
 'segment_actual_time_sum',
 'segment_osrm_time_sum',
 'segment_osrm_distance_sum']
```

```
Q1 = np.percentile(num_df[num_cols], 25)
Q3 = np.percentile(num_df[num_cols], 75)
IQR = Q3 - Q1

lower_bound = Q1 - (1.5 * IQR)
upper_bound = Q3 + (1.5 * IQR)

clipped_num_df = np.clip(num_df[num_cols], lower_bound, upper_bound)
display(clipped_num_df)

filtered_num_df = num_df[num_cols][(num_df[num_cols] >= lower_bound) | (num_df[num_cols] <= upper_bound)]
display(filtered_num_df)
```

	od_time_diff_hour	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_d
0	4.680733	280.000000	25.454847	161.000000	29.0	30.935898	161.000000	29.0	3
1	0.822223	49.000000	9.872146	23.000000	8.0	9.956600	23.000000	14.0	1
2	0.641675	38.000000	9.396525	17.000000	9.0	10.815900	17.000000	9.0	1
3	15.511243	543.225452	230.413864	543.225452	253.0	284.458191	543.225452	258.0	29
4	4.844371	290.000000	99.975594	253.000000	105.0	124.506302	252.000000	132.0	14
...	...	...	...	...	...	...	...	...	...
6676	3.126754	186.000000	28.114090	86.000000	30.0	34.922401	86.000000	30.0	3
6677	3.040278	181.000000	27.010925	62.000000	28.0	38.286697	61.000000	33.0	3
6678	4.300482	257.000000	57.762333	83.000000	62.0	73.462997	82.000000	62.0	6
6679	7.035331	421.000000	38.684837	282.000000	48.0	58.903702	281.000000	88.0	10
6680	5.906793	353.000000	66.081528	275.000000	68.0	80.578705	274.000000	67.0	8

6681 rows x 12 columns

	od_time_diff_hour	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_d
0	4.680733	280.0	25.454847	161.0	29.0	30.935898	161.0	29.0	3
1	0.822223	49.0	9.872146	23.0	8.0	9.956600	23.0	14.0	1
2	0.641675	38.0	9.396525	17.0	9.0	10.815900	17.0	9.0	1
3	15.511243	928.0	230.413864	561.0	253.0	284.458191	554.0	258.0	29
4	4.844371	290.0	99.975594	253.0	105.0	124.506302	252.0	132.0	14
...	...	...	...	...	...	...	...	...	...
6676	3.126754	186.0	28.114090	86.0	30.0	34.922401	86.0	30.0	3
6677	3.040278	181.0	27.010925	62.0	28.0	38.286697	61.0	33.0	3
6678	4.300482	257.0	57.762333	83.0	62.0	73.462997	82.0	62.0	6
6679	7.035331	421.0	38.684837	282.0	48.0	58.903702	281.0	88.0	10
6680	5.906793	353.0	66.081528	275.0	68.0	80.578705	274.0	67.0	8

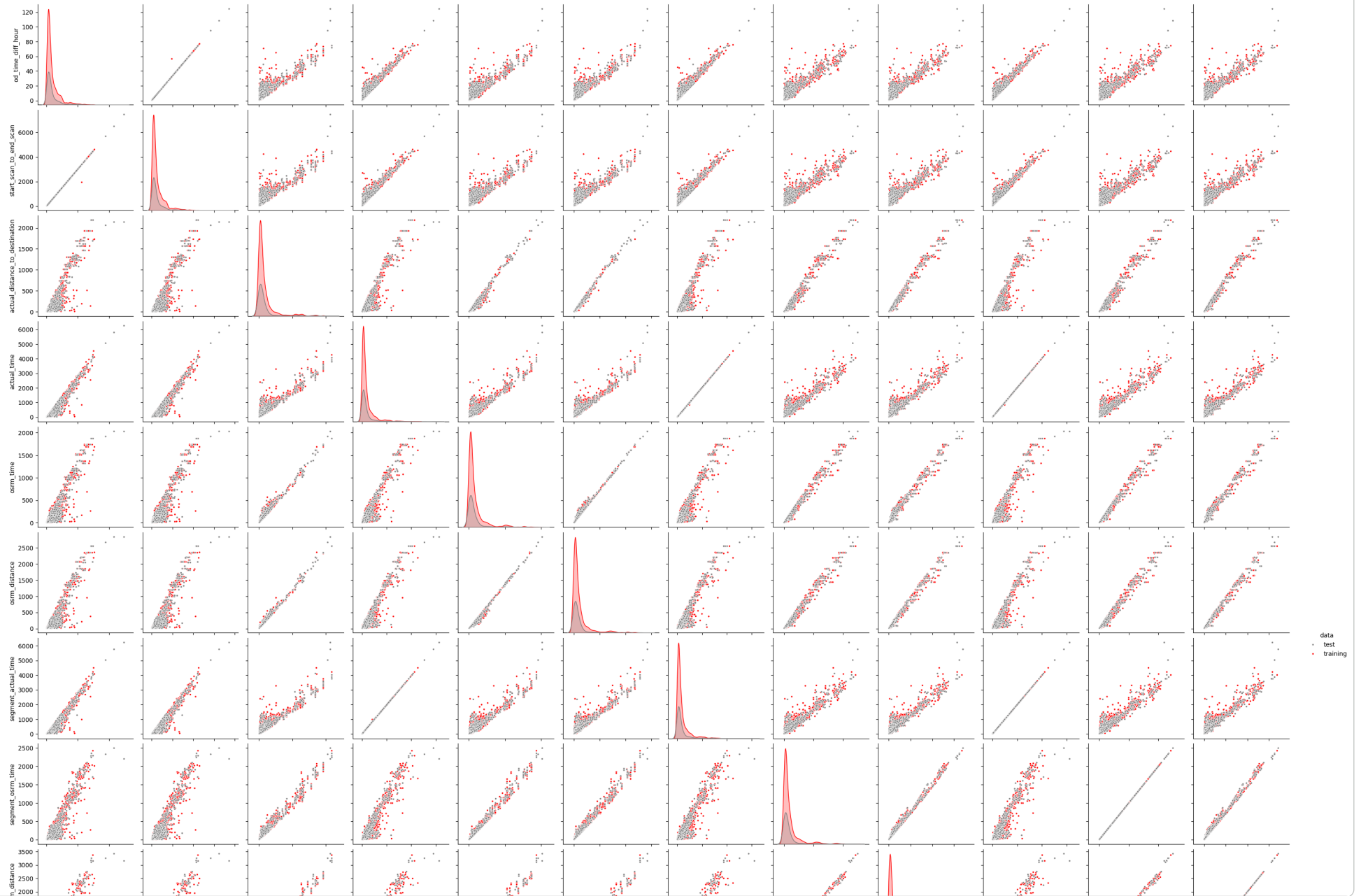
6681 rows x 12 columns

Next steps: [New interactive sheet](#) [New interactive sheet](#)

```
plt.figure(figsize=(14,0.05))
plt.axis('off')
plt.title(f' Pairplot Analysis',fontfamily='serif',fontweight='bold',fontsize=15,backgroundcolor='crimson',color='w')
sns.pairplot(data = trip_df,vars = num_cols,hue='data',markers = '.',palette=cp)
plt.show()
```

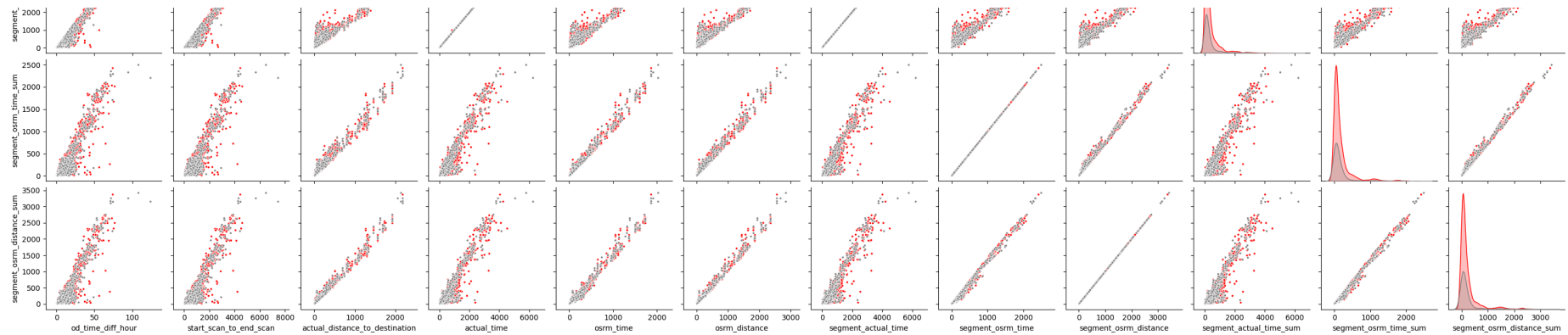


Pairplot Analysis



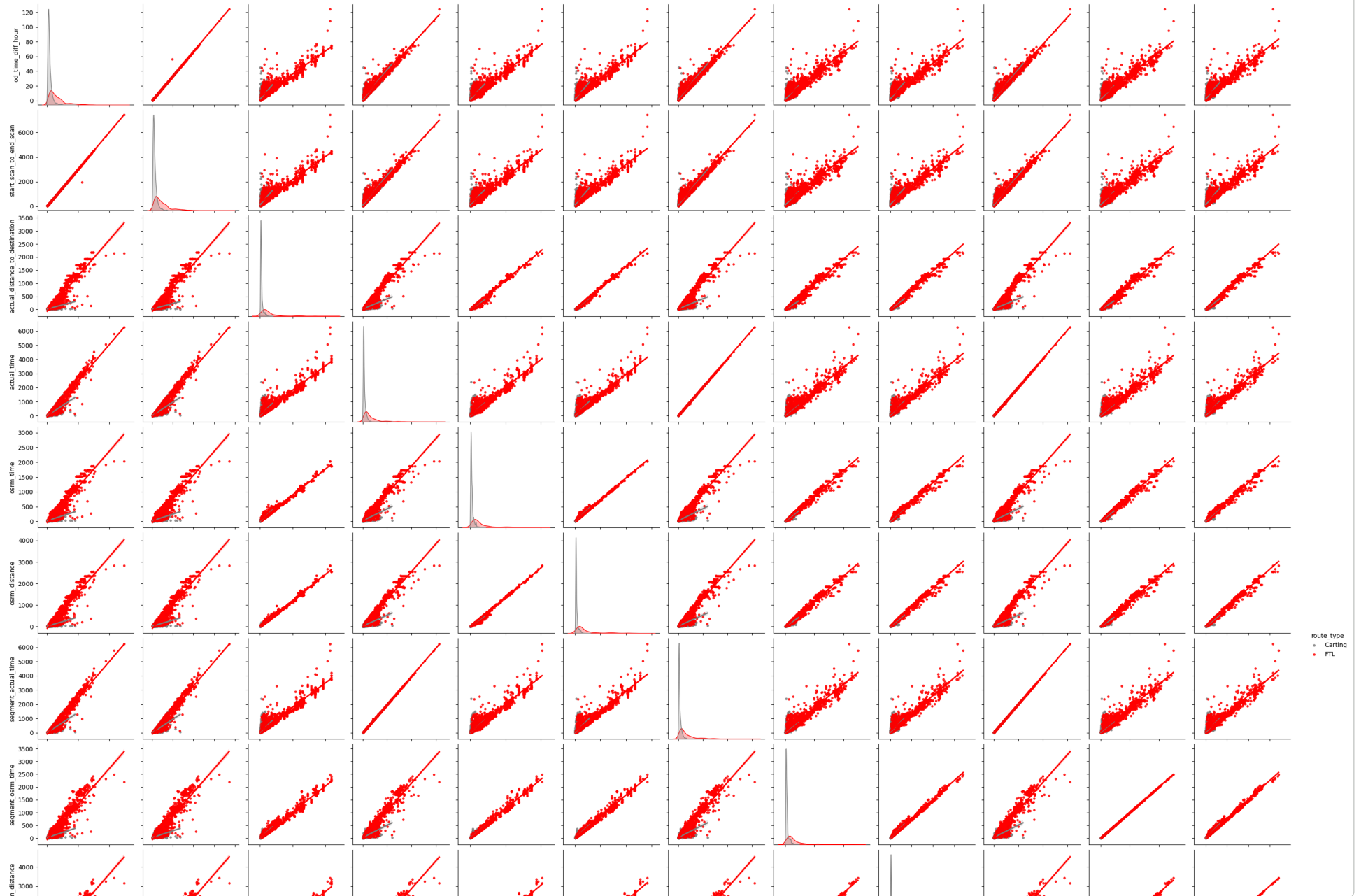
```
plt.figure(figsize=(14,0.05))
plt.axis('off')
```

```
plt.title(f' Pairplot Analysis',fontfamily='serif',fontweight='bold',fontsize=15,backgroundcolor='dimgrey',color='w')
sns.pairplot(data = trip_df,vars=num_cols,kind = 'reg',hue='route_type',markers = '.',palette=cp)
plt.show()
```



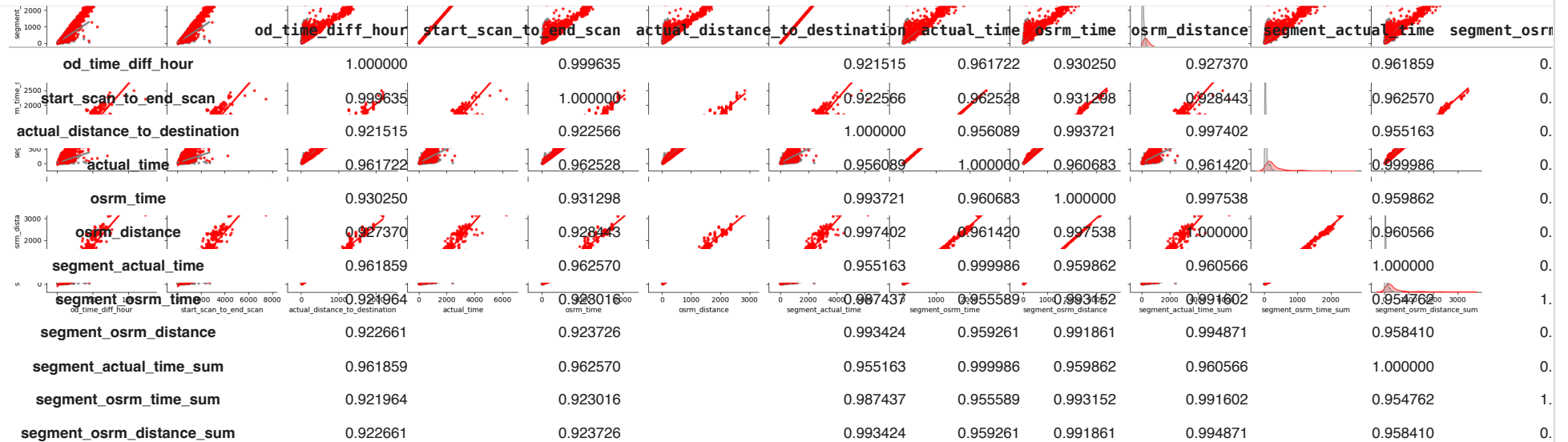


Pairplot Analysis



```
clipped_df_corr = clipped_num_df.corr()
clipped_df_corr
```

```
filtered_df_corr = filtered_num_df.corr()
filtered_df_corr
```



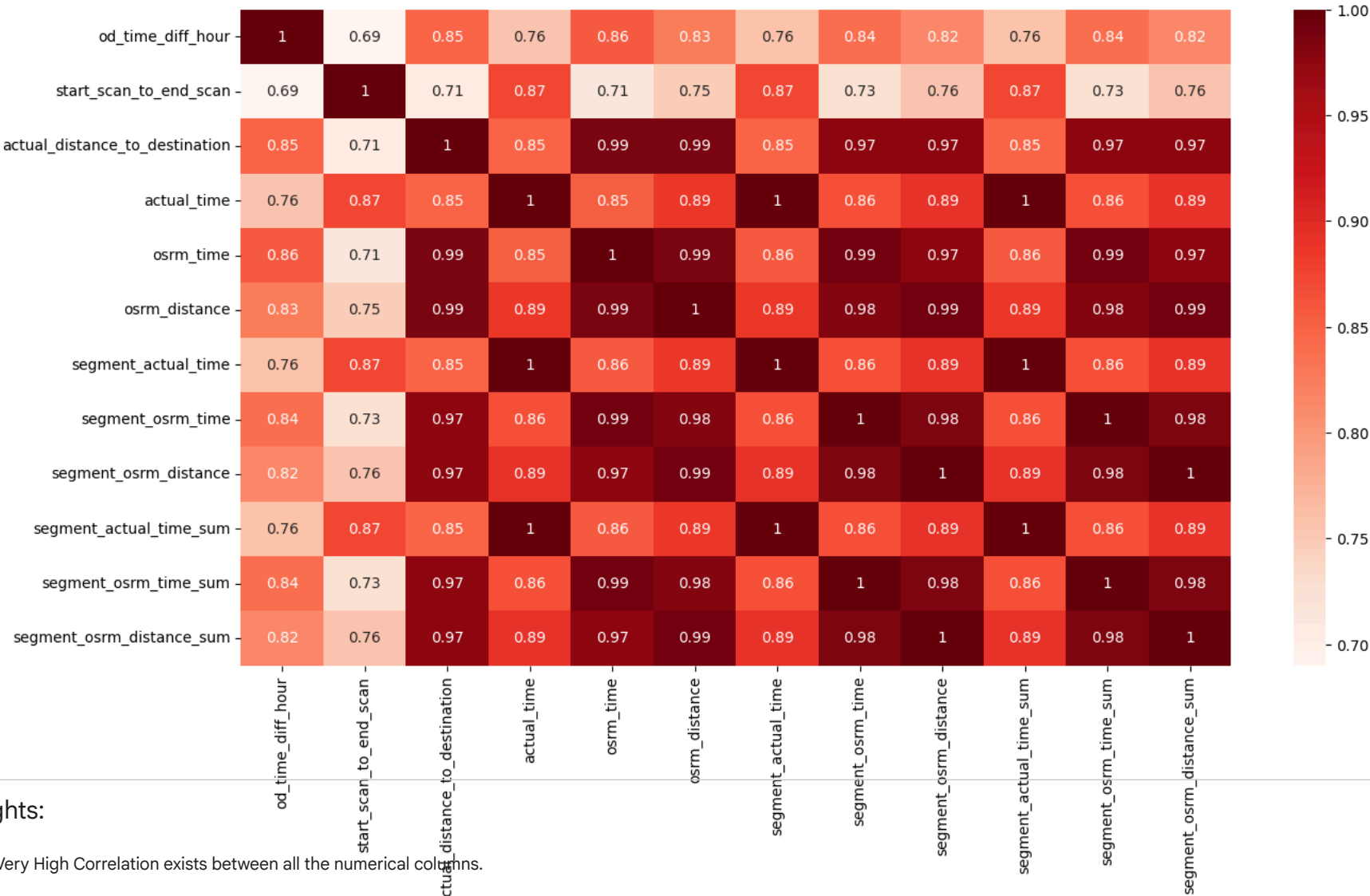
Next steps: [New interactive sheet](#)

```
plt.figure(figsize = (15,8))
plt.suptitle(f'Correlation Analysis- clipped_df',fontfamily='serif',fontweight='bold',fontsize=15,backgroundcolor='k',color='w')
sns.heatmap(data = clipped_df_corr,vmin=0.69, annot = True, cmap='Reds')
plt.show()

plt.figure(figsize = (15,8))
plt.suptitle(f'Correlation Analysis - filtered_df',fontfamily='serif',fontweight='bold',fontsize=15,backgroundcolor='r',color='w')
sns.heatmap(data = filtered_df_corr,vmin=0.85, annot = True, cmap='Greys')
plt.show()
```



Correlation Analysis- clipped_df

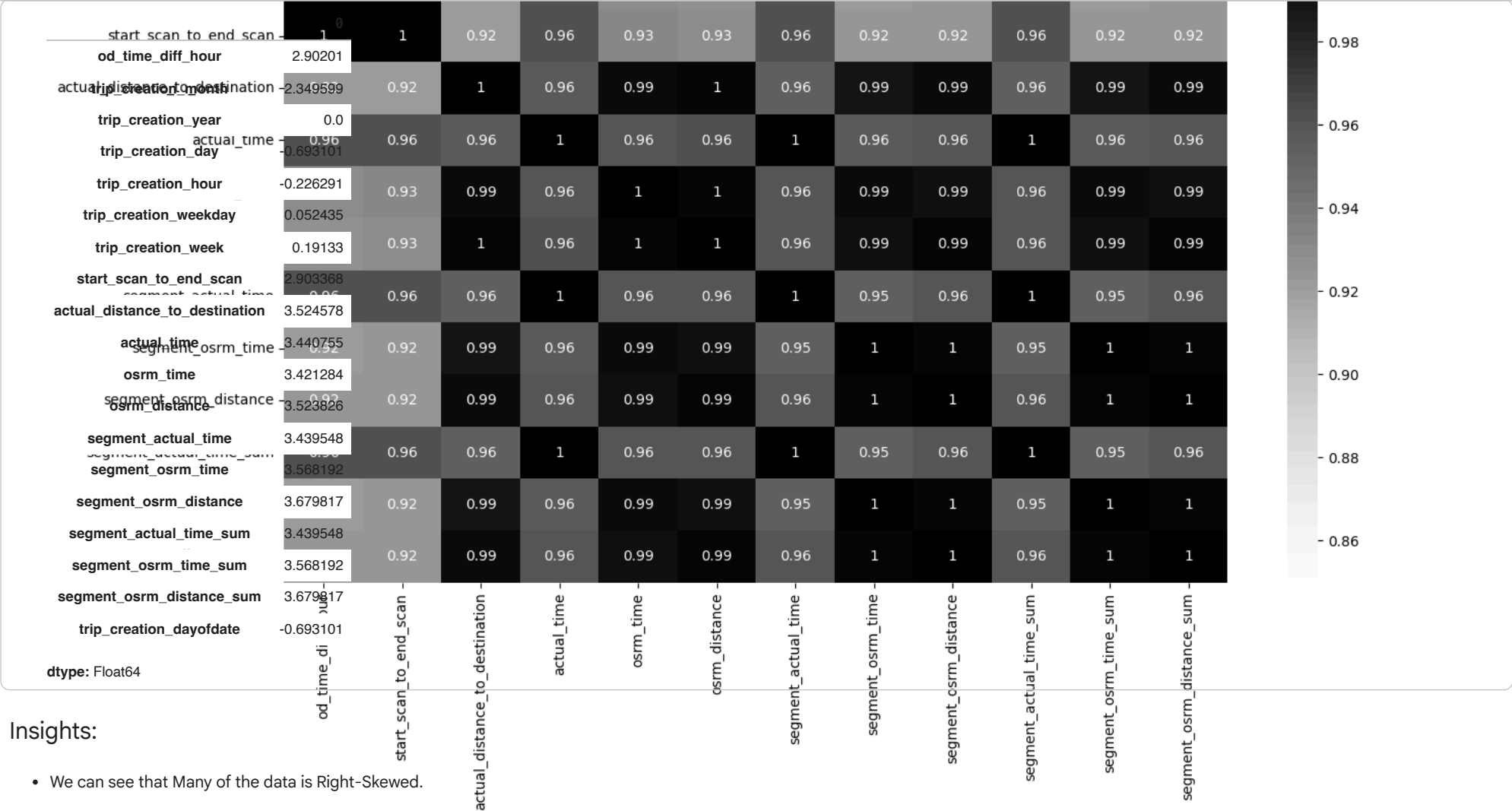


Insights:

- Very High Correlation exists between all the numerical columns.

```
trip_df.skew(numeric_only = True)
```





Insights:

- We can see that Many of the data is Right-Skewed.

One-hot encoding

```
trip_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6681 entries, 0 to 6680
Data columns (total 37 columns):
#   Column                Non-Null Count  Dtype
---  -
0   trip_uuid              6681 non-null   object
1   data                   6681 non-null   category
2   route_type             6681 non-null   category
```

```

3  od_start_time          6681 non-null  datetime64[ns]
4  od_end_time            6681 non-null  datetime64[ns]
5  od_time_diff_hour      6681 non-null  float64
6  trip_creation_time     6681 non-null  datetime64[ns]
7  trip_creation_month    6681 non-null  int32
8  trip_creation_year     6681 non-null  int32
9  trip_creation_day      6681 non-null  int32
10 trip_creation_hour     6681 non-null  int32
11 trip_creation_weekday  6681 non-null  int32
12 trip_creation_week    6681 non-null  UInt32
13 start_scan_to_end_scan 6681 non-null  float32
14 actual_distance_to_destination 6681 non-null  float32
15 actual_time           6681 non-null  float32
16 osrm_time            6681 non-null  float32
17 osrm_distance        6681 non-null  float32
18 segment_actual_time   6681 non-null  float32
19 segment_osrm_time     6681 non-null  float32
20 segment_osrm_distance 6681 non-null  float32
21 segment_actual_time_sum 6681 non-null  float32
22 segment_osrm_time_sum 6681 non-null  float32
23 segment_osrm_distance_sum 6681 non-null  float32
24 source_name           6681 non-null  object
25 source_city           6681 non-null  object
26 source_state          6681 non-null  object
27 source_place          6681 non-null  object
28 destination_name      6681 non-null  object
29 destination_city      6681 non-null  object
30 destination_state     6681 non-null  object
31 destination_place     6681 non-null  object
32 corridor             6681 non-null  object
33 state_corridor        6681 non-null  object
34 city_corridor         6681 non-null  object
35 trip_creation_day_week 6681 non-null  object
36 trip_creation_dayofdate 6681 non-null  int32
dtypes: UInt32(1), category(2), datetime64[ns](3), float32(11), float64(1), int32(6), object(13)
memory usage: 1.3+ MB

```

```
categorical_cols = ['data', 'route_type']
```

```

# one hot encoding the categorical features
ohe = OneHotEncoder(sparse_output=False)
encoded_cat_cols = ohe.fit_transform(trip_df[categorical_cols])

categorical_encoded_df = pd.DataFrame(encoded_cat_cols, columns=ohe.get_feature_names_out(categorical_cols))
display(categorical_encoded_df)

encoded_df = pd.concat([trip_df, categorical_encoded_df], axis=1)
encoded_df

```



	data_test	data_training	route_type_Carting	route_type_FTL
0	0.0	1.0	1.0	0.0
1	0.0	1.0	1.0	0.0
2	0.0	1.0	1.0	0.0
3	0.0	1.0	1.0	0.0
4	0.0	1.0	0.0	1.0
...	...	...	...	...
6676	1.0	0.0	1.0	0.0
6677	0.0	0.0	1.0	0.0
6678	1.0	0.0	1.0	0.0
6679	1.0	0.0	1.0	0.0

0-1 Minmax scaler

Next steps: [New interactive sheet](#)

- Most appropriate since the data is not gaussian

```
# Normalizing/Standardizing the numerical features using MinMaxScaler
min_max_scaler = MinMaxScaler()
min_max_scaled_numerical = min_max_scaler.fit_transform(trip_df[num_cols])

# Converting the scaled features back to a dataframe
min_max_scaled_df = pd.DataFrame(min_max_scaled_numerical, columns=num_cols)
min_max_scaled_df
```

	od_time_diff_hour	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_d							
0	0.034052	0.034047	0.007559	0.024297	0.011352	0.007724	0.024433	0.009244								
1	0.002898	0.002961	0.000399	0.002238	0.000987	0.000312	0.002250	0.003215								
2	0.001440	0.001480	0.000181	0.001279	0.001481	0.000616	0.001286	0.001206								
3	0.121499	0.121249	0.101729	0.088235	0.121915	0.097289	0.087606	0.101286								
4	0.035373	0.035392	0.041798	0.039003	0.048865	0.040781	0.039061	0.050643								
3	153671127071609610	training	Carting	2018-09-12	2018-09-12	...	15.511243	...	00:14:30.716368	07:54:13.838441	...	9	...	2018	...	12
6676	0.021505	0.021397	0.008780	0.012308	0.011846	0.009132	0.012377	0.009646								
6677	0.020807	0.020724	0.008274	0.008472	0.010859	0.010321	0.008359	0.010852								
6678	0.030982	0.030951	0.022403	0.011829	0.027641	0.022748	0.011734	0.022508								
6679	0.053063	0.053021	0.013637	0.043638	0.020731	0.017604	0.043723	0.032958								
6680	0.043951	0.043870	0.026225	0.042519	0.030602	0.025262	0.042598	0.024518								
6681	153861089559302126	test	Carting	2018-10-03	2018-10-04	3.126754	2018-10-03	23:54:55.593290	03:46:15.195842	10	2018	3				

Next steps: [New interactive sheet](#)

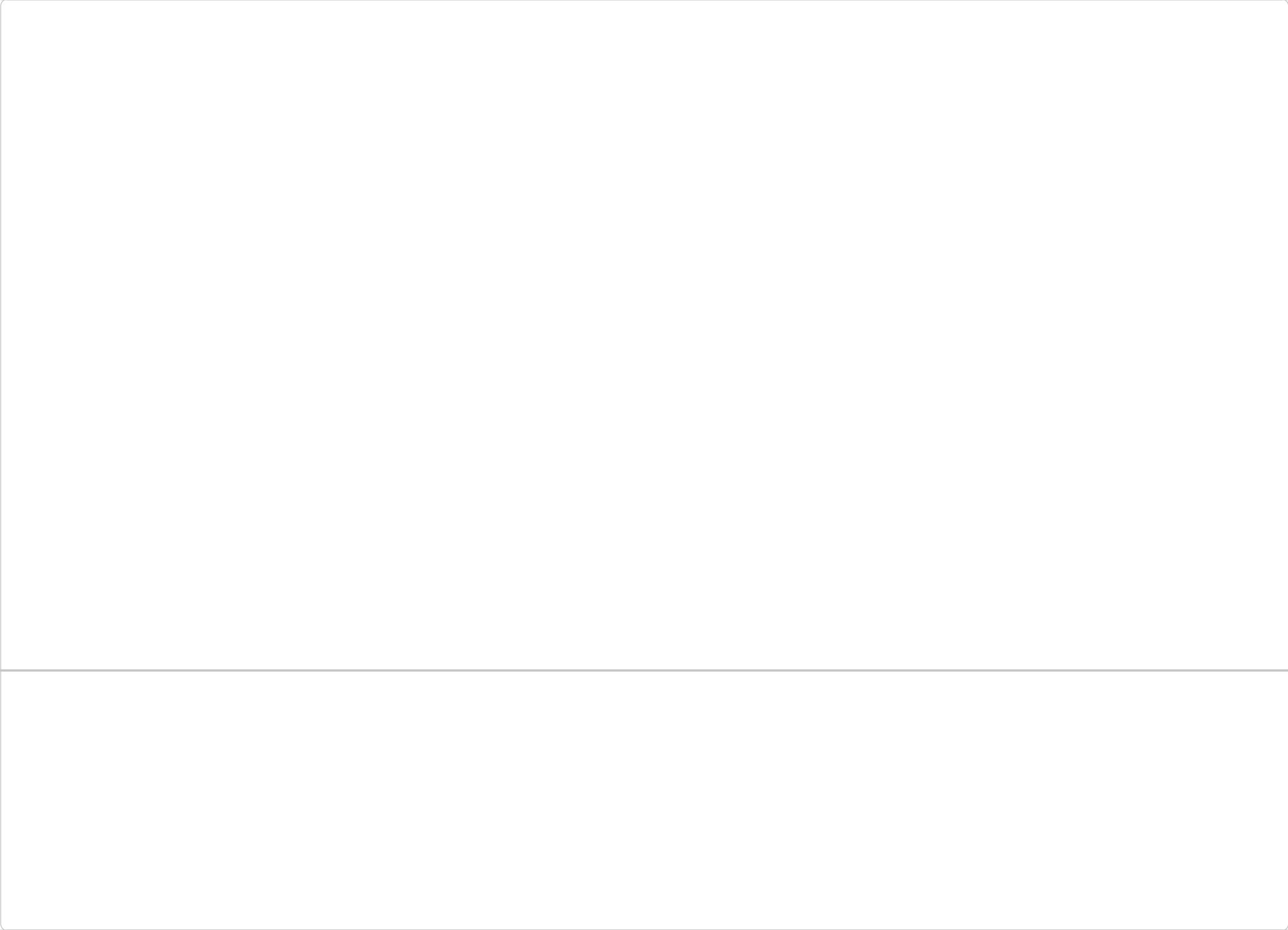
6677	153861089872028474	test	Carting	2018-10-03	2018-10-04	3.040278	2018-10-03	23:54:58.720536	02:57:23.720337	10	2018	3
------	--------------------	------	---------	------------	------------	----------	------------	-----------------	-----------------	----	------	---

plt.figure(figsize=(14,0.05))
plt.axis('off')

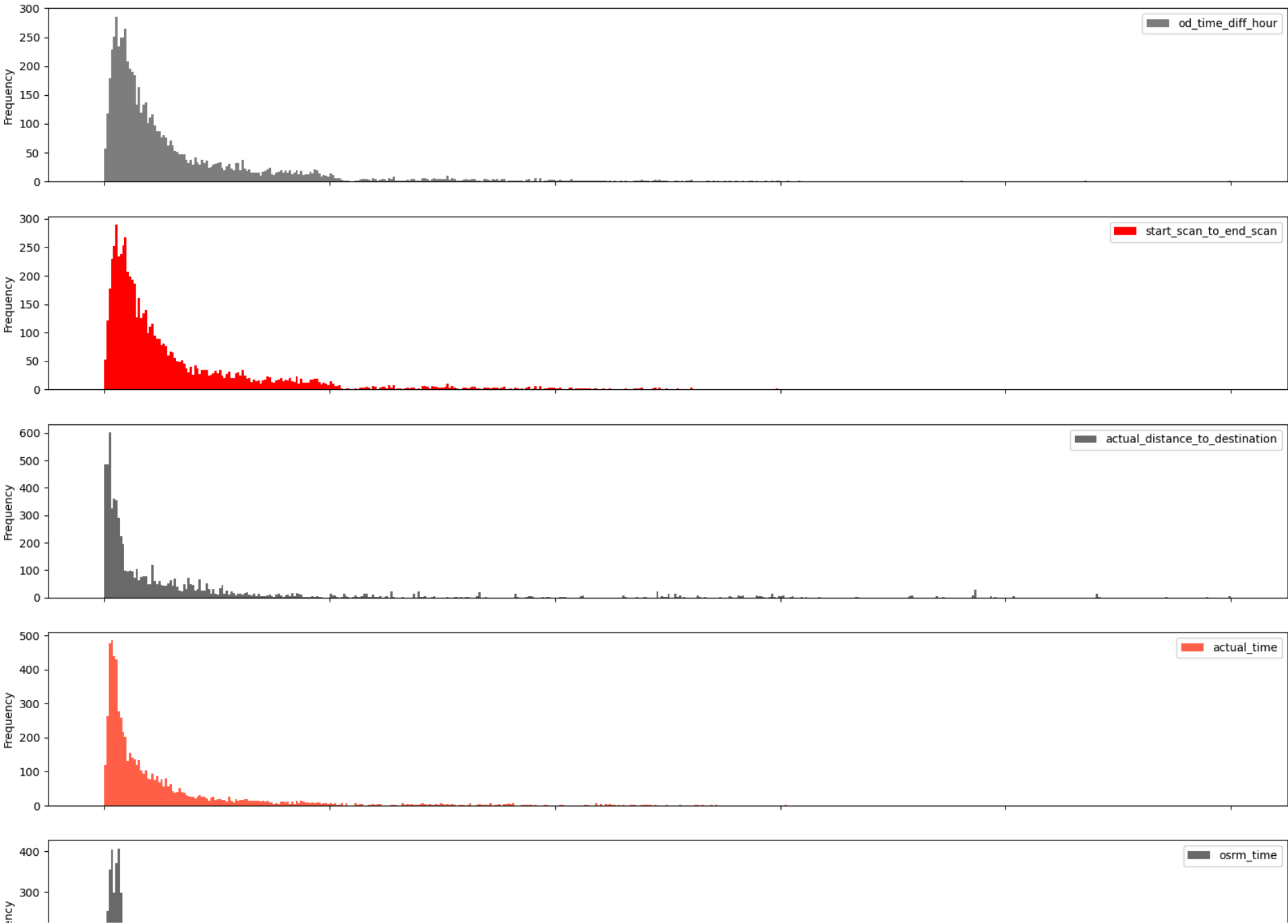
```
plt.suptitle(f'Min-Max scaled visualization of num_cols',fontfamily='serif',fontweight='bold',fontsize=15,backgroundcolor='k',color='w')
min_max_scaled_df.plot(kind='hist', figsize=(20,40),subplots=True,color=cp,bins=500)
plt.show()
```

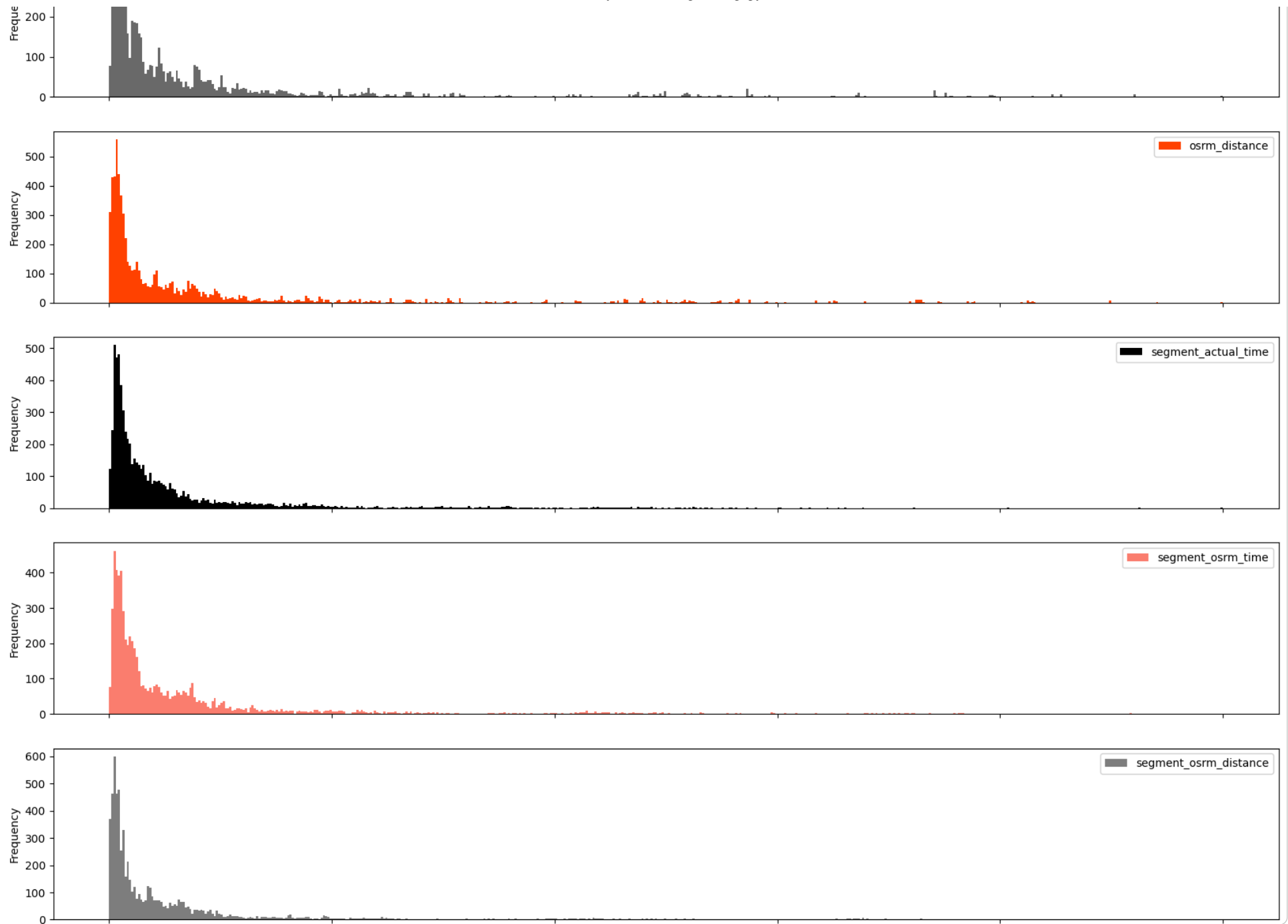
6679	153861106442901555	trip-	test	Carting	2018-10-04	2018-10-04	02:51:27.075797	02:51:27.075797	7.035331	2018-10-03	23:57:44.429324	10	2018	3
6680	153861118270144424	trip-	test	FTL	2018-10-04	2018-10-04	03:58:40.726547	03:58:40.726547	5.906793	2018-10-03	23:59:42.701692	10	2018	3

6681 rows x 41 columns



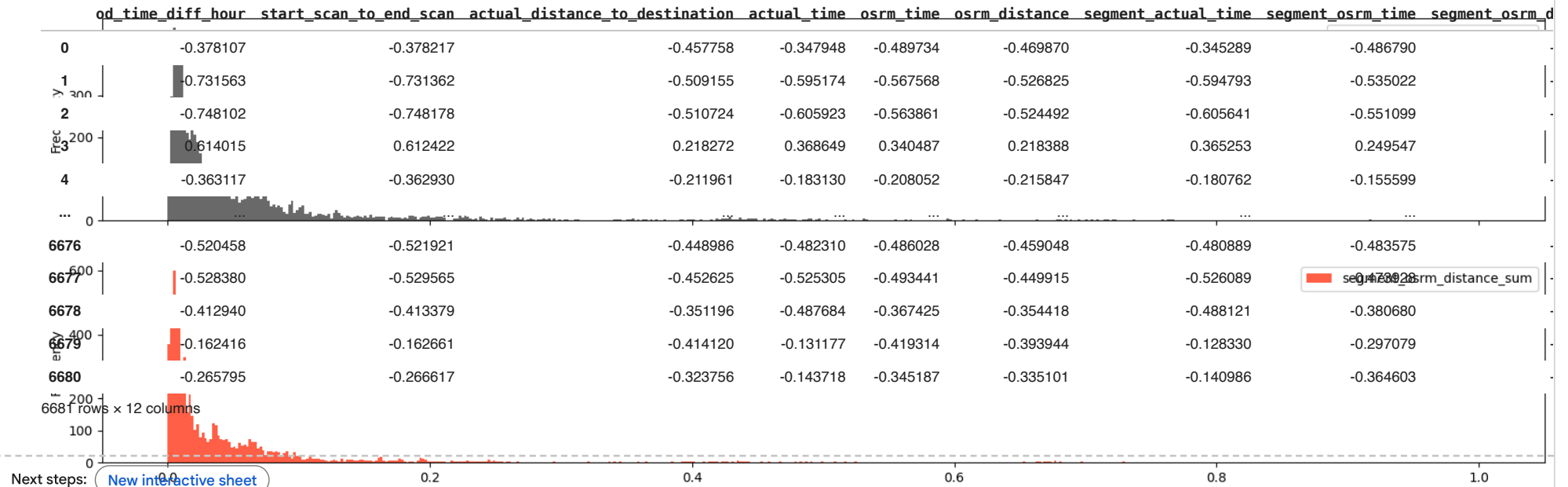
Min-Max scaled visualization of num_cols





```
# Just so to know ... cant do this as `data is not gaussian`
# Standardization works only with data which follows normal distribution
# Standardizing the numerical features using StandardScaler
std_scaler = StandardScaler()
std_scaled = std_scaler.fit_transform(trip_df[num_cols])

# Converting the scaled features back to a dataframe
std_scaled_df = pd.DataFrame(std_scaled, columns=num_cols)
std_scaled_df
```



```
plt.figure(figsize=(14,0.05))
plt.axis('off')
plt.suptitle(f'Standardized Num_cols scaled visualization',fontfamily='serif',fontweight='bold',fontsize=15,backgroundcolor='r',color='w')
std_scaled_df.plot(kind='hist', figsize=(20,30),subplots=True,color=cp,bins=500)
plt.show()
```



Standardized Num_cols scaled visualization

